1. Linear and Logistic Regression with Error Estimates

a) Linear Regression

Linear Regression model establish a linear relationship between the input variables(***X***) and single output variable(***Y***). When the input(***X***) is a single variable this model is called **Simple Linear Regression** and when there are multiple input variables(***X***), it is called **Multiple Linear Regression**.

Simple Linear Regression Model Representation

From the Dataset we have an input variable - ***X*** and one output variable - ***Y***. And we want to build linear relationship between these variables.

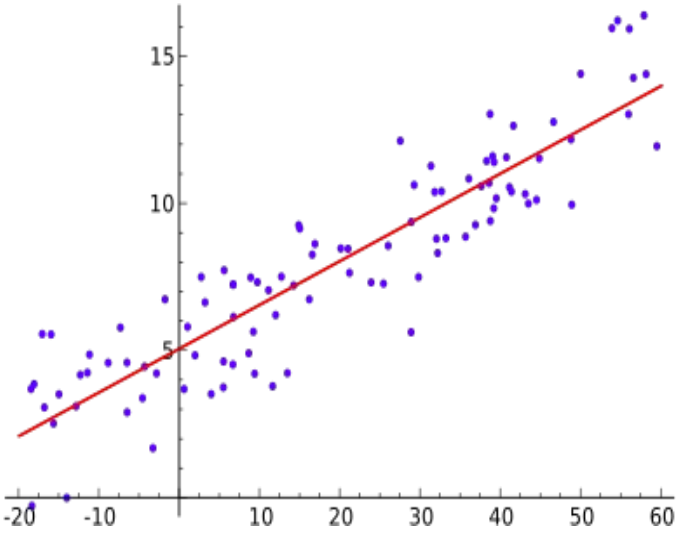
$$Y = \beta_0 + \beta_1 X$$

The β_1 is called a scale factor or **coefficient** and β_0 is called **bias coefficient**. The bias coefficient gives an extra degree of freedom to this model. This equation is similar to the line equation $y = mx + b$ with $m = \beta_1$ (Slope) and $b = \beta_0$(Intercept).

So in this Simple Linear Regression model we want to draw a line between *X* and *Y* which estimates the relationship between *X* and *Y*. Finding these coefficients is the learning procedure.
We can find these using different approaches. One is called **Ordinary Least Square Method** and other one is called **Gradient Descent Approach**. We will use Ordinary Least Square Method in Simple Linear Regression

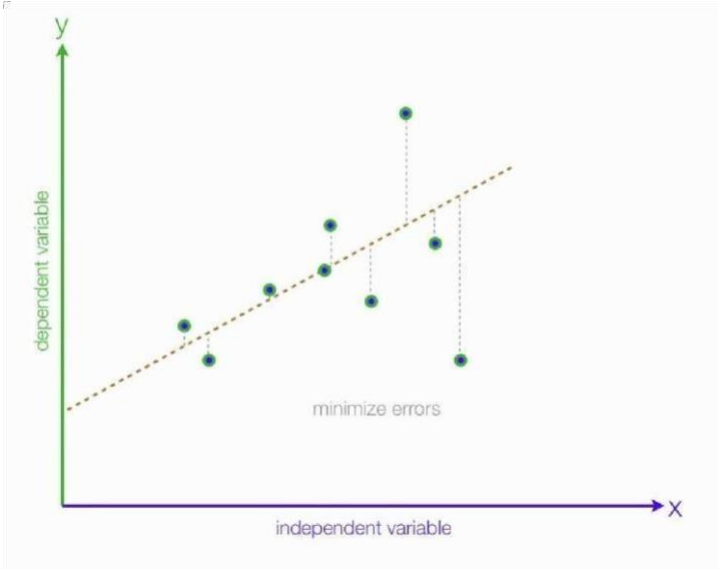
Ordinary Least Square Method

we are going to approximate the relationship between X and Y to a line. Let's say we have few inputs and outputs. And we plot these scatter points in 2D space, we will get something like the following image.



A line in the image. That's what we are going to accomplish. And we want to minimize the error of our model. A good model will always have least error. We can find this line by reducing the error.

The error of each point is the distance between line and that point. This is illustrated as follows.



And total error of this model is the sum of all errors of each point. ie.

$$D - \sum_{i=1}^{m} d_i^2$$

where,

d_i - Distance between line and i^{th} point.

m - Total number of points

You might have noticed that we are squaring each of the distances. This is because, some points will be above the line and some points will be below the line. We can minimize the error in the model by minimizing D. And after the mathematics of minimizing D, we will get;

$$\beta_1 = \frac{\sum_{i=1}^{m} [(x_i - \bar{x}_i)(y_i - \bar{y})]}{\sum_{i=1}^{m} [(x_i - \bar{x})^2]}$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

In these equations x is the mean value of input variable ***X*** and y is the mean value of output variable ***Y***. Now we have the model. This method is called **Ordinary Least Square Method**.

Evaluation of the Regression model:

In-order to find how good is our model we go for RMSE evaluation. There are many methods to evaluate models. We will use **Root Mean Squared Error** and **Coefficient of Determination (R^2 Score)**. Root Mean Squared Error is the square root of sum of all errors divided by number of values.

In Mathematical expression,

$$RMSE = \sqrt{\sum_{x=i}^{m} \frac{1}{m}(\widehat{y_i} - y_i)^2}$$

R^2-Score:

SS_t is the total sum of squares

$$SS_t = \sum_{i=1}^{m}(y_i - y)^2$$

And SS_r is the total sum of squares of residuals

$$SS_r = \sum_{i=1}^{m}(y_i - y_i)^2$$

R^2 score has expressed as

$$R^2 Score = 1 - \frac{SS_r}{SS_t}$$

R^2 Score usually range from 0 to 1. It will also become negative if the model is completely wrong.

b) Logistic Regression

Logistic regression is a fundamental classification technique. It belongs to the group of **linear classifiers** and is somewhat similar to polynomial and **linear regression**. Logistic regression is fast and relatively uncomplicated, and it's convenient for us to interpret the results. Although it's essentially a method for binary classification, it can also be applied to multiclass problems.

The nature of the dependent variables differentiates regression and classification problems. **Regression** problems have continuous and usually unbounded outputs. An example is when you're estimating the salary as a function of experience and education level. On the other hand, **classification** problems have discrete and finite outputs called **classes** or **categories**.

Objective:

Our goal is to find the **logistic regression function** $p(\mathbf{x})$ such that the **predicted responses** $p(\mathbf{x})$ are as close as possible to the **actual response** y_i for each observation $i = 1, \ldots, n$. Remember that the actual response can be only 0 or 1 in binary classification problems. This means that each $(\mathbf{x})$ should be close to either 0 or 1. That's why it's convenient to use the sigmoid function. Once we have the logistic

regression function $p(\mathbf{x})$, we can use it to predict the outputs for new and unseen inputs, assuming that the underlying mathematical dependence is unchanged.

Theory:

Mathematically, a binary logistic model has a target variable which is dichotomous in nature i.e with two possible values, labeled "0" or "1". In the logistic model, the log-odds (the logarithm of the odds) for the value labeled "1" is a linear combination of one or more independent variables ("predictors"); the independent variables can each be a binary variable or a continuous variable (any real value).

Logistic Regression uses a log of odds as the dependent variable. It predicts the probability of occurrence of a binary event utilizing a logit function.

General Linear Regression Equation:

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_n X_n$$

Where,

y is dependent variable

$X1, X2, ..., X_n$ are explanatory variables

Sigmoid Function

The sigmoid function, also called logistic function gives an 'S' shaped curve that can take any real- valued number and map it into a value between 0 and 1. If the curve goes to positive infinity, y predicted will become 1, and if the curve goes to negative infinity, y predicted will become 0. If the output of the sigmoid function is more than 0.5, we can classify the outcome as 1 or YES, and if it is less than 0.5, we can classify it as 0 or NO. If the output is 0.75, we can say in terms of probability as: There is a 75 percent chance that desired event will occur.

$$p = \frac{1}{(1 + e^{-y})}$$

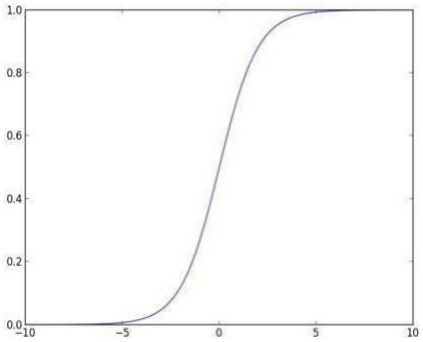


Fig.1. Sigmoid Function

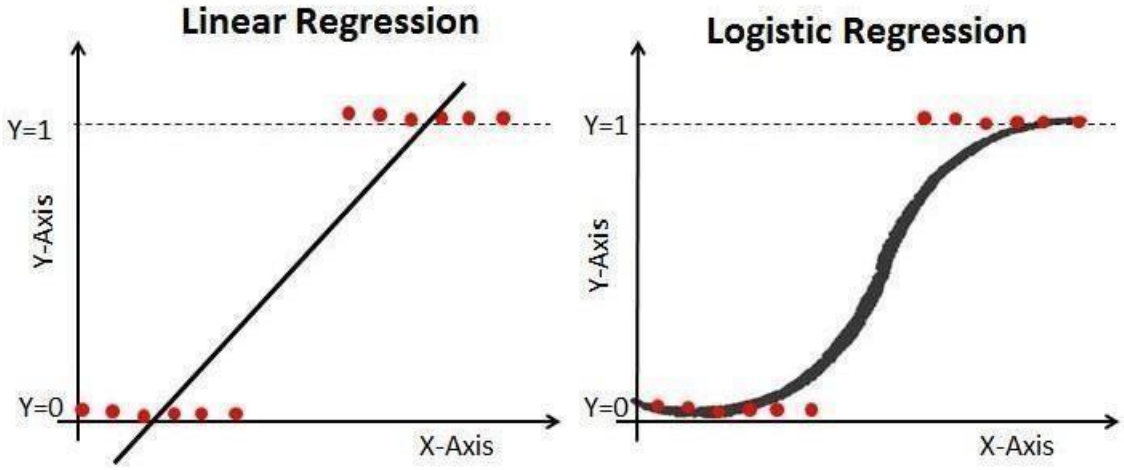
Apply Sigmoid function on linear regression we get:

$$p = 1/1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_n X n)}$$

Properties of Logistic Regression:

The dependent variable in logistic regression follows Bernoulli Distribution.

Linear regression is estimated using Ordinary Least Squares (OLS) while logistic regression is estimated using Maximum Likelihood Estimation (MLE) approach.



Maximum Likelihood Estimation Vs. Least Square Method

The MLE is a "likelihood" maximization method, while OLS is a distance-minimizing approximation method. Maximizing the likelihood function determines the parameters that are most likely to produce the observed data. From a statistical point of view, MLE sets the mean and variance as parameters in determining the specific parametric values for a given model. This set of parameters can be used for predicting the data needed in a normal distribution.

Ordinary Least squares estimates are computed by fitting a regression line on given data points that has the minimum sum of the squared deviations (least square error). Both are used to estimate the parameters of a linear regression model. MLE assumes a joint probability mass function, while OLS doesn't require any stochastic assumptions for minimizing distance.

Methodology

Logistic regression linear function $y = (\mathbf{x}) = b_0 + b_1x_1 + \cdots + b_rx_r$, also called the logit. The variables $b_{0,1,\ldots}, b_r$ are the **estimators** of the regression coefficients, which are also called the **predicted weights** or just **coefficients**.

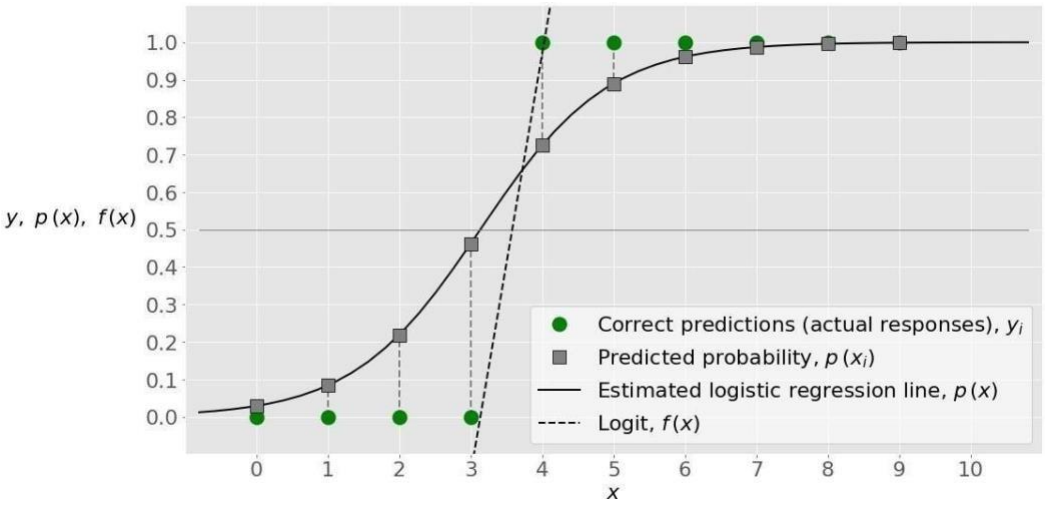
The logistic regression function $(\mathbf{x})$ is the sigmoid function of $(\mathbf{x})$: $p(\mathbf{x}) = 1 / (1 + \exp(-f(\mathbf{x}))$. As such, it's often close to either 0 or 1. The function $(\mathbf{x})$ is often interpreted as the predicted probability that the output for a given $\mathbf{x}$ is equal to 1. Therefore, $1 - (x)$ is the probability that the output is 0.

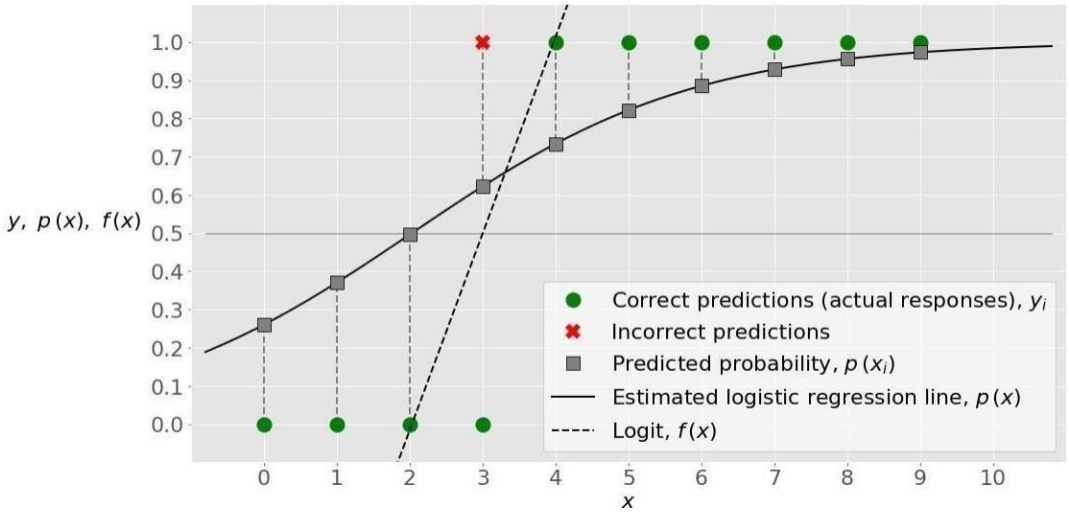
Logistic regression determines the best predicted weights $b_{0,1,}$, …, b_r such that the function $p(\mathbf{x})$is as close as possible to all actual responses y_i, $i = 1, \ldots, n$, where n is the number of observations. The process of calculating the best weights using available observations is called **model training** or **fitting**.

To get the best weights, we usually maximize the **log-likelihood function (LLF)** for all observations $i = 1, \ldots, n$. This method is called the **maximum likelihood estimation** and is represented by the equation $LLF = \Sigma_i(y_i \log(p(\mathbf{x}_i)) + (1 - y_i) \log(1 - p(\mathbf{x}_i)))$.

When $y_i = 0$, the LLF for the corresponding observation is equal to $\log(1 - p(\mathbf{x}_i))$. If $(\mathbf{x}_i)$ is close to $y_i = 0$, then $\log(1 - p(\mathbf{x}_i))$ is close to 0. This is the result you want. If $(\mathbf{x}_i)$ is far from 0, then $\log(1 - p(\mathbf{x}_i))$ drops significantly. You don't want that result because your goal is to obtain the maximum LLF. Similarly, when

$y_i = 1$, the LLF for that observation is $y_i \log(p(\mathbf{x}_i))$. If $(\mathbf{x}_i)$ is close to $y_i = 1$, then $\log(p(\mathbf{x}_i))$ is close to 0. If $(\mathbf{x}_i)$ is far from 1, then $\log(p(\mathbf{x}_i))$ is a large negative number.
There are several mathematical approaches that will calculate the best weights that correspond to the maximum LLF, but that's beyond the scope.





Logistic Regression in Python With scikit-learn:

There are several general steps you'll take when you're preparing your classification models:

1. **Import** packages, functions, and classes
2. **Get** data to work with and, if appropriate, transform it
3. **Create** a classification model and train (or fit) it with your existing data
4. **Evaluate** your model to see if its performance is satisfactory

Scikit-learn is a free machine learning library for Python.

Seaborn is a library for making statistical graphics in Python. It builds on top of matplotlib and integrates closely with pandas data structures. Seaborn helps you explore and understand your data. Its plotting functions operate on dataframes and arrays containing whole datasets and internally perform the necessary semantic mapping and statistical aggregation to produce informative plots.

Model Development and Prediction

Import the Logistic Regression module and create a Logistic Regression classifier object using Logistic Regression() function. Then, fit your model on the train set using fit() and perform prediction on the test set using predict()

Model Evaluation using Confusion Matrix

A confusion matrix is a table that is used to evaluate the performance of a classification model. You can also visualize the performance of an algorithm. The fundamental of a confusion matrix is the number of correct and incorrect predictions are summed up class-wise.

Ex no: 1
Date :

Linear and Logistic Regression with Error Estimation

a) Linear regression

Aim:

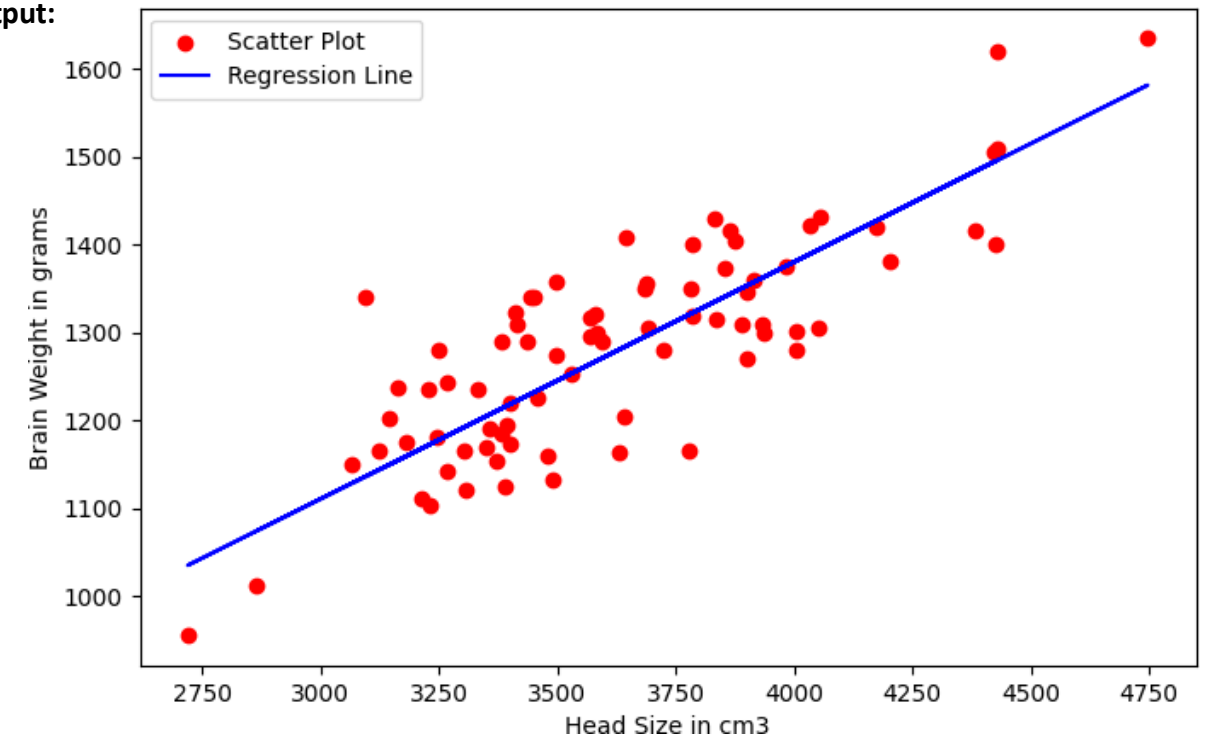
To implement linear regression in python using scikit learn

Source code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
```

```
data = pd.read_csv("../datasets/headbrain.csv")
X = data["Head Size(cm^3)"]
y = data["Brain Weight(grams)"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
random_state=42)
model = LinearRegression()
model.fit(np.expand_dims(X_train,axis=1),np.expand_dims(y_train,axis=1))
predictions = model.predict(np.expand_dims(X_test,axis=1))
plt.figure(figsize=(8,5))
plt.scatter(X_test,y_test,color="r",label='Scatter Plot')
plt.plot(X_test,predictions,color="b",label='Regression Line')
plt.xlabel('Head Size in cm3')
plt.ylabel('Brain Weight in grams')
plt.legend()
```

Output:



```
r2_score =
model.score(np.expand_dims(X_test,axis=1),np.expand_dims(y_test,axis=1))
mse = mean_squared_error(y_test,predictions)
print(f"RMSE VALUE : {np.sqrt(mse)}\nR2 SCORE : {r2_score}")
```

output:

```
RMSE VALUE : 68.91317515113433
R2 SCORE : 0.6692497355337241
```

b) Logistic Regression

Program 1- Preprocessing and implementing Logistic Regression on titanic dataset using Scikit Learn

AIM:

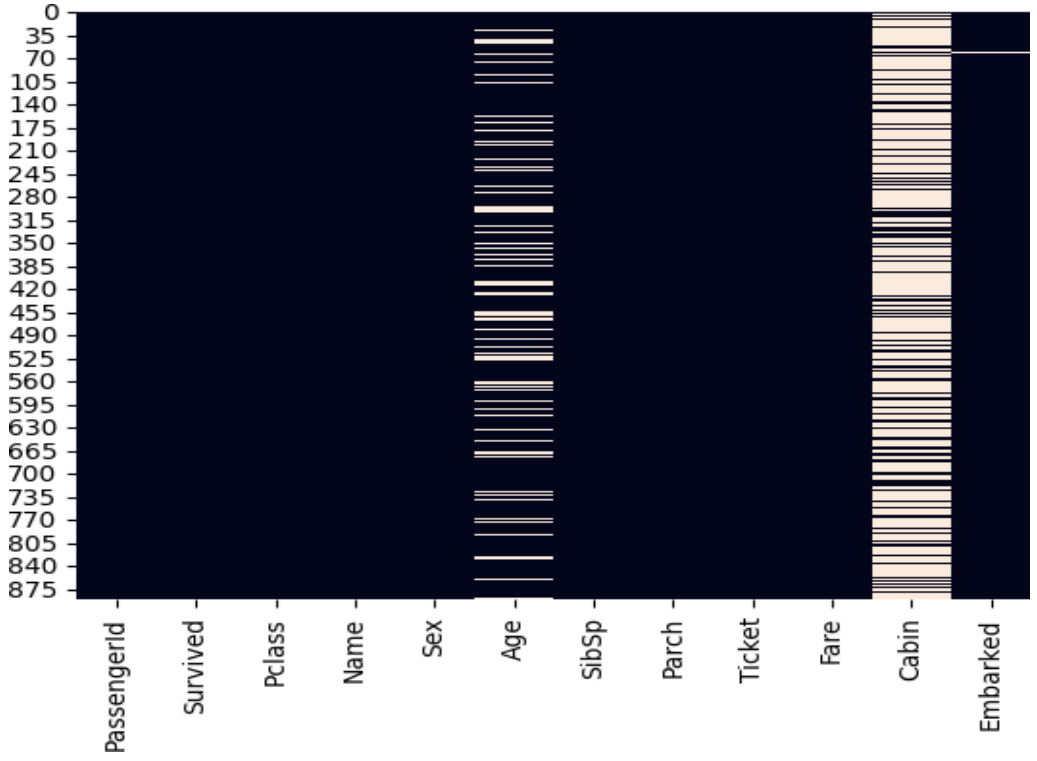
To preprocess and implement Logistic Regression in titanic dataset using Scikit Learn

SOURCE CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics  import  classification_report,confusion_matrix
```

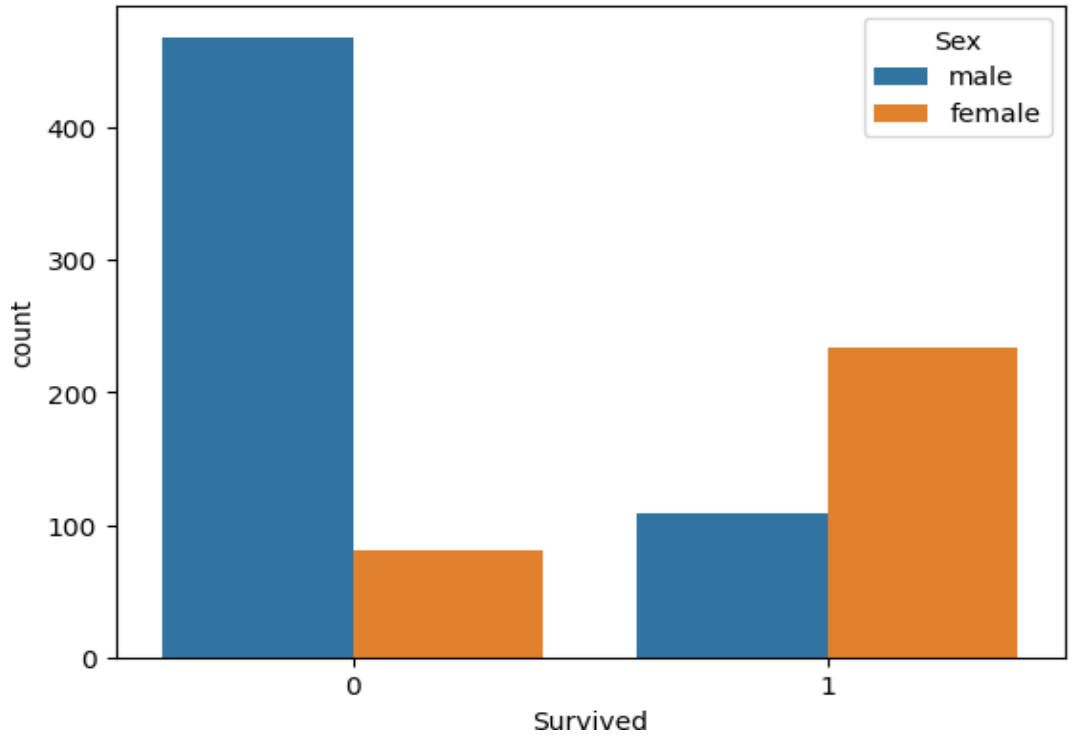
```
titanic_data = pd.read_csv("../datasets/titanic.csv")
sns.heatmap(titanic_data.isnull(),cbar=False)
```

output:



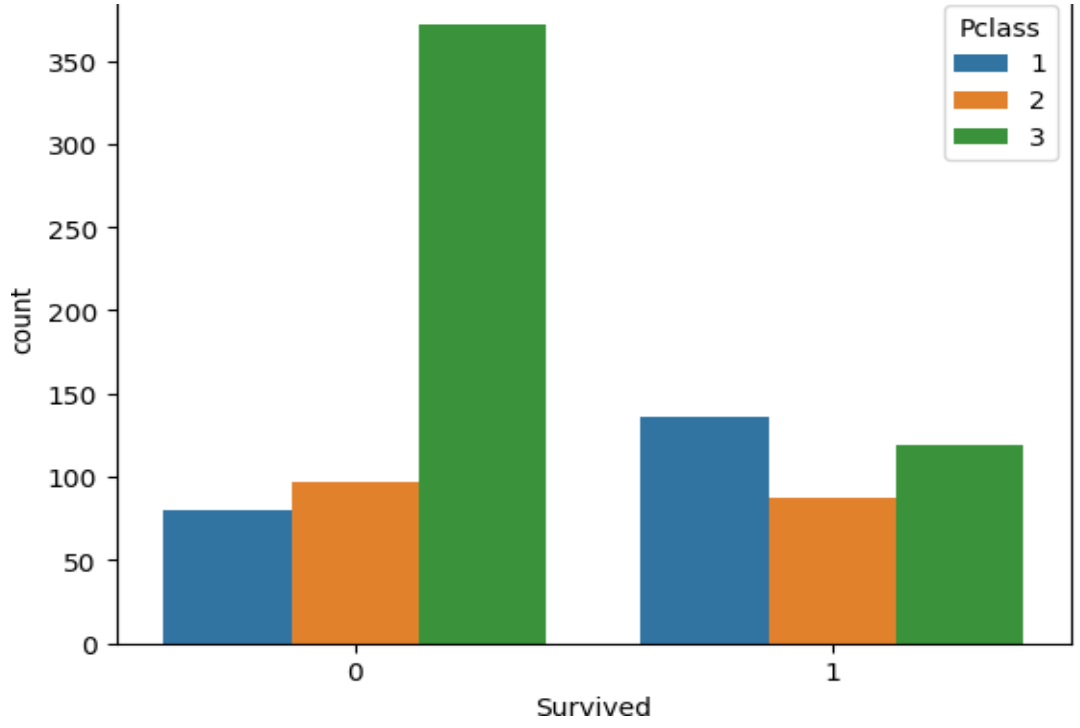
```
sns.countplot(x='Survived', hue='Sex', data=titanic_data)
```

output:




```
sns.countplot(x='Survived', hue='Pclass', data=titanic_data)
```

output:

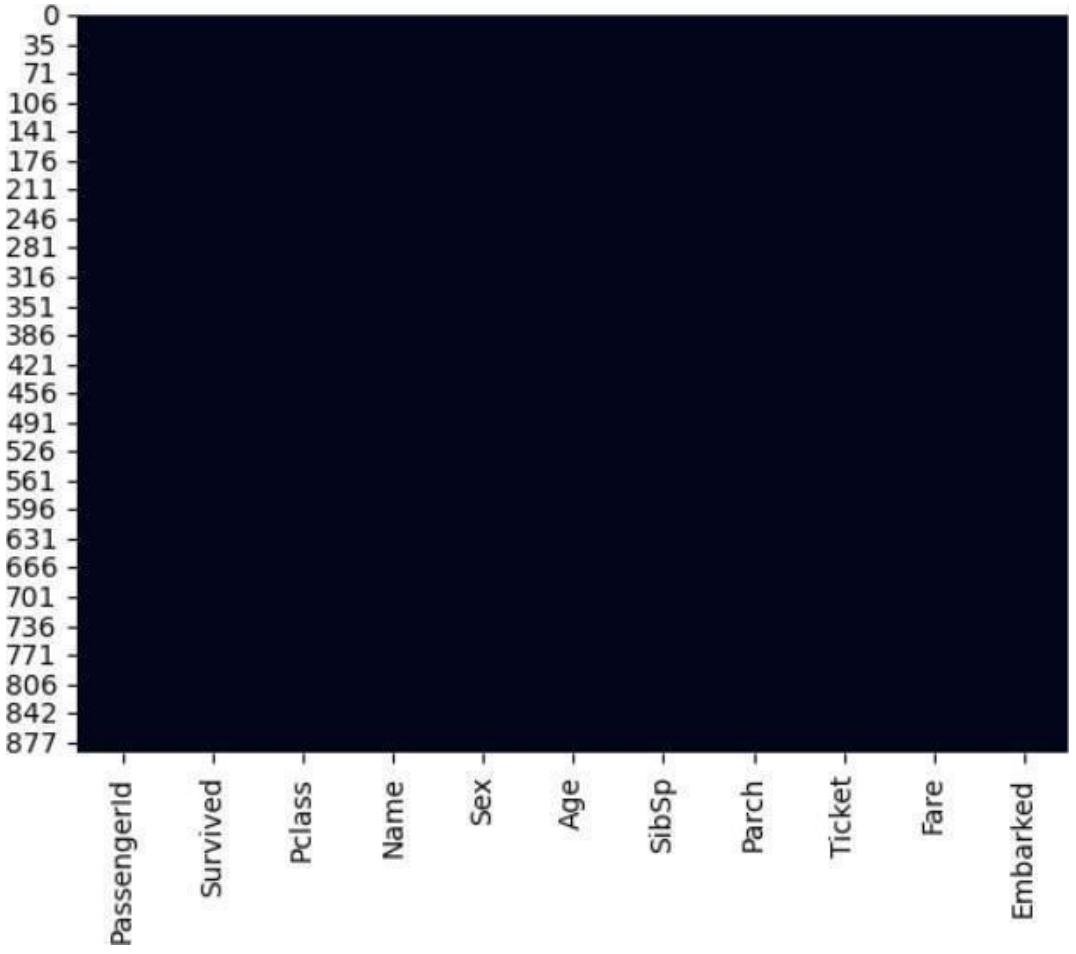


#Fill the null values of column [age] by the mean of age of peoples in same Pclass.

```
def get_age(row):
    if pd.isnull(row["Age"]):
        same_class = titanic_data[titanic_data["Pclass"] == row["Pclass"]]
        return same_class["Age"].mean()
    else:
        return row["Age"]

titanic_data["Age"] = titanic_data[["Age","Pclass"]].apply(get_age ,axis=1)
titanic_data.drop("Cabin",axis=1,inplace=True)
titanic_data.dropna(inplace=True)
```

```
sns.heatmap(titanic_data.isnull(),cbar=False)
```




```
sex_data = pd.get_dummies(titanic_data["Sex"]).astype(int)
embarked_data = pd.get_dummies(titanic_data['Embarked']).astype(int)
titanic_data = pd.concat([titanic_data, sex_data, embarked_data], axis = 1)
titanic_data.drop(['Name', 'PassengerId', 'Ticket', 'Sex', 'Embarked'], axis = 1, inplace
= True)

titanic_data.head(5)
```

output:

	Survived	Pclass	Age	SibSp	Parch	Fare	female	male	C	Q	S
0	0	3	22.0	1	0	7.2500	0	1	0	0	1
1	1	1	38.0	1	0	71.2833	1	0	1	0	0
2	1	3	26.0	0	0	7.9250	1	0	0	0	1
3	1	1	35.0	1	0	53.1000	1	0	0	0	1
4	0	3	35.0	0	0	8.0500	0	1	0	0	1

```
X = titanic_data.drop("Survived",axis = 1)
y = titanic_data["Survived"]
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.3)
model = LogisticRegression()
model.fit(X_train,y_train)
predictions = model.predict(X_test)
```

```
print("************Classification Report************\n")
print(classification_report(y_test, predictions))

print("***********Confusion Matrix***************")
print(confusion_matrix(y_test, predictions))
```

output:

```
************Classification Report************

              precision    recall  f1-score   support

           0       0.87      0.87      0.87       166
           1       0.78      0.78      0.78       101

    accuracy                           0.84       267
   macro avg       0.82      0.82      0.82       267
weighted avg       0.84      0.84      0.84       267

***********Confusion Matrix***************
[[144  22]
 [ 22  79]]
```

Program 2- Preprocessing and implementing Logistic Regression on Diabetes dataset using Scikit Learn.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report,confusion_matrix
```

```
diabetes_data = pd.read_csv("../datasets/diabetes.csv")
X = diabetes_data.drop("Outcome",axis=1)
y = diabetes_data["Outcome"]
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.3)
```

```
model = LogisticRegression()
model.fit(X_train,y_train)
predictions = model.predict(X_test)
```

```
print("************Classification Report************\n")
print(classification_report(y_test, predictions))

print("***********Confusion Matrix***************")
print(confusion_matrix(y_test, predictions))
```

output:

```
************Classification Report************

              precision    recall  f1-score   support

           0       0.81      0.87      0.84       161
           1       0.64      0.53      0.58        70

    accuracy                           0.77       231
   macro avg       0.72      0.70      0.71       231
weighted avg       0.76      0.77      0.76       231

***********Confusion Matrix***************
[[140  21]
 [ 33  37]]
```

Conclusion:

Thus we learned to fit Linear and Logistic Regression model for given Dataset via Python.

2. Implementation of Univariate Multivariate Gaussian density

Normal Distribution

Normal distribution or Gaussian distribution is a continuous probability distribution that describes data that cluster around a mean or average. The graph of the associated probability density function is bell-shaped, with a peak at the mean, and is known as the Gaussian function or bell curve. The variable is distributed normally with mean and variance.

It can be categories into

- **Univariate Density:** It involves single variable (one dimension)
  Example: Height of person
- **Multivariate Density:** It involves multivariable (two or more dimensions)
  Example: Height and weight of person

Univariate Density

Probability density function for univariate density is written as

$$p(x) = \frac{1}{\sqrt{2pi\sigma}} exp\left[-\frac{1}{2}\left(\frac{x \quad \mu}{\sigma}\right)^2\right]$$

where μ is mean, σ 2 is variance

Mean (μ) **:**

Mean is the average of the given feature x and it is given by

$$\mu = \frac{\sum x}{n}$$

Variance (σ^2) **:**

The spread of the data can be measured using variance.

$$\sigma^2 = \left(\frac{\sum (x - \mu)^2}{n}\right)$$

Example: Measure of the variability of the heights of the person.

- The normal density is traditionally described as a bell-shaped curve and is shown in Fig1.
- The normal distribution is symmetrical about mean *(μ)*.
- Peak of the univariate normal distribution occurs $x = \mu$ at and its value is $1/\sqrt{2\pi\sigma^2}$ which is shown in Fig.1.
- Width of the univariate normal distribution is proportional to standard deviation (*σ*).

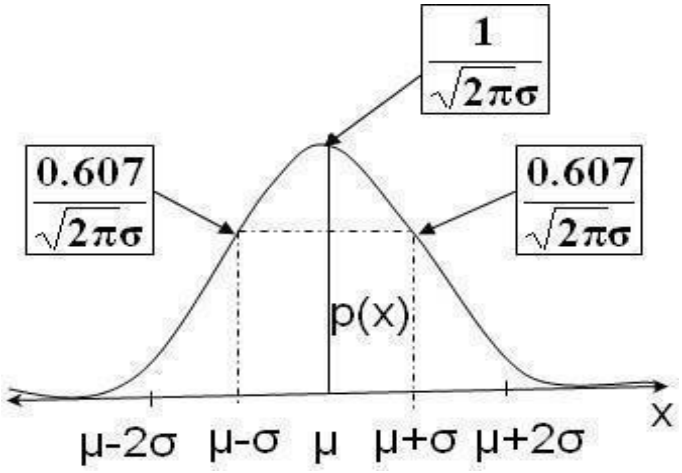


Fig.1. Peak of the univariate normal distribution occurs at x = μ

Multivariate Density

The general multivariate normal density in d-dimensions is written as

$$p(x) = \frac{1}{\sqrt{(2\pi)^d \ \Sigma|}} exp\left(-\frac{1}{2}(x-\mu)^t \Sigma^{-1}(x-\mu)\right)$$

Where,

x is the d-component column vector,

μ is the d-component mean vector,

$\sum$ is the d-by-d covariance matrix

$|\sum|$ is the determinant of covariance matrix

$\sum^{-1}$ is the inverse of covariance matrix

$(x-\mu)^t$ is transpose of $(x-\mu)$

Mean Vector (μ) and Covariance Matrix ($\sum$) :

$$\mu = \begin{bmatrix} \mu_1 \\ \mu_2 \\ . \\ . \\ . \\ \mu_n \end{bmatrix}, \Sigma = \begin{bmatrix} \sigma_{11} & \cdots & \sigma_{1d} \\ \vdots & \ddots & \vdots \\ \sigma_{d1} & \cdots & \sigma_{dd} \end{bmatrix}$$

$$\sigma_{i,j} = \sigma_{j,i} = \frac{1}{n} \sum^{n} [(x_i - \mu_i)(x_j - \mu_j)]$$

Where,

$\sigma 11$ is variance within $x1$

$\sigma 12$ is variance between $x1$ and $x2$

.

.

σ_{ij} is variance between x_i and x_j

Σ is symmetric and its diagonal elements are variances within x which can never be negative. Off-diagonal elements are the covariances which can be +ve and -ve.

Statistically Dependent Variables:

The variables which are causally related are called statistically dependent variables.

Example: engine temperature and oil temperature.

Statistically Independent Variables:

The variables which are not causally related are called statistically independent variables.

Example: oil pressure in engine and air pressure in tire.

If the variables are statistically independent, the covariances are zero and covariance matrix is diagonal.

$$\Sigma = \begin{bmatrix} {\sigma_1}^2 & 0 & \cdots & 0 \\ 0 & {\sigma_2}^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & {\sigma_d}^2 \end{bmatrix}, \quad \Sigma^{-1} = \begin{bmatrix} 1/{\sigma_1}^2 & 0 & \cdots & 0 \\ 0 & 1/{\sigma_2}^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1/{\sigma_d}^2 \end{bmatrix}$$

$$|\Sigma| = [{\sigma_1}^2 \times {\sigma_2}^2 \times \cdots \times \sigma_d^2]$$

Multivariate Density (Bivariate density):

$p(x1,x2)$ is a hill shaped surface over the $(x1,x2)$ plane. Peak of the bivariate normal distribution occurs at the point $(x1,x2) = (\mu1,\mu2)$ that is at the mean vector. The shape of the hump depends on the two variances ${\sigma_1}^2$, ${\sigma_2}^2$ and correlation coefficient (ρ) by,

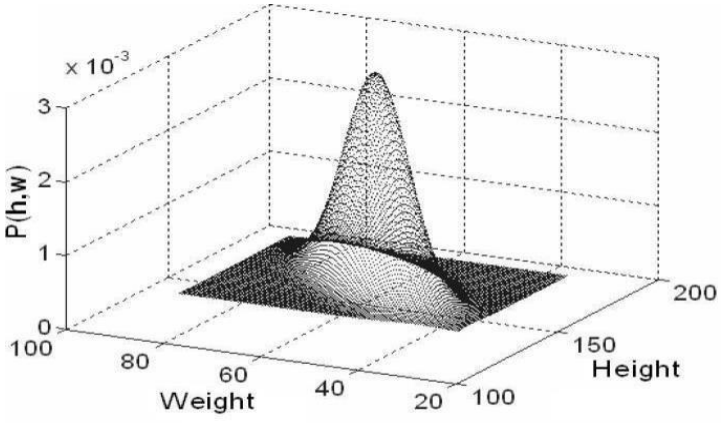
$$\rho = \frac{\sigma_{12}}{\sigma_1 \sigma_2}$$

Where,

$\sigma12$ is variance between $x1$ and $x2$

$\sigma1$ is variance of $x1$

$\sigma2$ is variance of $x2$



Bivariate Normal distribution

Ex no : 2
Date : **Implementation of Univariate and Multivariate Gaussian densities**

AIM:

To generate, visualize and analyze various univariate and multivariate normal distributions.

Part 1- Univariate Normal Distribution

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
```

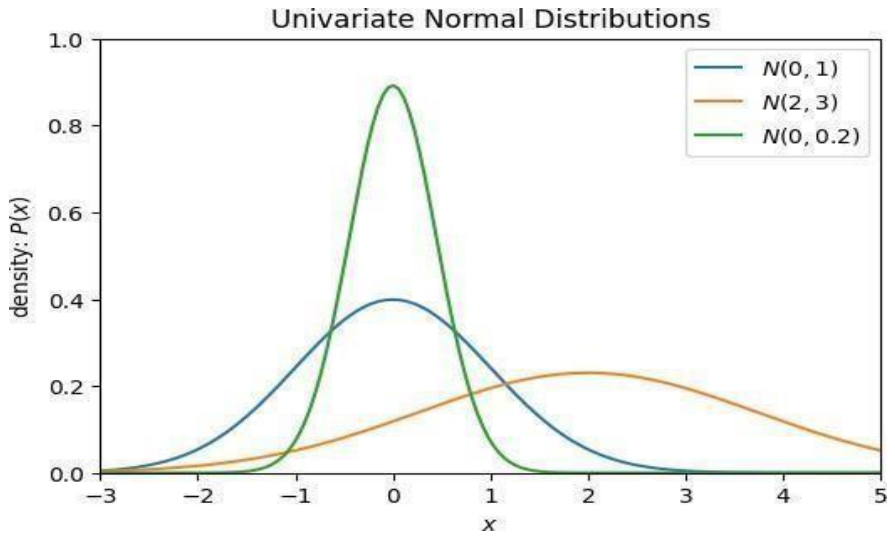
```
x = np.linspace(-3, 5, 150)
params = [(0, 1, "$N(0,1)$"), (2, 3, "$N(2,3)$"), (0, 0.2, "$N(0,0.2)$")]

fig, ax = plt.subplots(figsize=(6, 4))

for mean, variance, label in params:
    pdf = norm.pdf(x, loc=mean, scale=np.sqrt(variance))
    ax.plot(x, pdf, label=label)

ax.set_title("Univariate Normal Distributions")
ax.set_xlabel("$x$")
ax.set_ylabel("density: $P(x)$")
ax.set_xlim((-3, 5))
ax.set_ylim((0, 1))
ax.legend()
plt.show()
```

output:



Part 2 - Multivariate Normal Distribution

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal
```

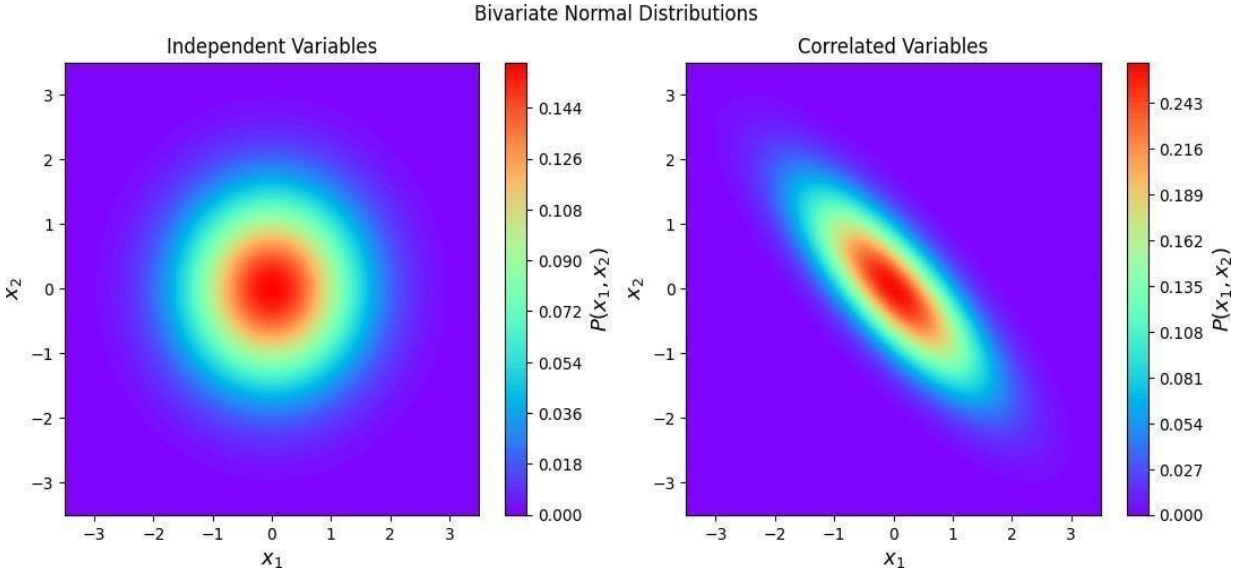
```
def generate_surface(mean, cov):
    x, y = np.linspace(-5, 5, 100), np.linspace(-5, 5, 100)
    x1, x2 = np.meshgrid(x, y)
    pos = np.dstack((x1, x2))
    pdf = multivariate_normal(mean=mean, cov=cov).pdf(pos)
    return x1, x2, pdf

distributions = [
    (np.array([0, 0]), np.eye(2)),
    (np.array([0, 0]), np.array([[1, -0.1], [-0.8, 1]]))
]

fig, ax = plt.subplots(1, 2, figsize=(13, 5))
plt.suptitle("Bivariate Normal Distributions")
ax[0].set_title("Independent Variables")
ax[1].set_title("Correlated Variables")

for i, dist in enumerate(distributions):
    x1, x2, pdf = generate_surface(dist[0], dist[1])
    con = ax[i].contourf(x1, x2, pdf, 100, cmap='rainbow')
    ax[i].set_xlabel("$x_1$", fontsize=13)
    ax[i].set_ylabel("$x_2$", fontsize=13)
    ax[i].axis([-3.5, 3.5, -3.5, 3.5])
    c_bar = fig.colorbar(con, ax=ax[i])
    c_bar.ax.set_ylabel("$P(x_1, x_2)$", fontsize=13)
plt.show()
```

output:



Conclusion:

Thus we have learned to Model and plot Univariate and Multivariate Gaussian Density function with various Mean,Variances and StdDeviation.

3. Dimensionality Reduction using Principal Component Analysis

Introduction:

Development in the data collection and storage techniques in recent days have led to an information overload. Researchers working on diversified domains such as engineering, astronomy, biology, remote sensing, consumer transactions, etc., come across an huge or high dimensional datasets which present many mathematical challenges to handle due to curse of dimensionality. As the dimensionality of the dataset increases, its performance decreases as seen in Fig.1. Moreover, if the dimensionality of the input space is higher, more feature vectors are needed for training. The major problem with these high-dimensional data sets is that, in most cases, not all the measured data are important for understanding the underlying phenomena of interest. Thus, dimension reduction is necessary for effective analysis of high dimensional data sets. Principal components analysis (PCA) and Linear discriminant analysis (LDA) are well-known schemes for dimension reduction. PCA finds a set of most representative projection vectors, and thus the projected samples preserves the most relevant information about original dataset.

Principal Components Analysis

Principal components analysis(PCA) is a useful statistical technique that has found application in many fields such as video/audio classification, face recognition, image compression etc., It is a simple method of extracting relevant information from high dimensional data sets. With minimal effort,

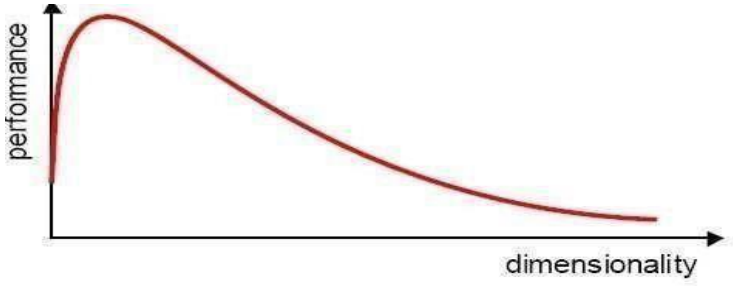
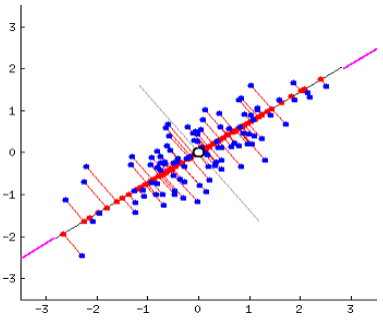


Fig.1: Curse of dimensionality

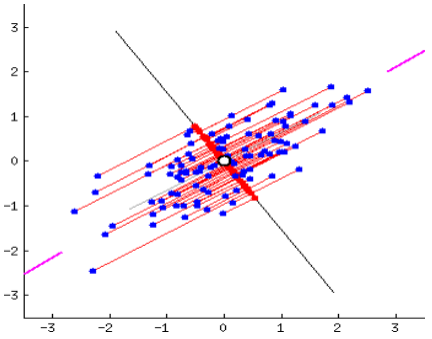
PCA provides a road map for reducing the complex data set to a lower dimensional dataset. Principal component analysis (PCA) was first introduced by Pearson in 1901 and later independently developed by Hotelling in 1933, where the name principal components first appears. In various fields, it is also known as the singular value decomposition (SVD), the Karhunen-Loeve transform, the Hotelling transform, and the empirical orthogonal function (EOF) method.

Principal components are new variables that are constructed as linear combinations or mixtures of the initial variables. These combinations are done in such a way that the new variables (i.e., principal components) are uncorrelated and most of the information within the initial variables is squeezed or compressed into the first components. So, the idea is 10-dimensional data gives you 10 principal components, but PCA tries to put maximum possible information in the first component, then maximum remaining information in the second and so on, until having something like shownin the scree plot below.

Geometrically principal components represent the directions of the data that explain a **maximal amount of variance**, that is to say, the lines that capture most information of the data. The relationship between variance and information, i.e, the larger the variance carried by a line, the larger the dispersion of the data points along it, and the larger the dispersion along a line, the more the information it has.



Principal Components 1

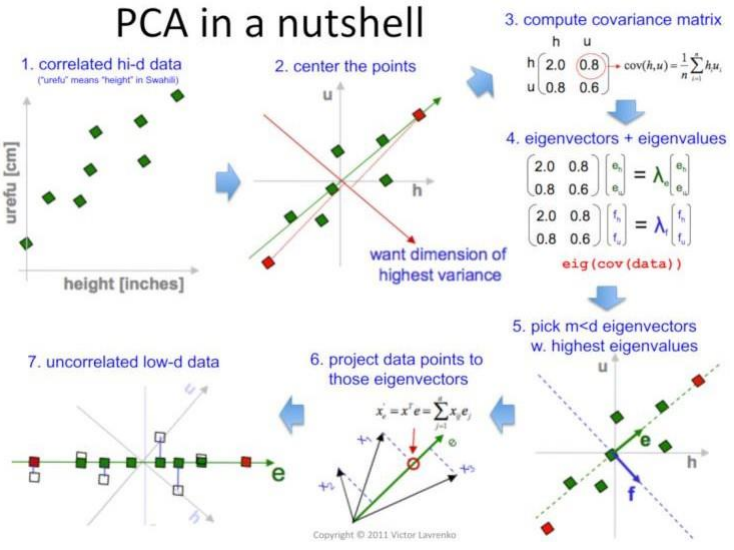


Principal Components 2

Principal Component Analysis – Algorithm

Principal component analysis (PCA) algorithm is used to convert *d* dimensional feature vectors into *m* dimensional feature vectors, where $m << d$. (i.e., *m* is very less than *d*)

1. The input dataset has *n* number of higher dimensional (*d*) feature vectors.
2. Compute mean vector and obtain the mean subtracted feature vectors.
3. Compute *d* x *d* dimensional covariance matrix for the mean subtracted feature vectors.
4. Calculate *d* Eigen values and *d* Eigen vectors of the covariance matrix.
5. Select largest *m* Eigen values and its Eigen vectors (Principal Components).
6. Project each of the higher dimensional feature vector onto *m* Eigen vectors to obtain the reduced m-dimensional feature vector.



Ex No : 3
Date :

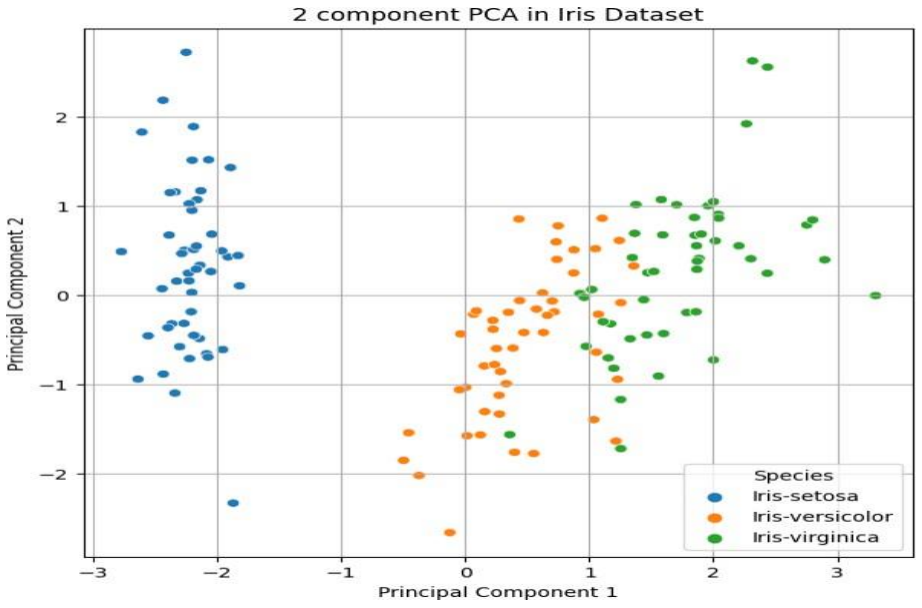
Implementation of PCA

Program 1- Implementation of PCA in Iris Data Set

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

```
data = pd.read_csv("../datasets/Iris.csv")
X = data[data.columns[1:-1]]
scaled_X=  StandardScaler().fit_transform(X)
reduced_X = PCA(n_components=2).fit_transform(scaled_X)
plt.figure(figsize=(7,7))
sns.scatterplot(x=np.ndarray.ravel(reduced_X[:,:1]),
                y=np.ndarray.ravel(reduced_X[:,1:]),
                hue=data['Species'])
plt.title("2 component PCA in Iris Dataset")
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.grid()
```

output:

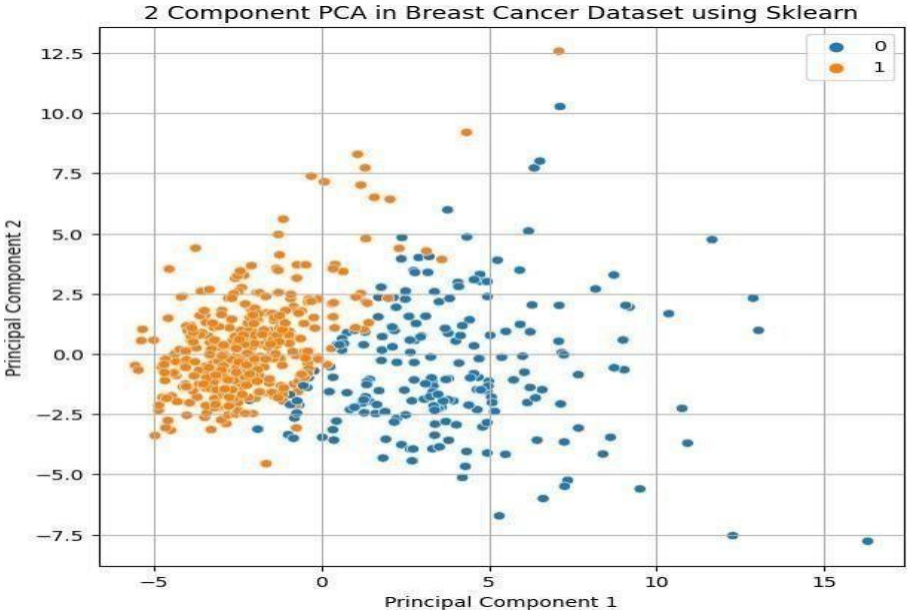


Program 2 - Implementing PCA for Breast cancer dataset using scikit learn

```
from sklearn.datasets import load_breast_cancer
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
data = load_breast_cancer()
```

```
X = data['data']
scaled_X=  StandardScaler().fit_transform(X)
reduced_X = PCA(n_components=2).fit_transform(scaled_X)
plt.figure(figsize=(7,7))
sns.scatterplot(x=np.ndarray.ravel(reduced_X[:,:1]),
                y=np.ndarray.ravel(reduced_X[:,1:]),
                hue=data['target'])
plt.title("2 Component PCA in Breast Cancer Dataset using Sklearn")
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.grid()
```

output:



Conclusion:

Thus we have learned to Redcue Feature Dimension of the Objects in the Dataset and only retain the feature attribute in a transformed space that best describe the object of interest via Python.

4. Clustering Algorithms

Introduction:

Clustering in a Machine Learining is an unsupervised learning method which deals with finding a structure in a collection of unlabeled data. A loose definition of clustering could be the process of organizing objects into groups whose members are similar in some way. A cluster is a collection of objects which are similar between them and are dissimilar to the objects belonging to other clusters.

Clustering:

Clustering is one of the most common exploratory data analysis technique used to get an intuition about the structure of the data. It can be defined as the task of identifying subgroups in the data such that data points in the same subgroup (cluster) are very similar while data points in different clusters are very different. In other words, we try to find homogeneous subgroups within the data such that data points in each cluster are as similar as possible according to a similarity measure such as euclidean-based distance or correlation-based distance. The decision of which similarity measure to use is application-specific.

Clustering analysis can be done on the basis of features where we try to find subgroups of samples based on features or on the basis of samples where we try to find subgroups of features based on samples.

We'll cover here clustering based on features. Clustering is used image segmentation/compression; where we try to group similar regions together, document clustering based on topics, etc.

Unlike supervised learning, clustering is considered an unsupervised learning method since we don't have the ground truth to compare the output of the clustering algorithm to the true labels to evaluate its performance. We only want to try to investigate the structure of the data by grouping the data points into distinct subgroups.

K-means which is considered as one of the most used clustering algorithms due to its simplicity. k-means clustering algorithm group objects based on attributes/features into k number of groups where k is a positive integer. The grouping (clustering) is done by minimizing the Euclidean distance between data and the corresponding cluster centroid. Thus the purpose of k-means clustering is to cluster the data.

Kmeans Algorithm

It is an iterative algorithm that tries to partition the dataset into *K*-pre-defined distinct non-overlapping subgroups (clusters) where each data point belongs to **only one group**. It tries to make the intra-cluster data points as similar as possible while also keeping the clusters as different (far) as possible. It assigns data points to a cluster such that the sum of the squared distance between the data points and the cluster's centroid (arithmetic mean of all the data points that belong to that cluster) is at the minimum. The less variation we have within clusters, the more homogeneous (similar) the data

points are within the same cluster.

The way kmeans algorithm works is as follows:

1. Specify number of clusters *K*.
2. Initialize centroids by first shuffling the dataset and then randomly selecting *K* data pointsfor the centroids without replacement.
3. Keep iterating until there is no change to the centroids. i.e assignment of data points toclusters isn't changing.

- Compute the sum of the squared distance between data points and all centroids.
- Assign each data point to the closest cluster (centroid).
- Compute the centroids for the clusters by taking the average of the all data points that belong
  to each cluster.

In Expectation-Maximization perspective The E-step is assigning the data points to the closest cluster. The M-step is computing the centroid of each cluster. Below is a break down of how we can solve it mathematically.

The objective function is: $$J = \sum_{i-1}^{m} \sum_{k=1}^{K} w_{ik} \|x_i - \mu_k\|^2$$

where $w_{ik} = 1$ for data point x_i if it belongs to cluster k; otherwise, $w_{ik} = 0$. Also, μ_k is the centroid of x_i's cluster.

It's a minimization problem of two parts. We first minimize J w.r.t. and treat w_{ik} fixed. Then we minimize J w.r.t. and treat μ_k fixed. Technically speaking, we differentiate J w.r.t. first and update w_{ik} cluster assignments (*E-step*). Then we differentiate J w.r.t. and recompute the centroids after the cluster assignments from previous step (*M-step*).

Therefore, E-step is: $$\frac{\partial J}{\partial w_{ik}} = \sum_{i=1}^{m} \sum_{k=1}^{K} \|x_i - \mu_k\|^2$$

$$\Rightarrow w_{ik} = \begin{cases} 1 & if\ k = argmin_j \|x_i - \mu_j\|^2 \\ 0 & otherwise \end{cases}$$

In other words, assign the data point x_i to the closest cluster judged by its sum of squared distance from cluster's centroid.

And M-step is: $$\frac{\partial J}{\partial \mu_k} = 2 \sum_{i=1}^{m} w_{ik}(x_i - \mu_k) = 0$$

$$\Rightarrow \mu_k = \frac{\sum_{i=1}^{m} w_{ik} x_i}{\sum_{i=1}^{m} w_{ik}}$$

Which is translates to recomputing the centroid of each cluster to reflect the new assignments.

Few things to note here:

- Since clustering algorithms including kmeans use distance-based measurements to determine the similarity between data points, it's recommended to standardize the data to have a mean of zero and a standard deviation of one since almost always the features in any dataset would have different units of measurements such as age vs income.
- Given kmeans iterative nature and the random initialization of centroids at the start of the algorithm, different initializations may lead to different clusters since kmeans algorithm may stuck in a local optimum and may not converge to global optimum. Therefore, it's recommended to run the algorithm using different initializations of centroids and pick the results of the run that that yielded the lower sum of squared distance.
- Assignment of examples isn't changing is the same thing as no change in within-cluster variation:

$$\frac{1}{m_k}\sum_{i=1}^{m_k}\|x_i - \mu_{c^k}\|^2$$

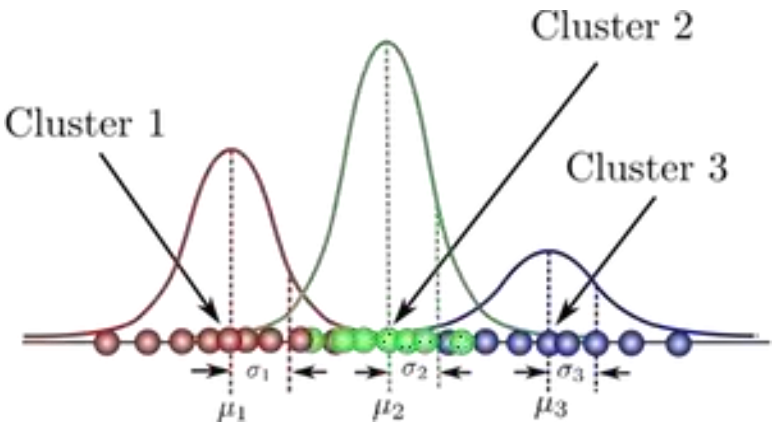
Implementation

Simple implementation of kmeans in this exercise is to just illustrate some concepts with use of Iris Dataset. Then we use `sklearn` implementation that is more efficient.

4. (b) Gaussian Mixture Models

Data measurements of many properties are often normally distributed, but with heterogeneous populations, sometimes data measurements reflect a mixture of normal distributions. Mixture models are a type of density model which comprise a number of component functions, usually Gaussian. These component functions are combined to provide a multi model density. Gaussian mixture models are formed by combining multivariate normal density components. Gaussian mixture models are often used for data clustering. Like k-means clustering, Gaussian mixture modeling uses an iterative algorithm that converges to a local optimum. Gaussian mixture modeling may be more appropriate than k-means clustering when clusters have different sizes and correlation within them.

Gaussian mixture model involves the mixture (i.e. superposition) of multiple Gaussian distributions. Rather than identifying clusters by "nearest" centroids, as in k-means in this experiment we fit a set of k gaussians to the data and then we estimate gaussian distribution parameters such as mean and variance for each cluster and weight of a cluster. After learning the parameters for each data point we can calculate the probabilities of it belonging to each of the clusters.



Maximum Likelihood

For most sensible models, we will find that certain data are more probable than other data. The aim of maximum likelihood estimation is to find the parameter (mean, covariance) value(s) that makes the observed data most likely. This is because the likelihood of the parameters given the data is defined to be equal to the probability of the data given the parameters.

However, in the case of data analysis, we have already observed all the data: once they have been observed they are fixed, there is no 'probabilistic' part to them anymore. We are much more interested in the likelihood of the model parameters that underly the fixed data.

Probability: Knowing parameters ⇒ Prediction of outcome Prediction of outcome

Likelihood: Observation of data =⇒Predictionofoutcome Estimation of parameters

For example, suppose you are interested in the heights of Americans. You have a sample of some number of Americans, but not the entire population, and record their heights. Further, you are willing to assume that heights are normally distributed with some unknown mean and variance. Thesample mean is then the maximum likelihood estimator of the population mean, and the sample variance is a close approximation to the maximum likelihood estimator of the population variance.

GMM

Gaussian mixture model (GMM) is a mixture of several Gaussian distributions and can therefore represent different subclasses inside one class. Figure.1 shows themixture of two Gaussians. The probability density function of GMM is defined as a weighted sum of Gaussians.

$$p(x \mid \Theta) = \sum_{k=1}^{K} \alpha_k p_k(x \mid \theta_k)$$

where k is the number of mixtures and

$$\sum_{k=1}^{K} \alpha_k = 1$$ are the mixture weights

where $\Theta = \{\alpha_1, ..., \alpha_k, \theta_1, ..., \theta_k\}$

We are given dataset $D = x_1, x_2, \ldots, x_n$ where x_i is a d−dimensional vector.
Assume that the points are generated in an identically independent fashion from an underlying density p(x). We further assume a Gaussian mixture model with K components.

$$p_k(x \mid \theta_k) = \frac{1}{(2\pi)^{\frac{d}{2}} |\Sigma_k|^{\frac{1}{2}}} \; e^{-\frac{1}{2}\left[(x-\mu_k)^T \; \Sigma_k^{-1}(x-\mu_k)\right]}$$

with its own parameters $\theta_k = \{\mu_k, \Sigma_k\}$We can compute the membership of data point x_i in cluster k given parameters Θ as

$$w_{i,k} = \frac{p_k\left(x_i \mid \theta_k\right) \cdot \alpha_k}{\sum_{m=1}^{K} p_m(x_i \mid \theta_m) \cdot \alpha_m}$$

GMM Training

Several approaches exist for estimating the parameters of the GMM given a set of data points. The most popular, and the one used here, is the expectation-maximization (EM) algorithm, which iteratively optimizes the model using maximum likelihood estimates. Expectation maximization (EM) algorithm is an iterative algorithm consisting of expectation step (E-step) and maximization step (M-step), and is widely used for model training

The EM Algorithm:

We define the EM (Expectation-Maximization) algorithm for Gaussian mixtures as follows.

E-Step: Denote the current parameter value as Θ. Compute w_{ik}, using equation $p_k(x \mid \theta_k)$ for all data points x_i, $1 \le i \le N$ and all mixture components $1 \le k \le K$. Note that for each data point x_i the membership weights are defined such that $\sum_{k=1}^{K} w_{ik} = 1$. This yields an N × K matrix of membership weights, where each of the rows sum to 1 .

M-Step: Now use the membership weights and the data to calculate new parameter values. Specifically,

$$\alpha_k^{new} = \frac{1}{N} \sum_{i=1}^{N} w_{ik} \cdot x_i \quad 1 \le k \le K.$$

$$\mu_k^{new} = \left(\frac{1}{\sum_{i=1}^{N} w_{ik}}\right) \sum_{i=1}^{N} w_{ik} \cdot x_i \quad 1 \le k \le K$$

$$\sum_{k}^{new} = \left(\frac{1}{\sum_{i=1}^{N} w_{ik}}\right) \sum_{i=1}^{N} w_{ik} \cdot (x_I - \mu_k^{new})(x_i - \mu_k^{new})^t \quad 1 \le k \le K$$

The equations in the M-Step need to be computed in this order, i.e., first compute the K new α′s,s, then the K new μ′s,s, and finally the K new ’s .

After we have computed all of the new parameters, the M-Step is complete and we can now go back and recompute the membership weights in the E-Step, then recompute the parameters again in the M-Step , and continue updating the parameters in this manner. Each pair of E and M steps is considered to be an iteration.

Initialization Issues

The initial parameters or weights can be chosen randomly (e.g. select K random data points as initial means and select the covariance matrix of the whole data set for each of the initial K covariance matrices) or could be chosen via some heuristic method (such as by using k-Means to cluster the data first and then defining weights based on k-Means memberships).

GMM Testing

After obtaining the maximum likelihood estimates of the various components of the mixture using EM algorithm the probability density function is found out using equation $p_k(x \mid \theta_k)$. Given a test data the probability density function is found out for each class as a weighted sum of Gaussians using the estimated parameters. The test data belongs to the class which has the highest probability.

Fig.2.: GMM-Training

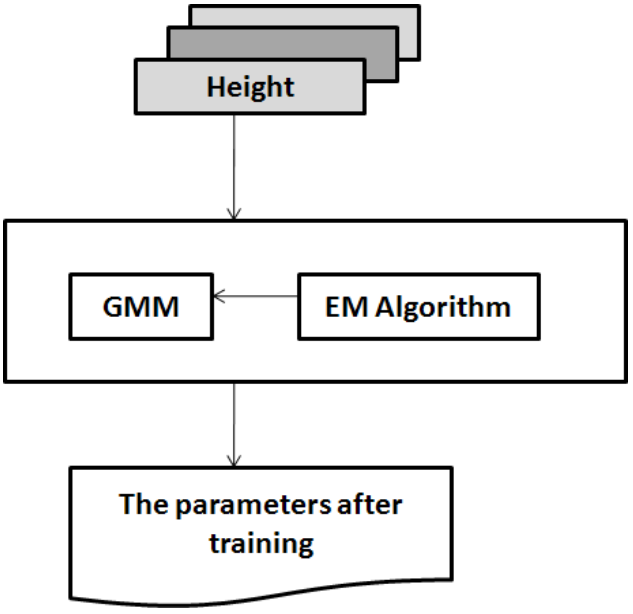
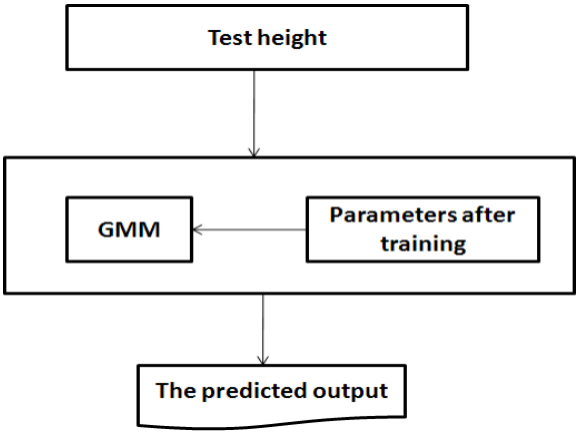


Fig.3.: GMM-Training



Algorithm

1. Initialize the mean μ_k, the covariance matrix Σ_k and the mixing coefficients α_k by somerandom values(or other values).
2. Compute the $w_{i,k}$ values for all k.
3. Again Estimate all the parameters using the current $w_{i,k}$ values.
4. Compute log-likelihood function.
5. Put some convergence criterion
6. If the log-likelihood value converges to some value (or if all the parameters converge to some values) then stop, else return to Step 2.

This algorithm only guarantee that we land to a local optimal point, but it do not guarantee that this local optima is also the global one. And so, if the algorithm starts from different initialization points, in general it lands into different configurations. For demonstraton purpose in this exercise data were generated from a random mixture of Gaussians for which we find the best fitclusters.

DATA generated from the stat package: scipy.stats

```
from scipy.stats import multivariate_normal
```

Ex No : 4
Date :

IMPLEMENTATION OF CLUSTERING ALGORITHMS

AIM:

To implement K-means clustering algorithm from scratch and also implement it with scikit learn.

a) Implementing K-Means Clustering

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans as sklKMeans
from sklearn.metrics import silhouette_score
```

```
data = pd.read_csv("../datasets/Iris.csv")
X = data[["SepalLengthCm","PetalLengthCm"]]

def kMeans(X,k):

    distortion = []
    for i in range(1,k+1):
        model =
sklKMeans(n_clusters=i,init='random',n_init='auto',max_iter=10).fit(X.values)
        distortion.append(model.inertia_)
    print(f"Silhoutte Score for {k} cluster is {silhouette_score(X,model.labels_)}")
    return model,distortion
k=5
model,distortion = kMeans(X,k)
```

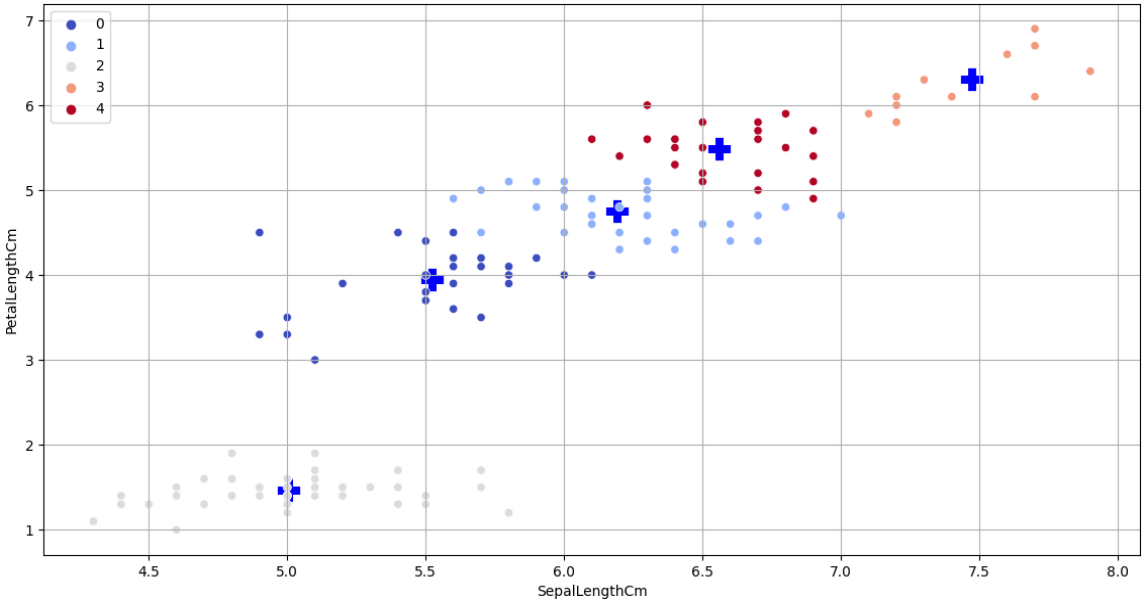
output:

Silhoutte Score for 5 cluster is 0.5615794939538516

```
Centroids = model.cluster_centers_
Index = model.labels_

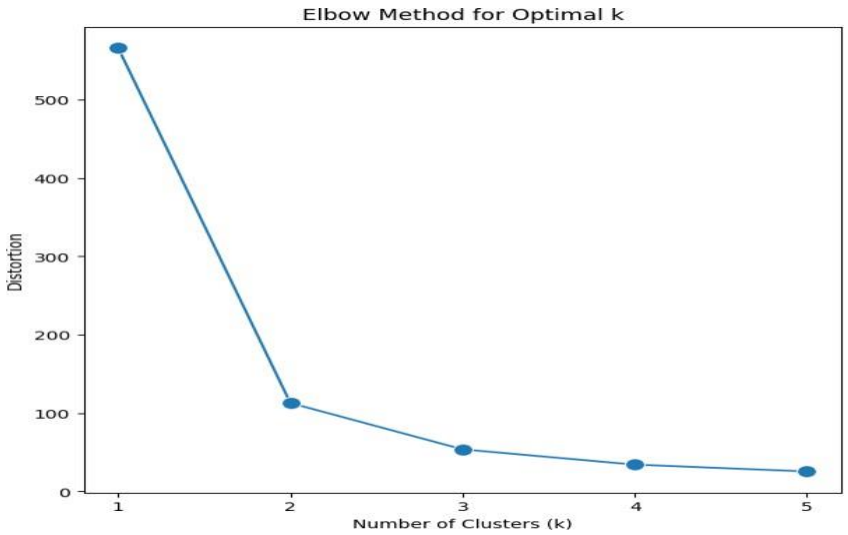
plt.figure(figsize=(14,7))
plt.scatter(x=centroids[:,0],y=centroids[:,1],marker="+",c='b',linewidth=16)
sns.scatterplot(data=data,x="SepalLengthCm",y="PetalLengthCm",hue=index,palette="c
oolwarm")
plt.grid()
plt.savefig("K-Means iris")
```

output:




```
plt.figure(figsize=(7,7))
sns.lineplot(x=np.arange(1,k+1),y=np.array(distortion),marker="o",markersize=10)
plt.xticks(np.arange(1,k+1))
plt.title("Elbow Method for Optimal k")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Distortion")
Text(0, 0.5, 'Distortion')
```

output:



b) Implementing GMM Clustering Algorithm

AIM:

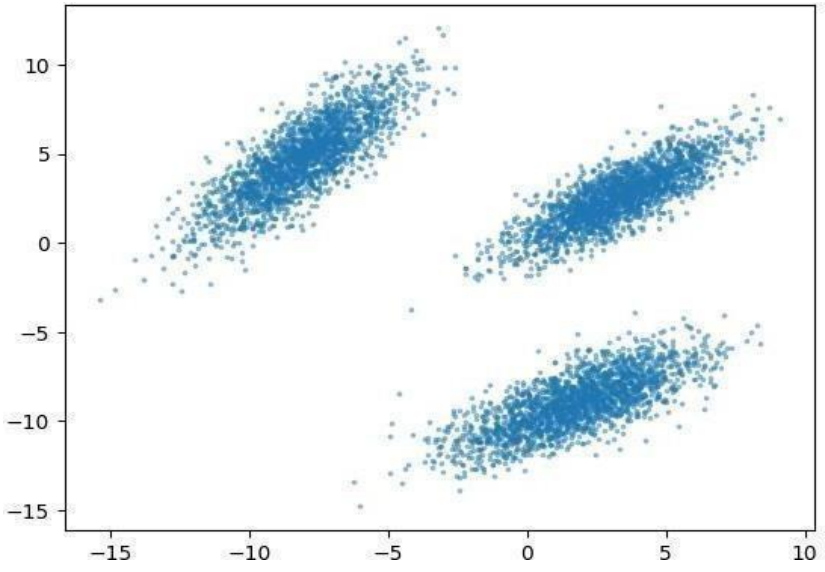
To implement GMM from scratch in python using Expectation and Maximization algorithm.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture
```

```
def gen_data(k=3, dim=2, points_per_cluster=2000, lim=[-10, 10]):
    x = []
    mean = np.random.rand(k, dim)*(lim[1]-lim[0]) + lim[0]
    for i in range(k):
        cov = np.random.rand(dim, dim+10)
        cov = np.matmul(cov, cov.T)
        _x = np.random.multivariate_normal(mean[i], cov, points_per_cluster)
        x += list(_x)
    x = np.array(x)
    if(dim == 2):
        fig = plt.figure()
        ax = fig.gca()
        ax.scatter(x[:,0], x[:,1], s=3, alpha=0.4)
        ax.autoscale(enable=True)
    return x
```

```
data = gen_data()
```

output:

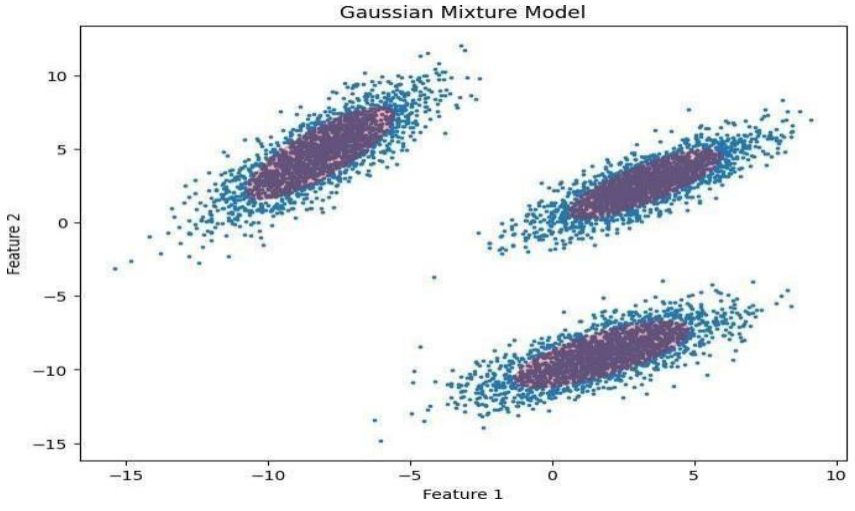


```
gmm = GaussianMixture(n_components=3)
gmm.fit(data)
```

```
plt.figure(figsize=(8, 6))
plt.scatter(data[:, 0], data[:, 1],s=3)
for covar, mean in zip(gmm.covariances_, gmm.means_):
    v, w = np.linalg.eigh(covar)
    angle = np.arctan2(w[0][1], w[0][0])
    angle = 180 * angle / np.pi
    v = 2.0 * np.sqrt(2.0) * np.sqrt(v)
    ell = plt.matplotlib.patches.Ellipse(mean, v[0], v[1],
                                         180 + angle,color='r',
                                         alpha=0.3)
    plt.gca().add_patch(ell)

plt.xlabel('Feature  1')
plt.ylabel('Feature 2')
plt.title('Gaussian Mixture Model')
plt.show()
```

output:



Conclusion:

Thus we learned how to implement and evaluate the output of the clustering using GMM via Python.

5. Classification Algorithms

Neural Network:

A neural network is a machine that is designed to model the way in which the brain performs a particular task or functions; the network is usually implemented by using electronic components or is simulated in software on a digital computer. To achieve good performance, neural networks employs a massive interconnections of simple computing cells referred to as "neurons" or "processing units". Thus the definition of neural network is "A neural network is a massively parallel distributed processor made up of simple processing units, which has a natural capability for storing experimental knowledge and making it available for use".

Backpropagation neural network (BPNN)

Backpropagation neural network (BPNN) is a multi layer feedforward neural network (i.e., propagating the error backward to adjust the weights). The basic idea is to efficiently compute partial derivatives of an approximating function f(W,X) realized by the network with respect to all the elements of the adjustable weights vector W for a given value of input vector X. Fig. 1. shows the architecture of backpropagation neural network.

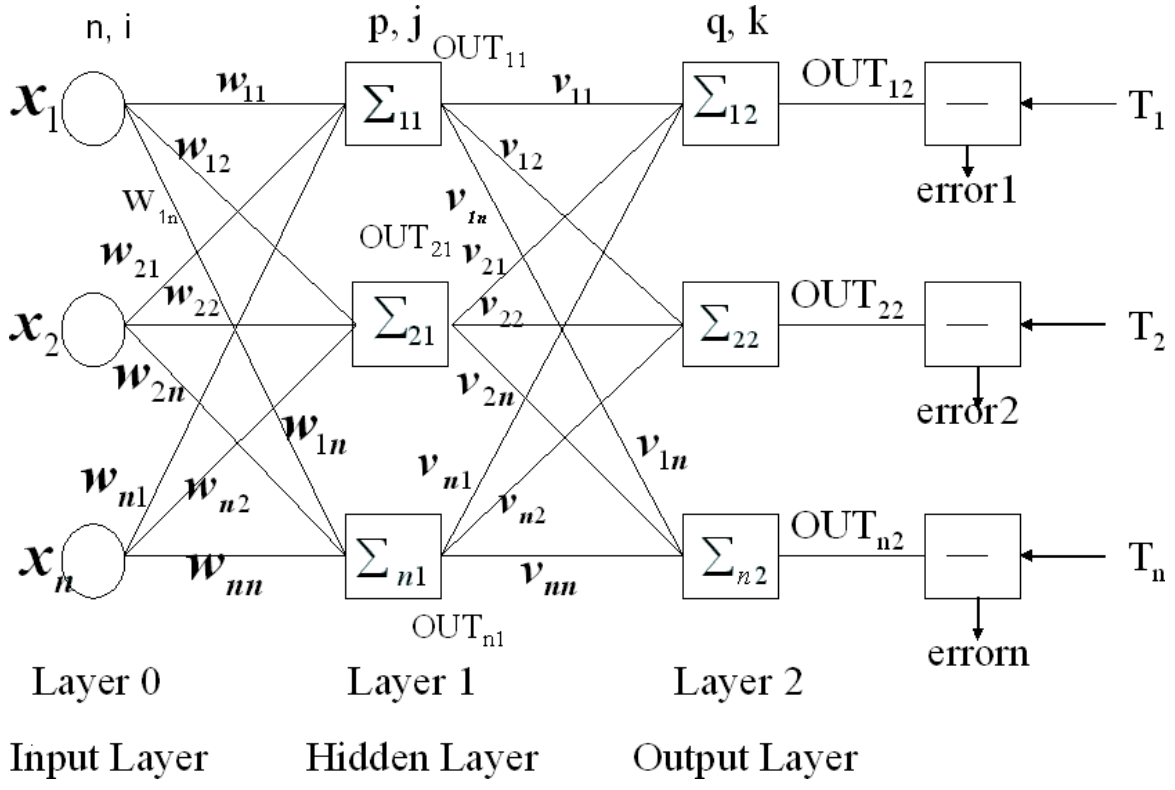


Fig.1. Architecture of Back-Propagation neural network

Backpropagation Training Algorithm:

1. Select the next training pair from the training set and apply to the network.
2. Calculate the output of the network.
3. Calculate the error between the output of the network and the desired output.
4. Adjust the weights (V,W matrix) in such a way that it minimize the error.
5. Repeat steps 1 to 4 for all the training pairs.
6. Repeat steps 1 to 5 until the network recognizes the training set or for certainnumber of iterations called epochs. 1

The activation function used by BPNN training algorithm is sigmoid or squashing or logistic function, and it is defined as

$$OUT = \frac{1}{(1 + e^{-NET})}$$

BPNN training algorithm uses the derivative of activation function and defined as

$$\frac{\partial_{OUT}}{\partial_{NET}} = OUT(1 - OUT)$$

BPNN training algorithm consists of two passes: (i)Forward pass and (ii)Reverse pass.

(i) Forward pass:

In this pass, output of the network is calculated as,

$$\begin{aligned}
&\mathrm{NET}_{1j} = x_1 w_{11} + x_2 w_{21} + \ldots + x_n \\
&\mathrm{OUT}_{1j} = 1/(1 + e^{-NET_{1j}}) \\
&\mathrm{NET}_{1k} = OUT_{11V_{11}} + OUT_{21V_{21}} + \ldots + OUT_{n1V_{n1}} \\
&\mathrm{OUT}_{1k} = 1/(1 + e^{a} NET_{1k})
\end{aligned}$$

This is repeated for all the neurons.

(ii) Reverse pass:

This pass consist of two parts. They are

(a) Adjusting the weights of the output layer:

To adjust the weights of the output layer generalized delta rule is used.

$$\delta_{qk} = OUT_{qk}(1 - OUT_{qk})(Target - OUT_{qk})$$

The new weight of the V matrix is calculated as

$$V_{pq}(n+1) = V_{pq}(n) + \eta \delta_{qk} OUT_{pj}$$

where $V_{pq}(n+1)$ is new weight,
$V_{pq}(n)$ is old weight and
η is learning or training rate coefficient.

(b) Adjusting the weights of the hidden layer:

$$\delta_{pj} = OUT_{pj}(1 - OUT_{pj}) \sum_{q=1}^{n} \delta_{qk} V_{pq}$$

Where δ_{pj} is error for neuron p in the hidden layer j,
OUT_{pj} is output of neuron p in the hidden layer j,
δqk is error neuron q in the output layer k and
V_{pq} is weight from neuron p in the hidden layer to neuron q in the output layer.

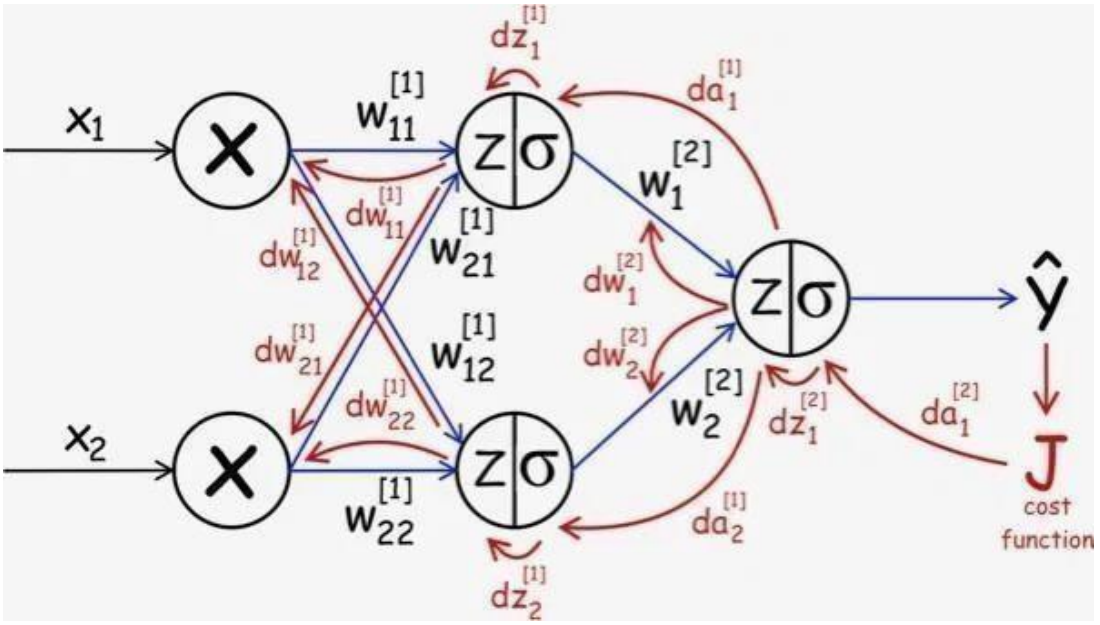
The new weight of the W matrix is calculated as

$$W_{mp}(n+1) = W_{mp} + \eta \delta_{pj} x_m$$

where $W_{mp}(n+1)$ is new weight,
$W_{mp}(n)$ is old weight and
η is learning or training rate coefficient.

With these Paramaters and functions we can implement BPNN algorithm using python.

BackPropagation implementation model in Detail



BPNN Implementation Algorthim:

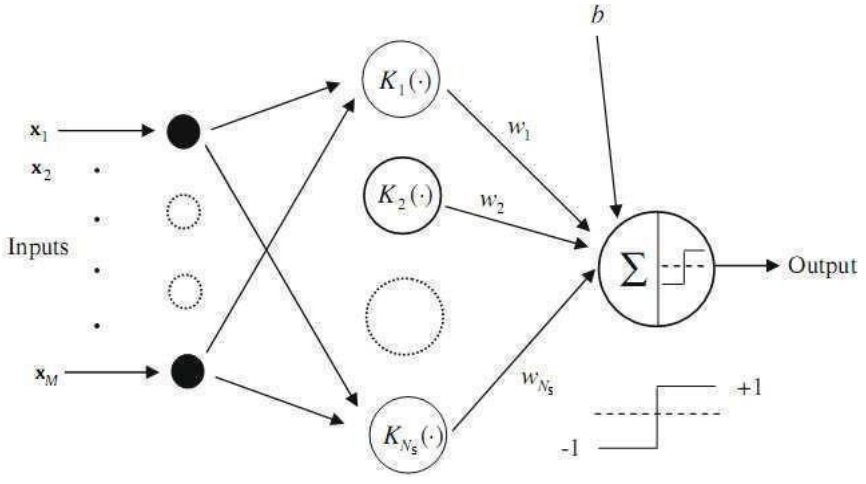
- Initialize $W^{[1]}..W^{[L]}, b^{[1]}...b^{[L]}$
- Set $A^{[0]} = X$ (Input) , $L =$ Total Layers
- Loop epoch = 1 to max iteration
  - Forward Propagation
    - Loop $l = 1$ to $L - 1$
      - $Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$
      - $A^{[l]} = g\left(b^{[l]}\right)$
      - Save $A^{[l]}, W^{[l]}$ in memory for later use
    - $Z^{[L]} = W^{[L]} A^{[L-1]} + b^{[L]}$
    - $A^{[L]} = \sigma\left(Z^{[L]}\right)$
  - Cost $J = -\frac{1}{n}\left(Y log\left(A^{[2]}\right) - (1-Y)\, log\left(1 - A^{[2]}\right)\right)$
  - Backward Propagation
    - $dA^{[L]} = -\frac{Y}{A^{[L]}} + \frac{1-Y}{1-A^{[L]}}$
    - $dZ^{[L]} = dA^{[L]} \sigma'\left(dA^{[L]}\right)$
    - $dW^{[L]} = dZ^{[L]} dA^{[L-1]}$
    - $db^{[L]} = dZ^{[L]}$
    - $dA^{[L-1]} = dZ^{[L]} W^{[L]}$
    - Loop $l = L - 1$ to 1
      - $dZ^{[l]} = dA^{[l]} g'\left(dA^{[l]}\right)$
      - $dW^{[l]} = dZ^{[l]} dA^{[l-1]}$
      - $db^{[l]} = dZ^{[l]}$
      - $dA^{[l-1]} = dZ^{[l]} W^{[l]}$
  - Update W and b
    - Loop $l = 1$ to L
      - $W^{[l]} = W^{[l]} - \alpha . dW^{[l]}$
      - $b^{[l]} = b^{[l]} - \alpha . db^{[l]}$

Support Vector Machine(SVM)

Support vector machine (SVM) is based on the principle of structural risk minimization (SRM). Like RBFNN, support vector machines can be used for pattern classification and nonlinear regression. SVM constructs a linear model to estimate the decision function using non-linear class boundaries based on support vectors. If the data are linearly separated, SVM trains linear machines for an optimal hyperplane that separates the data without error and into the maximum distance between the hyperplane and the closest training points. The training points that are closest to the optimal separating hyperplane are called support vectors. Fig shows the architecture of the SVM. SVM maps the input patterns into a higher dimensional feature space through some nonlinear mapping chosen a priori. A linear decision surface is then constructed in this high dimensional feature space. Thus, SVM is a linear classifier in the parameter space, but it becomes a nonlinear classifier as a result of the nonlinear mapping of the space of the input patterns into the high dimensional feature space, SVM also supports the kernel method also called the **kernel SVM** whichallows us to tackle non-linearity.

SVM Principle

Support vector machine (SVM) ban be used for classifying the obtained data. SVM are a set of related supervised learning methods used for classification and regression. They belong to a family of generalized linear classifiers. Let us denote a feature vector (termed as pattern) by $x = (x_1, x_2, ..., x_n)$ and its class label by y such that $y = \{-1, +1\}$. Therefore, consider the problem of separating the set of n-training patterns belonging to two classes.

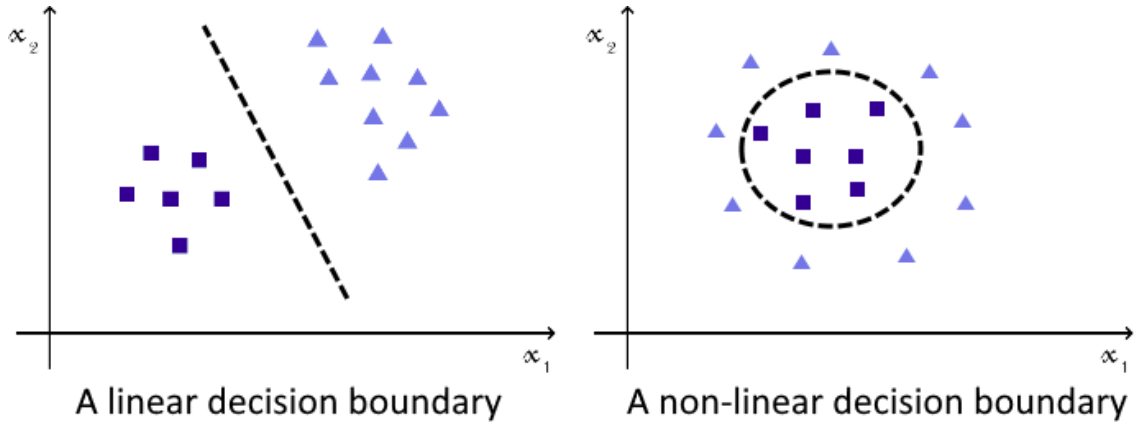


Architecture of the SVM (N s is the number of support vectors)

$$(x_i, y_i), \quad x_i \epsilon \mathbb{R}^K, \quad y = \{+1, -1\}, \quad i = 1, 2, ..., n$$

A decision function g (x) that can correctly classify an input pattern x that is not necessarily from the training set.

SVM for Linearly Separable Data



A linear SVM is used to classify data sets which are linearly separable. The SVM linear classifier tries to maximize the margin between the separating hyperplane. The patterns lying on the maximal margins are called support vectors. Such a hyperplane with maximum margin is called maximum margin hyperplane. In case of linear SVM, the discriminant function is of the form:

$$g(x) = w^t x + b$$

such that $g(x_i) \geq 0 \, for \, y_i = +1$ and $g(x_i) \leq 0 \, for \, y_i = -1$. In other words, training samples from the two different classes are separated by the hyperplane $g(x) = w^t x + b = 0$ SVM finds the hyperplane that causes the largest separation between the decision function values from the two classes. Now the total width between two margins is , which is to be maximized. Mathematically, this hyperplane can be found by minimizing the following cost function: $\frac{2}{w^t w}$

$$J(w) = \frac{1}{2} w^t w$$

Subject to separability constraints

$$g(x_i) \geq +1 \; for \; y_i = +1$$

$$g(x_i) \leq 1 \; for \; y_i = -1$$

Equivalently, these constraints can be re-written more compactly as

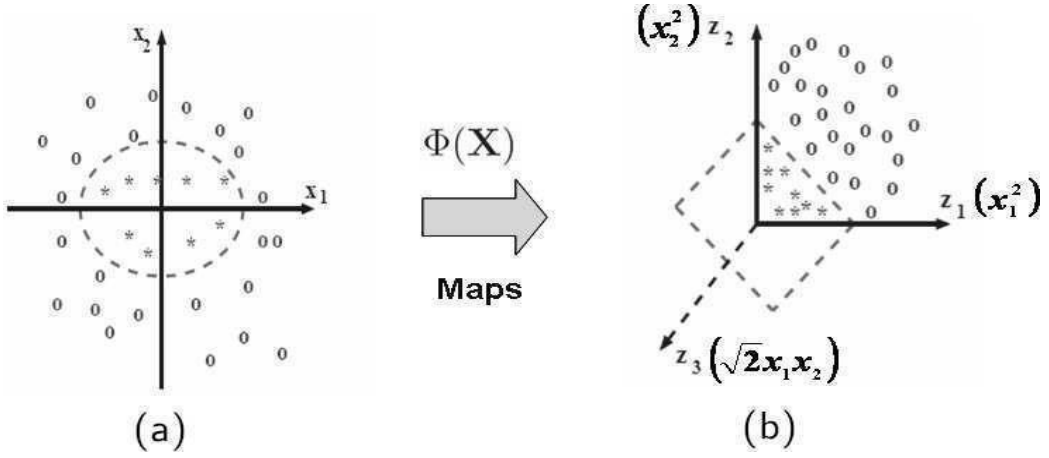
$$y_i(w^t x_i + b) \geq 1; \quad i = 1, 2, ..., n$$

For the linearly separable case, the decision rules defined by an optimal hyperplane separating the binary decision classes are given in the following equation in terms of the support vectors:

$$Y = sign\left(\sum_{i=1}^{i=N_s} y_i \alpha_i (x x_i) + b\right)$$

where Y is the outcome, y_i is the class value of the training example , and x represents the inner product. The vector corresponds to an input and the vectors x_i, $i = 1, \ldots, N_s$, are the support vectors. In Eq. 10.4, b and α i are parameters that determine the Hyperplane.

SVM for Linearly Non-separable Data



For non-linearly separable data, it maps the data in the input space into a high dimension space $x \epsilon \mathbb{R}^I \mapsto \Phi(x) \epsilon \mathbb{R}^H$ with kernel function, $\Phi(x)$ to find the separating hyperplane. A high-dimensional version of Eq. 10.4 is given as follows:

$$Y = sign\left(\sum_{i=1}^{N} y_i \alpha_i K(x, x_i) + b\right)$$

Kernel

The learning of the hyperplane in linear SVM is done by transforming the problem to higher dimension space. This is where the kernel plays role.The function K is defined as the kernel function for generating the inner products to construct machines with different types of non-linear decision surfaces in the input space.

$$K(x, x_i) = \Phi(x) \quad \Phi(x_i)$$

An example for SVM kernel function $\Phi(x)$ maps 2-dimensional input space (x_i, x_2) to higher 3-dimensional feature space $(x_1^2, x_2^2, \sqrt{2}x_1 x_2)$ The kernel function may be any of the symmetric functions that satisfy the Mercer's conditions

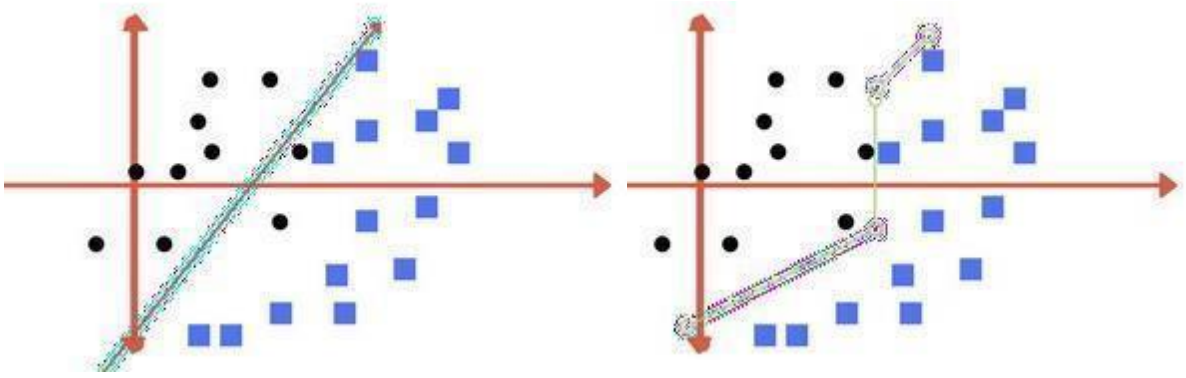
For **linear kernel** the equation for prediction for a new input using the dot product between the input (x) and each support vector (xi) is calculated

The **polynomial kernel** $K(x^T x_i + 1)^p$Polynomial and exponential kernels calculates separation line in higher dimension. This is called **kernel trick**

Tuning parameters: Regularization, Gamma and Margin. Regularization

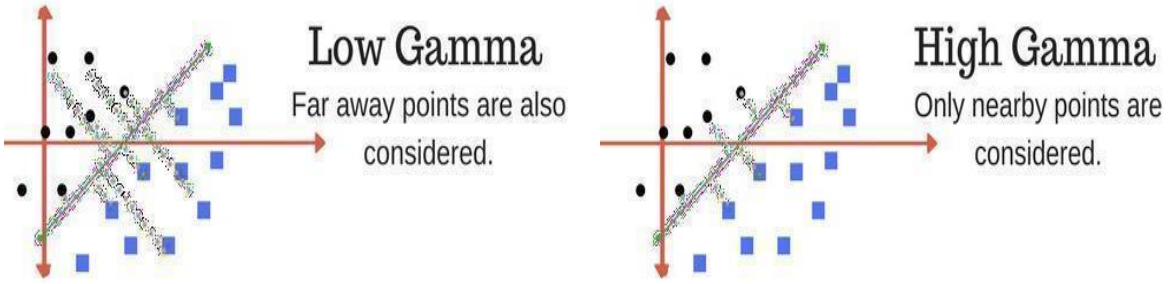
The Regularization parameter (often termed as C parameter in python's sklearn library) tells the SVM optimization how much we want to avoid misclassifying each training example.

The images below (same as image 1 and image 2 in section 2) are example of two different regularization parameter. Left one has some misclassification due to lower regularization value. Higher value leads to results like right one.



Gamma

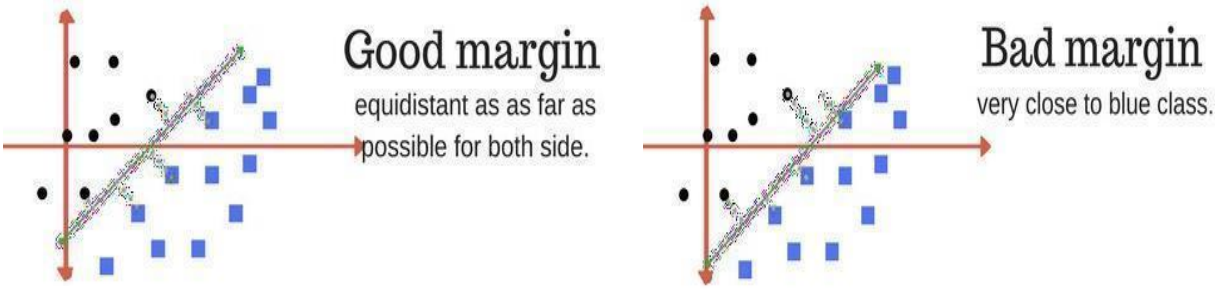
The gamma parameter defines how far the influence of a single training example reaches, with low values meaning 'far' and high values meaning 'close'. In other words, with low gamma, points far away from plausible separation line are considered in calculation for the separation line. Where as high gamma means the points close to plausible line are considered in calculation.



Margin

And finally last but very important characteristic of SVM classifier. SVM to core tries to achieve a good margin. A margin is a separation of line to the closest class points.

A good margin is one where this separation is larger for both the classes. Images below gives to visual example of good and bad margin. A good margin allows the points to be in their respective classes without crossing to other class.



SVMs, always guarantee convergence due to no restricition on dimensionality of solution space.

Implementation:

From the dataset.

1. Splitting the dataset into training and test samples.
2. Classifying the predictors and target.
3. Initializing Support Vector Machine and fitting the training data.
4. Predicting the classes for test set.
5. Attaching the predictions to test set for comparison.

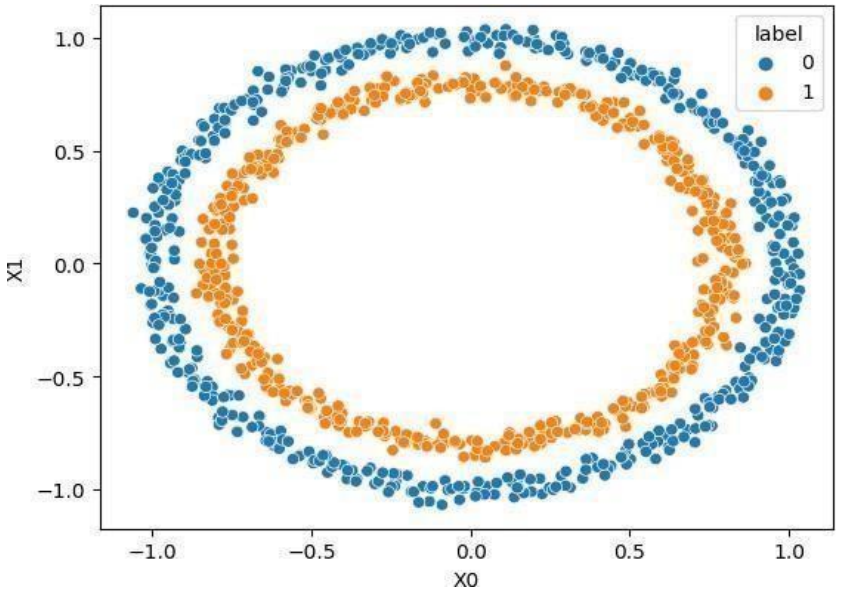
Ex No : 5 Date :	IMPLEMENTATION OF CLASSIFICATION ALGORITHMS

a) Back Propogation Neural Network

```
from sklearn.datasets import make_circles
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

n_samples = 1000
X,y = make_circles(n_samples,
                   noise = 0.03,
                   random_state=42)
circles = pd.DataFrame({"X0":X[:,0],"X1":X[:,1],"label":y})
sns.scatterplot(data=circles,x="X0",y="X1",hue="label")
```

output:




```
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

X_train,X_test,y_train,y_test = train_test_split(X,y,random_state=42,test_size=0.3)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(16,activation='relu',input_shape=(2,)),
    tf.keras.layers.Dense(8,activation='relu'),
    tf.keras.layers.Dense(1,activation='sigmoid')
])
```

```
model.compile(
    loss='binary_crossentropy',
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
    metrics=['accuracy']

)
model.summary()
```

output:

```
Model: "sequential_3"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense_9 (Dense)             (None, 16)                48

 dense_10 (Dense)            (None, 8)                 136

 dense_11 (Dense)            (None, 1)                 9

=================================================================
Total params: 193 (772.00 Byte)
Trainable params: 193 (772.00 Byte)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

```
history=model.fit(X_train,y_train,epochs=10)
```

output:

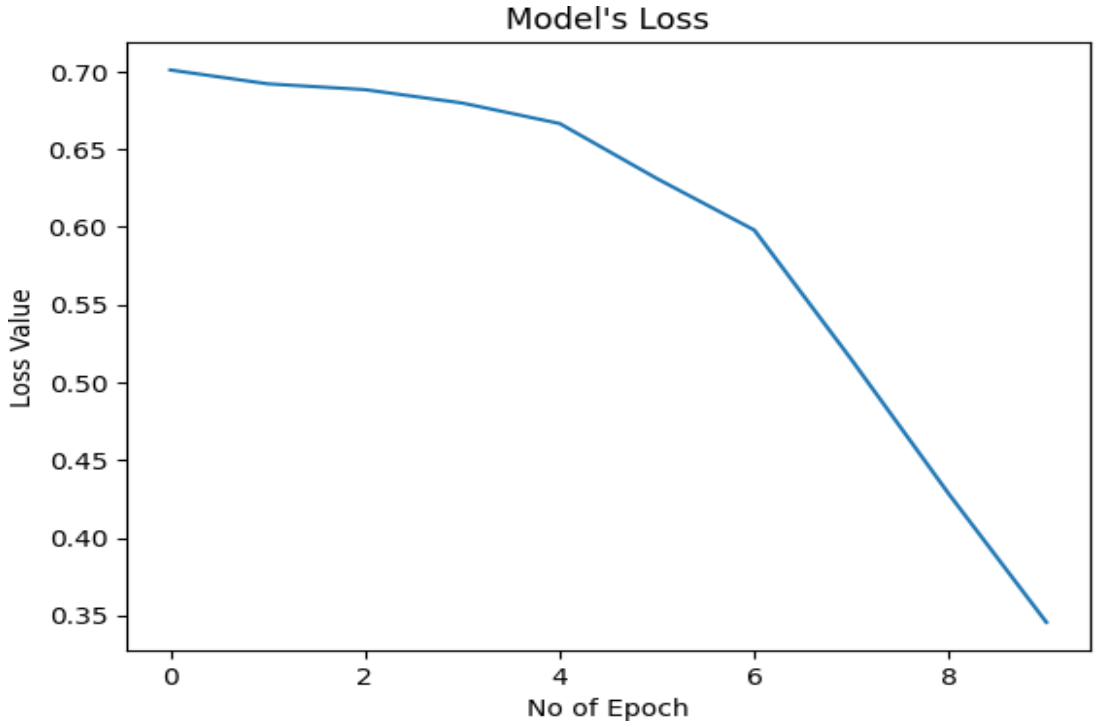
```
Epoch 1/10
22/22 [==============================] - 1s 2ms/step - loss: 0.6954 - accuracy: 0.5071
Epoch 2/10
22/22 [==============================] - 0s 2ms/step - loss: 0.6699 - accuracy: 0.6286
Epoch 3/10
22/22 [==============================] - 0s 2ms/step - loss: 0.6424 - accuracy: 0
.
.
.
Epoch 9/10
22/22 [==============================] - 0s 2ms/step - loss: 0.1730 - accuracy: 0.9929
Epoch 10/10
22/22 [==============================] - 0s 2ms/step - loss: 0.1169 - accuracy: 0.9986
```

```
model.evaluate(X_test,y_test)
```

output:

```
10/10 [==============================] - 0s 2ms/step - loss: 0.1207 - accuracy: 0.9967
[0.1207062378525734, 0.996666669845581]
```

```
plt.title("Model's Loss")
plt.plot(histoy.history['loss'])
plt.xlabel("No of Epoch")
plt.ylabel("Loss Value")
```




```
def plotq(model,X,y):
    x_min,x_max = X[:,0].min()- 0.1,X[:,0].max() + 0.1
    y_min , y_max = X[:,1].min()-0.1 ,X[:,1].max()+0.1

    xx,yy = np.meshgrid(np.linspace(x_min,x_max,100),
                        np.linspace(y_min,y_max,100))

    x_in = np.c_[xx.ravel(),yy.ravel()]
    y_pred = model.predict(x_in)

    if len(y_pred[0]) > 1:
      print("doing multiclass classification")

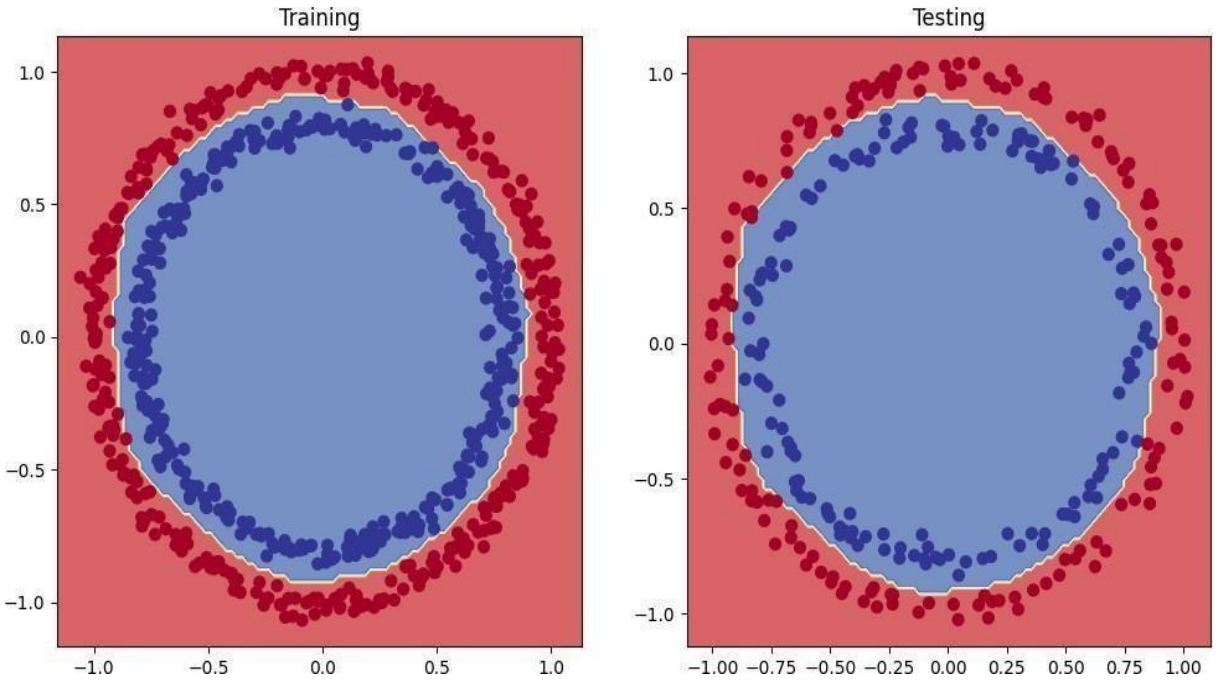
      y_pred = np.argmax(y_pred,axis=1).reshape(xx.shape)
    else:
      print("doing binary")
      y_pred = np.round(y_pred).reshape(xx.shape)

    plt.contourf(xx,yy,y_pred,cmap=plt.cm.RdYlBu,alpha=0.7)
    plt.scatter(X[:,0],X[:,1],c=y,s=40,cmap=plt.cm.RdYlBu)
    plt.xlim(xx.min(),xx.max())
    plt.ylim(yy.min(),yy.max())
```

```
plt.figure(figsize=(12,6))
plt.subplot(1,2,1)
plt.title('Training')
plotq(model,X_train,y_train)
plt.subplot(1,2,2)
plt.title('Testing')
plotq(model,X_test,y_test)
```

output:

313/313 [==============================] - 0s 821us/step
doing binary
313/313 [==============================] - 0s 801us/step
doing binary



b) Support Vector Machine

AIM :
To implement SVM using scikit-learn package.

Source code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
```

```
iris = datasets.load_iris()
X = iris.data[:, :2]
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
```

```
svm_classifier = SVC(kernel='linear', C=1)
 # Linear kernel with regularization parameter C=1
svm_classifier.fit(X_train, y_train)
y_pred = svm_classifier.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

report = classification_report(y_test, y_pred)
print("Classification Report:\n", report)
```

output:

Accuracy: 0.80

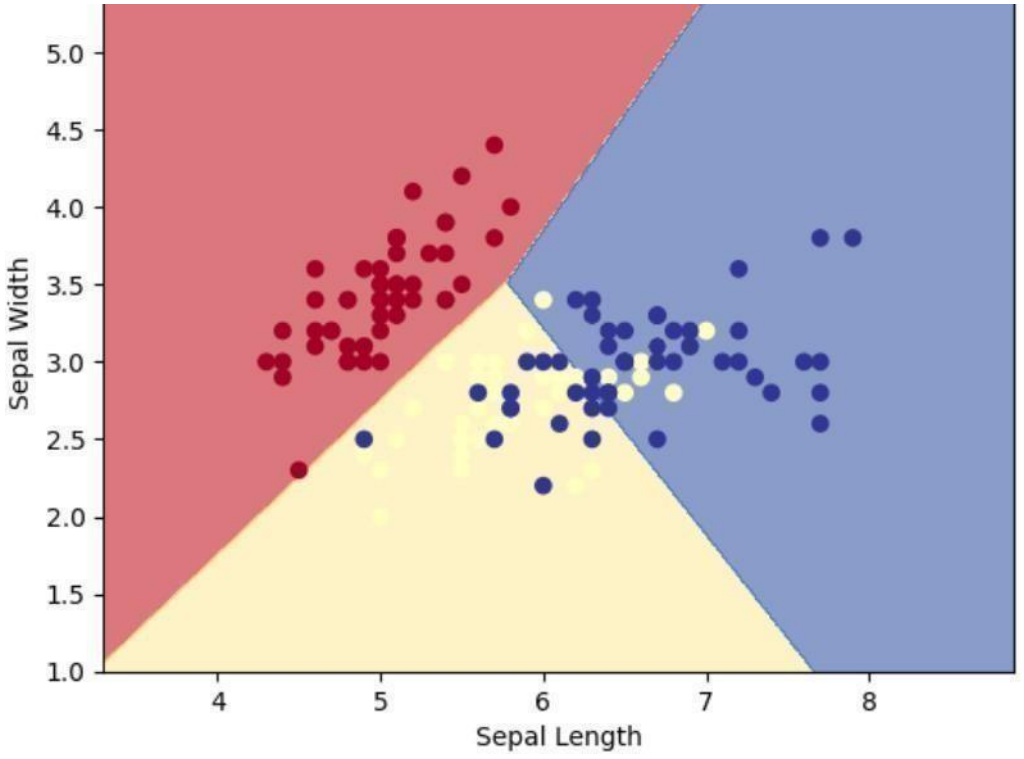
Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	0.70	0.54	0.61	13
2	0.62	0.77	0.69	13
accuracy			0.80	45
macro avg	0.78	0.77	0.77	45
weighted avg	0.81	0.80	0.80	45

```
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01), np.arange(y_min, y_max,
0.01))
Z = svm_classifier.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, cmap=plt.cm.RdYlBu, alpha=0.6)
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.RdYlBu)
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.title('SVM Classification')
plt.show()
```

output:



Conclusion:

Thus we learned to implement and evaluate the output of the classification process performed by supervised Support Vector Machine Classifier via Python.

6. Decision Tree

Brief Introduction to Decision Trees

A decision tree is a supervised machine learning algorithm that can be used for both classification and regression problems. A decision tree is simply a series of sequential decisions made to reach a specific result. The features/attributes and conditions can change based on the data and complexity of the problem but the overall idea remains the same. So, a decision tree makes a series of decisions based on a set of features/attributes present in the data. Feature importance and the sequence of attributes were checked and decided on the basis of criteria like **Gini Impurity Index** or **Information Gain.**

A decision tree is a visual representation of a problem. A decision tree helps decompose a complex problem into smaller, more manageable undertakings. This allows the decision-makers to make smaller determinations along the way to achieve the optimal overall decision. Decision tree analysis is a formal, structured approach to making decisions also intuitive to classify a pattern through a sequence of questions in which the next question is depends upon the answer to the currentquestion. But this approach is useful for nonmetric data only. Because all the questions can be askedin a yes/no (or) true/false style.

Such a sequence of questions is displayed in a directed decision tree (or) tree. This tree can be used for classification. The classification of a particular pattern begins at the root node - which asks for the value of a particular property of the pattern.The root node has different possible value and so it has different links. Based on the answer we follow the appropriate link to a subsequent node. But we follow only one link. Continue the process until we reach a leaf node, which has no further question. Each leaf node having a category label, and the test pattern is assigned the category of the leaf node reached.

Advantages of Decision Trees

Train fast, Evaluate fast, Compact models, Intelligible if small, Do feature selection, Don't use all features, Experts understand/accept them, Easy to convert to rules, Can handle missing values.

Disadvantages of Decision Trees

Not good at regression (predicting continuous values), Not good at non-axis parallel Disadvantages of Decision Trees Not good at regression (predicting continuous values), Not good at non-axis parallel splits, Trees for problems with continuous attributes can be large, Large trees are not intelligible, Split ordering often counterintuitive to experts, Not good at learning frommany inputs (e.g., pixels), The tree is very broad and depth. Therefore if we want to search any element inthe tree, it is complicated and it takes much time.

Types

Decision tree has three other names: Classification tree analysis is a term used when the predicted outcome is the class to which the data belongs.Regression tree analysis is a term used when the predicted outcome can be considered as a real number (e.g. the price of a house, or a

patients length of stay in a hospital). CART analysis is a term used to refer to both of the above procedures.

Classification and Regression Tree

CART is a binary recursive tree [16]. The tree will progressively split the set of training data into smaller and smaller subsets. If all the samples in each subset had the same category label then it would be ideal. In that case, we say that each subset was pure 1 and could terminate that portion of the tree. Usually for each branch we will decide to either stop splitting and accept an imperfect decision (or) to select another property and grow the tree further. The number of splits at a node is related to the property to be tested at each node. The root node splits the full training set. For nonnumerical data, there is a problem for geometrical interpretation of how the query at a node splits the data. But for numerical data, we can easily visualize the decision boundaries that are produced by decision tree. That decision boundaries are perpendicular to the co-ordinate axes. Therefore, a property query T at each node N makes the data to reach the immediate descendant nodes as pure as possible. Classification tree is built through a process known as binary recursive partitioning. This is an iterative process of splitting the data into partitions, and then splitting it up further on each of the branches.

Finding the Initial Split

The process starts with a training set consisting of pre-classified records. Pre-classified means that the target value, or dependent variable, has a known class or label. The goal is to build a tree that distinguishes among the classes. For simplicity, assume that there are only two target classes and that each split is binary partitioning. The splitting criterion easily generalizes to multipleclasses, and any multi-way partitioning can be achieved through repeated binary splits [16]. Every possible split is tried and considered, and the best split is the one which produces the largest decrease in diversity of the classification label within each partition (this is just another way of saying ”the increase in homogeneity”). This is repeated for all values, and the winner is chosen as the best splitter for that node. The process is continued at the next node and, in this manner, a full tree Fig. 14.5 is generated for the given training data Table. 14.1.

R_1 / w_1 (black)		R_2 / w_2 (red)	
x_1	x_2	x_1	x_2
0.15	0.83	0.10	0.29
0.09	0.55	0.08	0.15
0.29	0.35	0.23	0.16
0.38	0.70	0.70	0.19
0.52	0.48	0.62	0.47
0.57	0.73	0.91	0.27
0.73	0.75	0.65	0.90
0.47	0.06	0.75	0.36^{*} (0.32^{+})

Pruning the Tree

An alternate approach to stop splitting is pruning. In this method, a tree is gro wnfully until leaf nodes have minimum impurity. After that, all pairs of neighboring leaf nodes are considered for elimination. Any pair whose elimination yields a small increase in impurity is eliminated and the

common parent node is declared a leaf. Such merging (or) joining of the two leaf nodes is the inverse of splitting. In pruning, directly use all information in the training set. For small problems the computational cost is low. But it is not possible for larger problem, because of using all information.

Pruning is the process of removing leaves and branches to improve the performance of the decision tree when it moves from the training data (where the classification is known) to real-world applications (where the classification is unknown-it is what you are trying to predict). The tree makes the best split at the root node where there are the largest number of records and, hence, a lot of information. Each subsequent split has a smaller and less representative population with which to work. Toward the end, training records at a particular node display patterns that are peculiar only

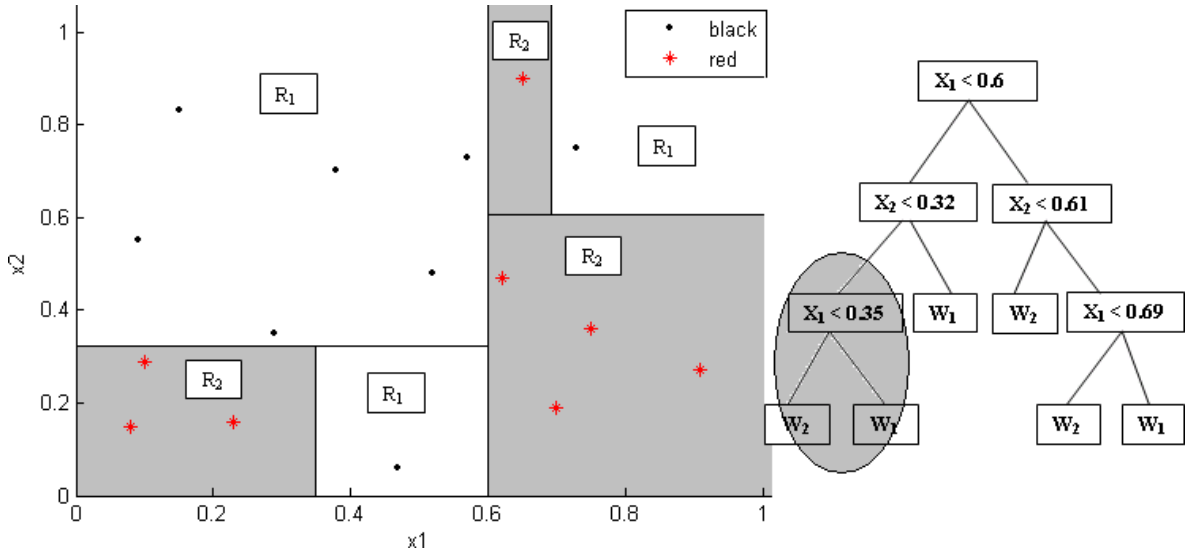


Fig.1. Decision region and unpruned Classification tree

to those records. These patterns can become meaningless and sometimes harmful for prediction if you try to extend rules based on them to larger populations. For example, say the classification tree is trying to predict height and it comes to a node containing one tall person named X and several other shorter people. It can decrease diversity at that node by a new rule saying ”people named Xare tall” and thus classify the training data.

Pruning methods solve this problem-they let the tree grow to maximum size, then remove smaller branches that fail to generalize. Rather than purity, we calculate the impurity. Let denote the impurity of a node N. If all the patterns that reach the node bear the same category label then $i(N)$ to be 0. The popular measurement of impurity is entropy impurity.

$$i(N) = -\sum_{j} P(w_j) log_2 P(w_j)$$

-where $P(w_j)$ is the fraction of patterns at node N that are in category w_j.

In Fig.1 we apply number of search of the $n-1$ positions for the x_1 feature and $n-1$ position for the x_2 feature we find the greatest reduction in the impurity occurs near $x_{1s}=0.6$, and hence this becomes the decision criterion at the root node. Then continue for each subtree until each final node represent a single category (has the lowest impurity, 0). Suppose if all the patterns are of the same category then impurity is 0. Otherwise, impurity is positive value. Therefore to choose the query that decreases the impurity as much as possible. The impurity reduction corresponds to an information obtained by the query.

Since the tree is grown from the training data set, when it has reached full structure it usually suffers from over-fitting (i.e. it is ”explaining” random elements of the training data that are not likely to be features of the larger population of data). This results in poor performance on real life data. Therefore, it has to be pruned using the validation data set. If pruning were invoked in Fig.1 the pair of leaf nodes at the left would be the first to be deleted (gray shading) because theimpurity is increased the least. If the point marked $*$ in the x_2 Fig.1 is moved slightly, the decision in the Fig.1 is moved slightly, the decision region and tree differ significantly, as shown in Fig.2

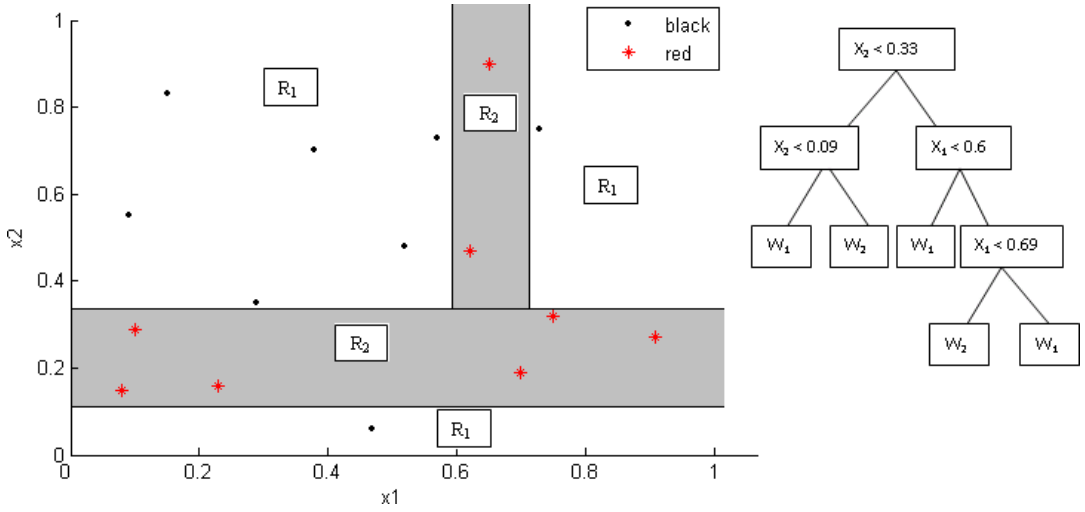


Fig.2.Decision region and classification tree

In Fig.2. we find the greatest reduction in the impurity occurs near $x_{2s}=0.33$, and hence this becomes the decision criterion at the root node

Random Forest

Random Forest is a tree-based supervised machine learning algorithm that leverages the power of multiple decision trees for making decisions. As the name suggests, it is a “forest” of trees,it is a forest of randomly created decision trees an ensemble of decision trees, usually trained with the “bagging” method. The general idea of the bagging method is that a combination of learning models increases the overall result. Each node in the decision tree works on a random subset of features to calculate the output and then combines the output of individual decision trees to generatethe final output."Tree" and "Forest," a Random Forest is essentially a collection of Decision Trees.

A decision tree is built on an entire dataset, using all the features/variables of interest, whereas a random forest randomly selects observations/rows and specific features/variables to buildmultiple decision trees from and then averages the results. After a large number of trees are built using this method, each tree "votes" or chooses the class, and the class receiving the most votes by asimple majority is the "winner" or predicted class

One big advantage of random forest is that it can be used for both classification and regression problems, which form the majority of current machine learning systems.

Working of Random Forest Algorithm

The working of Random Forest algorithm with the help of following steps −

- **Step 1** − First, start with the selection of random samples from a given dataset.
- **Step 2** − Next, this algorithm will construct a decision tree for every sample. Then it will get the prediction result from every decision tree.
- **Step 3** − In this step, voting will be performed for every predicted result.
- **Step 4** − At last, select the most voted prediction result as the final prediction
- result.The following diagram will illustrate its working −

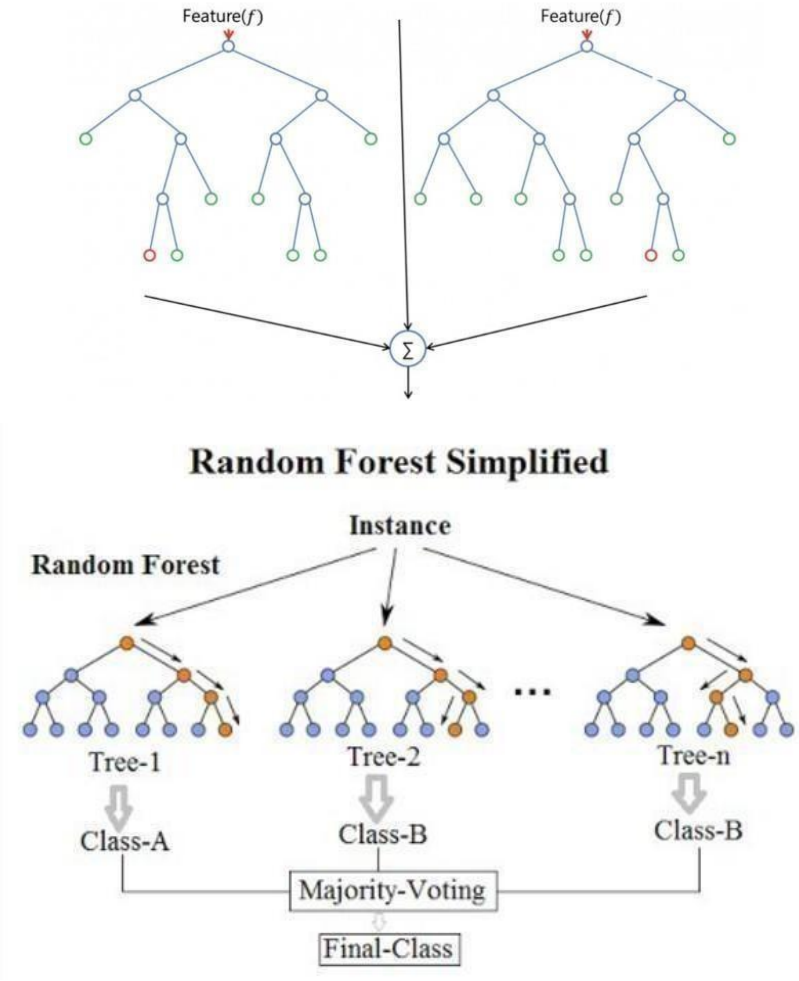


Fig.1. Random Forest

The hyperparameters in random forest are either used to increase the predictive power of the model or to make the model faster. The hyperparameters of sklearns built-in random forest function.

1. Increasing the predictive power

Firstly, there is the **n_estimators** hyperparameter, which is just the number of trees the algorithm builds before taking the maximum voting or taking the averages of predictions. In general, a higher number of trees increases the performance and makes the predictions more stable, but it also slows down the computation.

Another important hyperparameter is **max_features,** which is the maximum number of features random forest considers to split a node. Sklearn provides several options, all described in the documentation.

The last important hyperparameter is **min_sample_leaf.** This determines the minimum number of leafs required to split an internal node.

2. Increasing the model's speed

The **n_jobs** hyperparameter tells the engine how many processors it is allowed to use. If it has a value of one, it can only use one processor. A value of "-1" means that there is no limit.

The **random_state** hyperparameter makes the model's output replicable. The model will always produce the same results when it has a definite value of random_state and if it has been given the same hyperparameters and the same training data.

Lastly, there is the **oob_score** (also called oob sampling), which is a random forest cross-validation method. In this sampling, about one-third of the data is not used to train the model and can be used to evaluate its performance. These samples are called the out-of-bag samples. It's very similar to the leave-one-out-cross-validation method, but almost no additional computational burden goes along with it.

Ex no : 6
Date :

Decision Tree and Random forest using scikit Learn

AIM:

To implement Decision tree abd random forest classifier in kyphosis dataset using scikit learn.

Source Code:

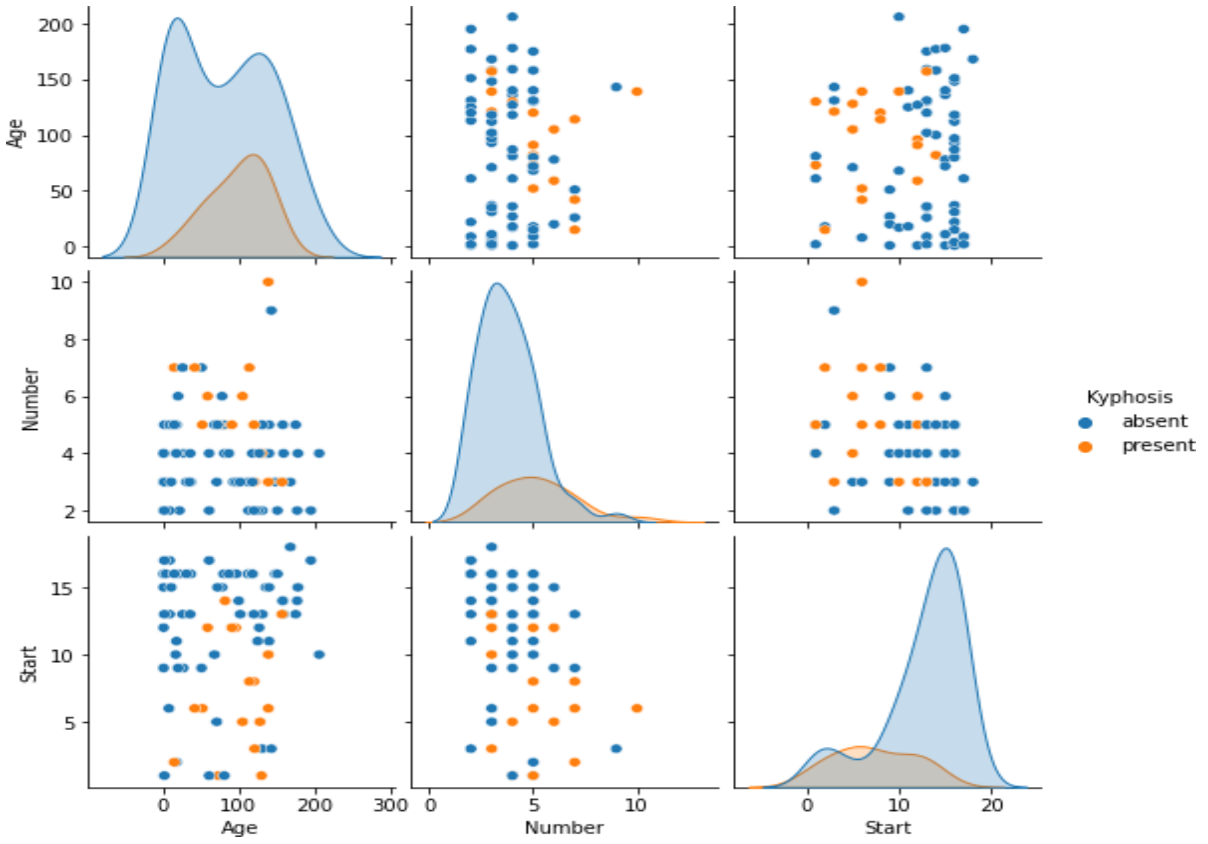
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
kyphosis_df = pd.read_csv('./datasets/kyphosis.csv')
display(kyphosis_df.head())
kyphosis_df.info()
```

output:

	Kyphosis	Age	Number	Start
0	absent	71	3	5
1	absent	158	3	14
2	present	128	4	5
3	absent	2	5	1
4	absent	1	4	15

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 81 entries, 0 to 80
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Kyphosis   81 non-null     object
 1   Age        81 non-null     int64
 2   Number     81 non-null     int64
 3   Start      81 non-null     int64
dtypes: int64(3), object(1)
memory usage: 2.7+ KB
```

```
sns.pairplot(kyphosis_df, hue = 'Kyphosis')
plt.show()
```




```
from sklearn.model_selection import train_test_split
X = kyphosis_df.drop('Kyphosis', axis = 1)
y = kyphosis_df['Kyphosis']
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.3)
```

```
from sklearn.tree import DecisionTreeClassifier
decision_tree_model = DecisionTreeClassifier()
decision_tree_model.fit(X_train, y_train)
decision_tree_y_pred = decision_tree_model.predict(X_test)
```

```
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
print("Classification Report for Decision Tree Model using Kyphosis Data:")
print(classification_report(y_test,decision_tree_y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, decision_tree_y_pred))
```

output:

```
Classification Report for  Decision Tree Model  using Kyphosis Data:
              precision    recall  f1-score   support

      absent       0.74      0.74      0.74        19
     present       0.17      0.17      0.17         6

    accuracy                           0.60        25
   macro avg       0.45      0.45      0.45        25
weighted avg       0.60      0.60      0.60        25

Confusion Matrix:
[[14  5]
 [ 5  1]]
```

```
from sklearn.ensemble import RandomForestClassifier
random_forest_model = RandomForestClassifier()
random_forest_model.fit(X_train, y_train)
random_forest_y_pred = random_forest_model.predict(X_test)
print("Classification Report for Random Forest Model using Kyphosis Data:")
print(classification_report(y_test, random_forest_y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, random_forest_y_pred))
```

output:

```
Classification Report for Random Forest Model using Kyphosis Data:
              precision    recall  f1-score   support

      absent       0.78      0.95      0.86        19
     present       0.50      0.17      0.25         6

    accuracy                           0.76        25
   macro avg       0.64      0.56      0.55        25
weighted avg       0.71      0.76      0.71        25

Confusion Matrix:
[[18 1]
 [ 5  1]]
```

Conclusion:

Thus we learned to implement and evaluate the output of the Decision Treeand Random forest model via Python.

7. Convolution Neural Network

The field of machine learning has taken a dramatic twist in recent times, with the rise of the Artificial Neural Network (ANN). These biologically inspired computational models are able to far exceed the performance of previous forms of artificial intelligence in common machine learning tasks. One of the most impressive forms of ANN architecture is that of the Convolutional Neural Network (CNN). CNNs are primarily used to solve difficult image-driven pattern recognition tasks and with their precise yet simple architecture, offers a simplified method of getting started with ANNs. In the domain of Computer Vision, Deep Learning architecture has been constructed and perfected with time, primarily over **Convolutional Neural Network** algorithm

A **Convolutional Neural Network (ConvNet/CNN)** is a Deep Learning algorithm which can take in an input image, assign importance (learnable weights and biases) to various aspects/objects in the image and be able to differentiate one from the other. The pre-processing required in a ConvNet is much lower as compared to other classification algorithms. While in primitive methods filters are hand-engineered, with enough training, ConvNets have the ability to learn these filters/characteristics.

The architecture of a ConvNet is analogous to that of the connectivity pattern of Neurons in the Human Brain and was inspired by the organization of the Visual Cortex. Individual neurons respond to stimuli only in a restricted region of the visual field known as the Receptive Field. A collection of such fields overlap to cover the entire visual area.

The CNN is a combination of two basic building blocks:

1. **The Convolution Block** — Consists of the Convolution Layer and the Pooling Layer. This layer forms the essential component of *Feature-Extraction*
2. **The Fully Connected Block** — Consists of a fully connected simple neural network architecture. This layer performs the task of *Classification* based on the input from the convolutional block.

Layers used to build ConvNets

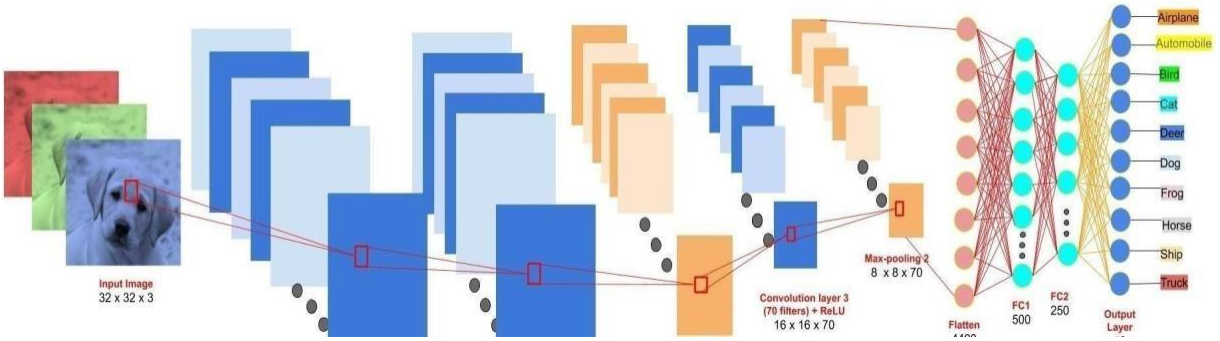
A covnets is a sequence of layers, and every layer transforms one volume to another through differentiable function.

Types of layers:

Let's take an example by running a covnets on of image of dimension 32 x 32 x 3.

1. **Input Layer:** This layer holds the raw input of image with width 32, height 32 and depth 3.
2. **Convolution Layer: The convolution layers are the main powerhouse of a CNN model**. This layer computes the output volume by computing dot product between all filters and image patch. Suppose we use total 12 filters for this layer we'll get output volume of dimension 32 x 32 x 12.

CNN- Architecture Deployment for CIFAR-10 Object Classification

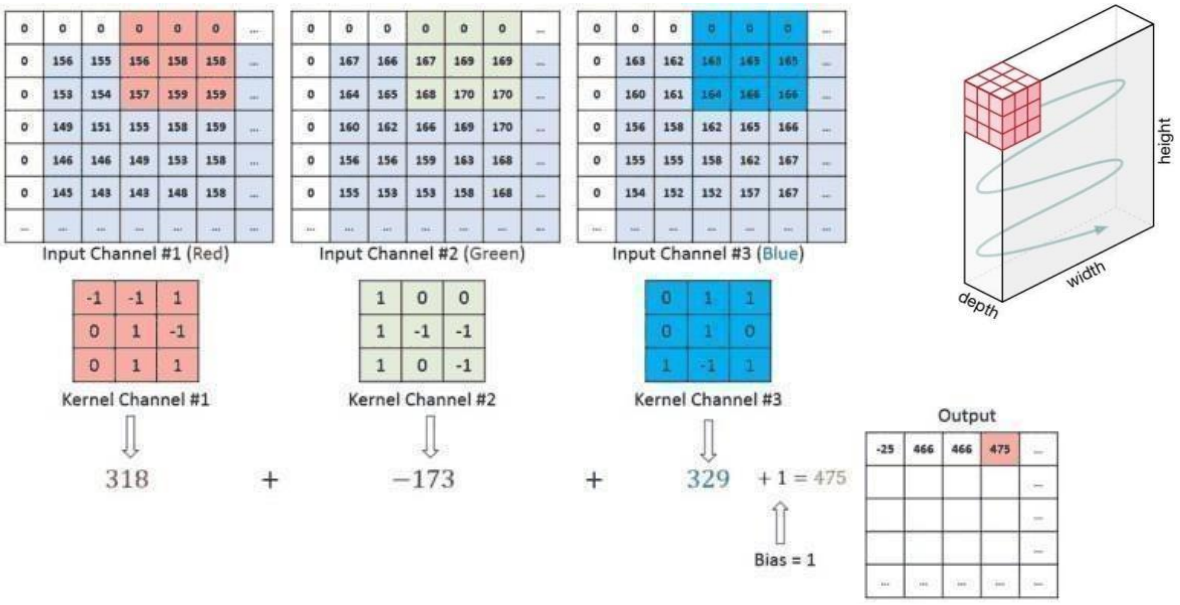


No. of Trainable Parameters/weights:

CONV1:	CONV2	CONV3	FC1	FC2	SoftMax
((3 x (3x3)) +1)x25 700	((25 x(3x3) +1) x50 11300	((50x(3x3) +1)x70 31570	(500 x 4480) + 500 2240500	(250 x 500) + 250 125250	(10 x 250) + 10 2510

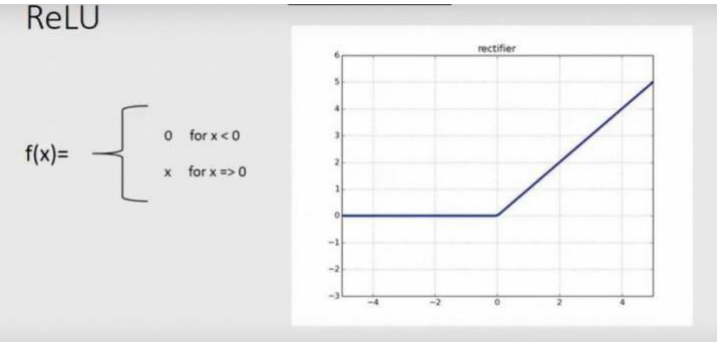
Convolution Kernel size : 3 x 3
Max Pooling Window size: 2 x 2

Total Trainable Parameters: 24,11,830

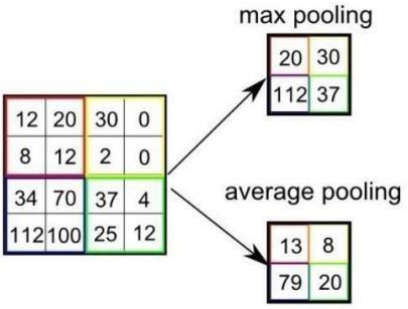


3. **Activation Function Layer:** This layer will apply element wise activation function to the output of convolution layer. Some common activation functions are RELU: max(0, x), Sigmoid:

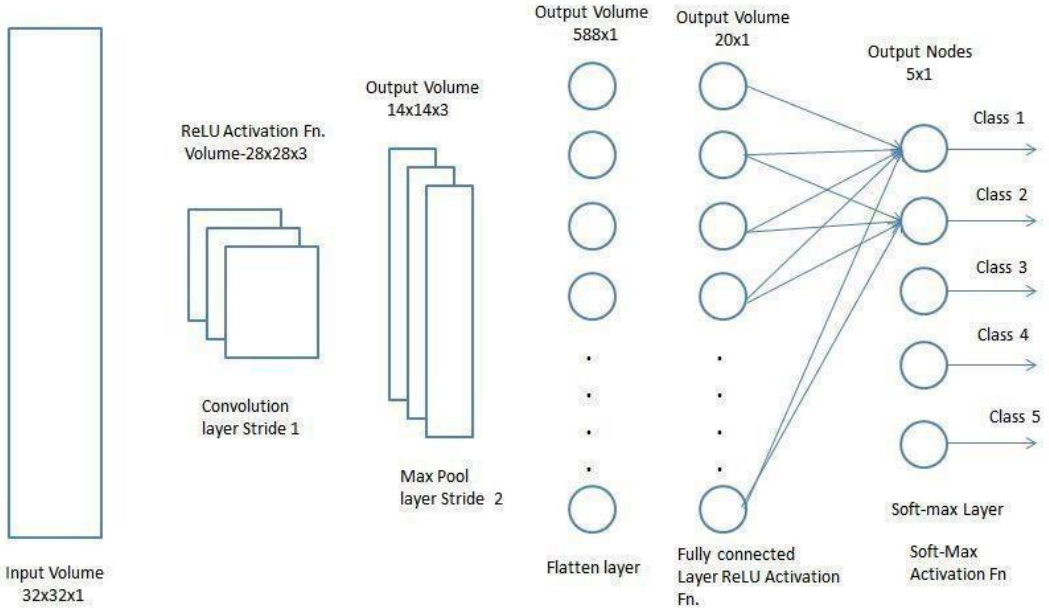
$1/(1+e^{-x})$, , tanh, Leaky RELU, etc. The volume remains unchanged hence output volume will have dimension 32 x 32 x 12.



4. **Pool Layer:** This layer is periodically inserted in the covnets and its main function is to reduce the size of volume which makes the computation fast reduces memory and also prevents from overfitting. Two common types of pooling layers are **max pooling** and **average pooling**. If we use a max pool with 2 x 2 filters and stride 2, the resultant volume will be of dimension 16x16x12.



5. **Fully-Connected Layer:** This layer is regular neural network layer which takes input from the previous layer and computes the class scores and outputs the 1-D array of size equal to the number of classes.



Training the CNN

The CNN is trained over a larger no of image during the training phase and for each time, the error generated is fed back into to CNN to adjust the values of the matrices in each layer. The basic working is similar to that of the training of a Simple Neural Network. The concept is **backpropagation.** The mathematical background of backpropagation and the adjustment of values will be studied in future .

backpropagation applied to every iteration of training. Over a series of epochs, the model is able to distinguish between dominating and certain low-level features in images and classify them using the **Softmax Classification** technique

There are various architectures of CNNs available which have been key in building algorithms which power and shall power AI as a whole in the foreseeable future. Some of them have beenlisted below:

1. LeNet
2. AlexNet
3. VGGNet

Deep learning Framework keras

Keras is a deep learning API written in Python, running on top of the machine learning platform TensorFlow. It was developed with a focus on enabling fast experimentation. *Being able to go from idea to result as fast as possible is key to doing good research.*

TensorFlow 2: an approchable, highly-productive interface for solving machine learning problems, with a focus on modern deep learning. It provides essential abstractions and building blocks for developing and shipping machine learning solutions with high iteration velocity.

Building CNN with keras

2D convolution layer (e.g. spatial convolution over images).

This layer creates a convolution kernel that is convolved with the layer input to produce a tensor of outputs. If use_bias is True, a bias vector is created and added to the outputs. Finally, if activationis not None, it is applied to the outputs as well.

When using this layer as the first layer in a model, provide the keyword argument input_shape (tuple of integers, does not include the sample axis), e.g. input_shape=(128, 128, 3) for 128x128 RGB pictures in data_format="channels_last".

```
tf.keras.layers.Conv2D(
    filters,
    kernel_size,
    strides=(1, 1),
    padding="valid",
    data_format=None,
    dilation_rate=(1, 1),
    groups=1,
    activation=None,
    use_bias=True,
```

```
    kernel_initializer="glorot_uniform",
    bias_initializer="zeros",
    kernel_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    bias_constraint=None,
    **kwargs
)
```

Key Arguments to discuss

- **filters**: Integer, the dimensionality of the output space (i.e. the number of output filters in theconvolution).
- **kernel_size**: An integer or tuple/list of 2 integers, specifying the height and width of the 2D convolution window. Can be a single integer to specify the same value for all spatial dimensions.
- **strides**: An integer or tuple/list of 2 integers, specifying the strides of the convolution along the height and width. Can be a single integer to specify the same value for all spatial dimensions. Specifying any stride value != 1 is incompatible with specifying any `dilation_rate` value != 1.
- **padding**: one of `"valid"` or `"same"` (case-insensitive). `"valid"` means no padding. `"same"` results in padding evenly to the left/right or up/down of the input such that output has the same height/width dimension as the input.
- **activation**: Activation function to use. If you don't specify anything, no activation is applied (see `keras.activations`).
- **use_bias**: Boolean, whether the layer uses a bias vector.
- **kernel_initializer**: Initializer for the `kernel` weights matrix.
- **bias_initializer**: Initializer for the bias vector (see `keras.initializers`).
- **kernel_regularizer**: Regularizer function applied to the `kernel` weights matrix .

MaxPooling2D class

```
tf.keras.layers.MaxPooling2D(
     pool_size=(2, 2),strides=None,padding="valid",data_format=None,**kwargs)
```

Max pooling operation for 2D spatial data.

Downsamples the input representation by taking the maximum value over the window defined by `pool_size` for each dimension along the features axis. The window is shifted by `strides` in each dimension. The resulting output when using "valid" padding option has a shape(number of rows or columns) of: `output_shape = (input_shape - pool_size + 1) / strides)`

The resulting output shape when using the "same" padding option is: `output_shape = input_shape / strides`

relufunction

```
tf.keras.activations.relu(x, alpha=0.0, max_value=None, threshold=0)
```

Applies the rectified linear unit activation function.

With default values, this returns the standard ReLU activation: `max(x, 0)`, the element-wise maximum of 0 and the input tensor.

`softmax` function

```
tf.keras.activations.softmax(x, axis=-1)
```

Softmax converts a real vector to a vector of categorical probabilities.

The elements of the output vector are in range (0, 1) and sum to 1.

`compile` method

Configures the model for training.

```
Model.compile(
    optimizer="rmsprop",
    loss=None,
    metrics=None,
    loss_weights=None,
    weighted_metrics=None,
    run_eagerly=None,
    steps_per_execution=None,
    **kwargs
)
```

FITmethod

```
Trains the model for a fixed number of epochs (iterations on a dataset)

Model.fit(
    x=None,
    y=None,
    batch_size=None,
    epochs=1,
    verbose=1,
    callbacks=None,
    validation_split=0.0,
    validation_data=None,
    shuffle=True,
    class_weight=None,
    sample_weight=None,
    initial_epoch=0,
    steps_per_epoch=None,
    validation_steps=None,
    validation_batch_size=None,
    validation_freq=1,
    max_queue_size=10,
    workers=1,
    use_multiprocessing=False,
)
```

`evaluate` method

```
Returns the loss value & metrics values for the model in test mode.

Model.evaluate(
    x=None,
    y=None,
    batch_size=None,
    verbose=1,
```

```
    sample_weight=None,
    steps=None,
    callbacks=None,
    max_queue_size=10,
    workers=1,
    use_multiprocessing=False,
    return_dict=False,
)
```

`predict` method

Generates output predictions for the input samples.

```
Model.predict(
    x,
    batch_size=None,
    verbose=0,
    steps=None,
    callbacks=None,
    max_queue_size=10,
    workers=1,
    use_multiprocessing=False,
)
```

Ex no : 7
Date :

Image classification using CNN

AIM:

To build a CNN architecture to classify handwritten digits using the MNIST dataset.

ABOUT THE DATASET:

MNIST ("Modified National Institute of Standards and Technology") is the de facto “hello world” dataset of computer vision.

The dataset is formatted as csv files in which each row has the flattened pixels of an image and the class label for the image.

Neural Network Architecture

Layer (type)	Output Shape
Conv2D	(None, 26, 26, 64)
Conv2D	(None, 24, 24, 64)
MaxPooling2D	(None, 12, 12, 64)
BatchNormalization	(None, 12, 12, 64)
Conv2D	(None, 10, 10, 128)
Conv2D	(None, 8, 8, 128)
MaxPooling2D	(None, 4, 4, 128)
BatchNormalization	(None, 4, 4, 128)
Conv2D	(None, 2, 2, 256)
MaxPooling2D	(None, 1, 1, 256)
Flatten	(None, 256)
BatchNormalization	(None, 256)
Dense	(None, 512)
Dense	(None, 10)

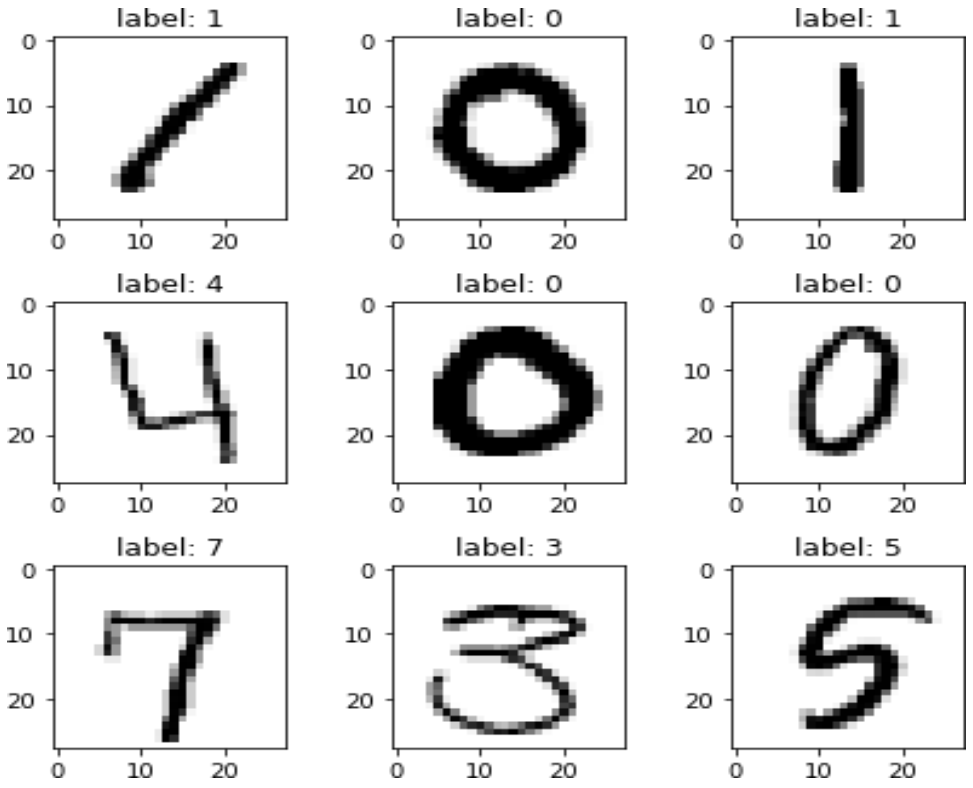
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics  import confusion_matrix

import tensorflow as tf
from  tensorflow.keras.models  import  Sequential
from tensorflow.keras.layers import (
    Conv2D,    MaxPooling2D,
    Dense,   Dropout,   Flatten,
    BatchNormalization
)
from keras.utils.np_utils import to_categorical
import joblib
```

```
train = pd.read_csv('datasets/mnist/train.csv')
test = pd.read_csv('datasets/mnist/test.csv')
X_train = train.drop(['label'], axis=1).values
y_train = train['label'].values
X_test  = test.values
X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)
X_train = X_train /255
X_test  = X_test/255
y_train  = to_categorical(y_train)
```

```
fig, axes = plt.subplots(3, 3, figsize=(6, 6))
for i, ax in enumerate(axes.flat):
    ax.imshow(X_train[i].squeeze(), cmap='binary')
    digit = y_train[i].argmax()
    ax.set(title = f"label: {digit}")
fig.tight_layout()
```

output:




```
def get_model():
    model=Sequential()

    model.add(Conv2D(filters=64, kernel_size = (3,3), activation="relu",
input_shape=(28,28,1)))
    model.add(Conv2D(filters=64, kernel_size = (3,3), activation="relu"))

    model.add(MaxPooling2D(pool_size=(2,2)))
    model.add(BatchNormalization())
    model.add(Conv2D(filters=128, kernel_size = (3,3), activation="relu"))
    model.add(Conv2D(filters=128, kernel_size = (3,3), activation="relu"))

    model.add(MaxPooling2D(pool_size=(2,2)))
    model.add(BatchNormalization())
    model.add(Conv2D(filters=256, kernel_size = (3,3), activation="relu"))

    model.add(MaxPooling2D(pool_size=(2,2)))

    model.add(Flatten())
    model.add(BatchNormalization())
    model.add(Dense(512,activation="relu"))

    model.add(Dense(10,activation="softmax"))

    model.compile(loss="categorical_crossentropy", optimizer="adam",
metrics=["accuracy"])
    return model
```

```
tf.random.set_seed(0)
model = get_model()
print(model.summary())
```

output:

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 26, 26, 64)        640
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 24, 24, 64)        36928
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 12, 12, 64)        0
_________________________________________________________________
batch_normalization (BatchNo (None, 12, 12, 64)        256
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 10, 10, 128)       73856
_________________________________________________________________
conv2d_3 (Conv2D)            (None, 8, 8, 128)         147584
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 4, 4, 128)         0
_________________________________________________________________
batch_normalization_1 (Batch (None, 4, 4, 128)         512
_________________________________________________________________
conv2d_4 (Conv2D)            (None, 2, 2, 256)         295168
_________________________________________________________________
max_pooling2d_2 (MaxPooling2 (None, 1, 1, 256)         0
_________________________________________________________________
flatten (Flatten)            (None, 256)               0
_________________________________________________________________
batch_normalization_2 (Batch (None, 256)               1024
_________________________________________________________________
dense (Dense)                (None, 512)               131584
_________________________________________________________________
dense_1 (Dense)              (None, 10)                5130
=================================================================
Total params: 692,682
Trainable params: 691,786
Non-trainable params: 896
_________________________________________________________________
None
```

```
history = model.fit(
    X_train,   y_train,
    batch_size = 64,
    epochs     =     20,
    validation_split=.2
)
tf.keras.models.save_model(model,"models/mnist_cnn.h5")
joblib.dump(history.history, "models/mnist_cnn.history")
```

Use only the below lines if model is not re trained

```
model = tf.keras.models.load_model("models/mnist_cnn.h5")
history = joblib.load("models/mnist_cnn.history")
```

output:

```
Epoch 1/20
525/525 [==============================] - 14s 22ms/step - loss: 0.1081 - accuracy
: 0.9670 - val_loss: 0.0950 - val_accuracy: 0.9710
Epoch 2/20
525/525 [==============================] - 11s 21ms/step - loss: 0.0453 - accuracy
: 0.9861 - val_loss: 0.0567 - val_accuracy: 0.9837
Epoch 3/20
525/525 [==============================] - 11s 21ms/step - loss: 0.0345 - accuracy
: 0.9893 - val_loss: 0.0435 - val_accuracy: 0.9887
Epoch 4/20
525/525 [==============================] - 11s 21ms/step - loss: 0.0214 -
.
.
.
Epoch 19/20
525/525 [==============================] - 11s 21ms/step - loss: 0.0099 - accuracy
: 0.9971 - val_loss: 0.0334 - val_accuracy: 0.9926
Epoch 20/20
525/525 [==============================] - 11s 20ms/step - loss: 0.0050 - accuracy
: 0.9983 - val_loss: 0.0518 - val_accuracy: 0.9920
```

```
fig, ax = plt.subplots(1,2, figsize=(15,5))
ax[0].plot(history['loss'], color='b', label="Training loss")
ax[0].plot(history['val_loss'], color='r', label="validation loss",axes =ax[0])
ax[0].legend()

ax[1].plot(history['accuracy'], color='b', label="Training accuracy")
ax[1].plot(history['val_accuracy'], color='r',label="Validation accuracy")
ax[1].legend()
```

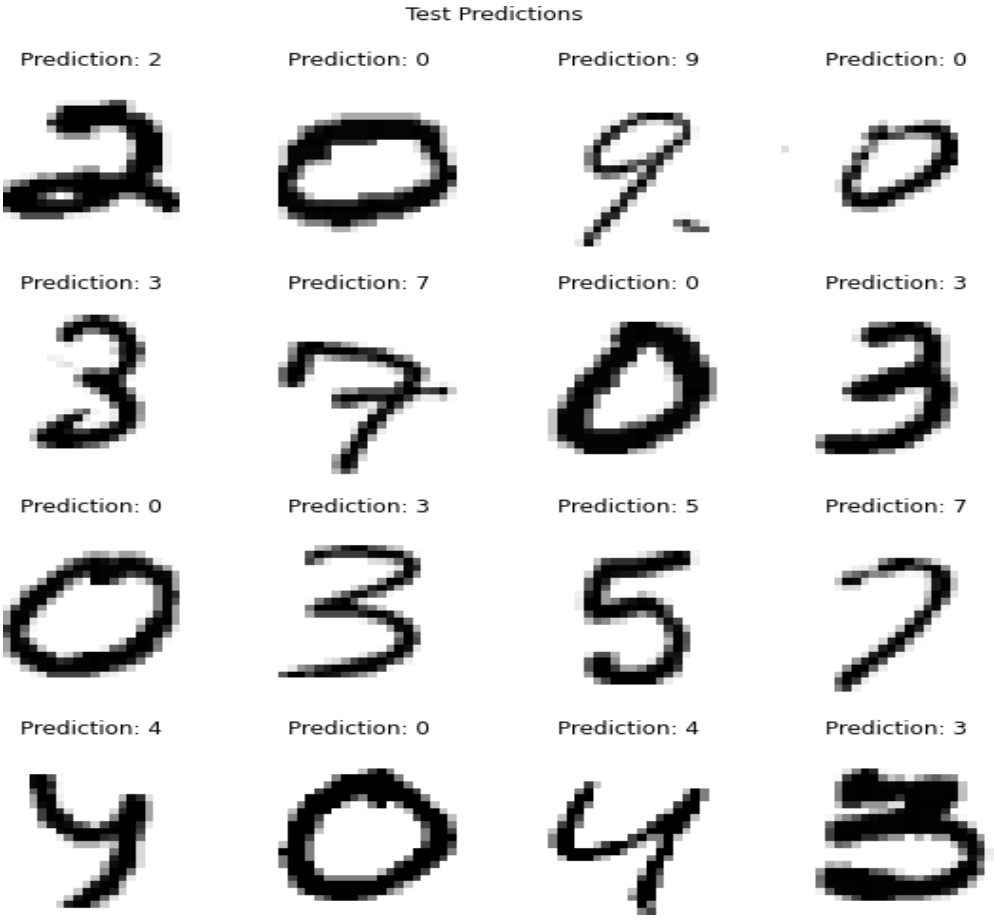
```
<matplotlib.legend.Legend at 0x7f8a2daea750>
```




```
y_pred = model.predict(X_test)
fig, axis = plt.subplots(4, 4, figsize=(8,10))
for i, ax in enumerate(axis.flat):
    ax.imshow(X_test[i].squeeze(), cmap='binary')
    ax.set(title = f"Prediction: {y_pred[i].argmax()}");
```

```
    ax.axis("off")
fig.suptitle("Test Predictions")
fig.tight_layout(rect=[0, 0.05, 1, 0.95])
```

output:



Conclusion:

Thus we learned to implement Convolution Neural Network with Keras Deep Learning Framework via Python.

8. Sequence Prediction Using Recurrent Neural Network (RNN)

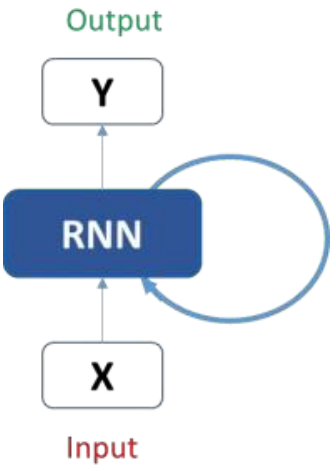
Recurrent Neural Networks.

A **recurrent neural network** (**RNN**) is a class of artificial neural networks where connections between nodes form a directed graph along a temporal sequence. This allows it to exhibit temporal dynamic behavior. Derived from feedforward neural networks, RNNs can use theirinternal state (memory) to process variable length sequences of inputs. This makes them applicable to tasks such as unsegmented, connected handwriting recognition or speech recognition structured sequence of data.

The term "recurrent neural network" is used indiscriminately to refer to two broad classes of networks with a similar general structure, where one is finite impulse and the other is infinite impulse. Both classes of networks exhibit temporal dynamic behavior. A finite impulse recurrent network is a directed acyclic graph that can be unrolled and replaced with a strictly feedforward neural network, while an infinite impulse recurrent network is a directed cyclic graph that can notbe unrolled.

Both finite impulse and infinite impulse recurrent networks can have additional stored states, and the storage can be under direct control by the neural network. The storage can also be replaced by another network or graph, if that incorporates time delays or has feedback loops. Such controlled states are referred to as gated state or gated memory, and are part of long short-term memory networks (LSTMs) and gated recurrent units. This is also called Feedback Neural Network (FNN).

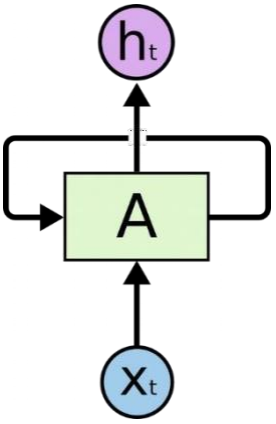
The concept we can lean on when faced with time sensitive data – Recurrent Neural Networks (RNN),A typical RNN looks like this:



This may seem intimidating at first. But once we unfold it, things start looking a lot simpler:

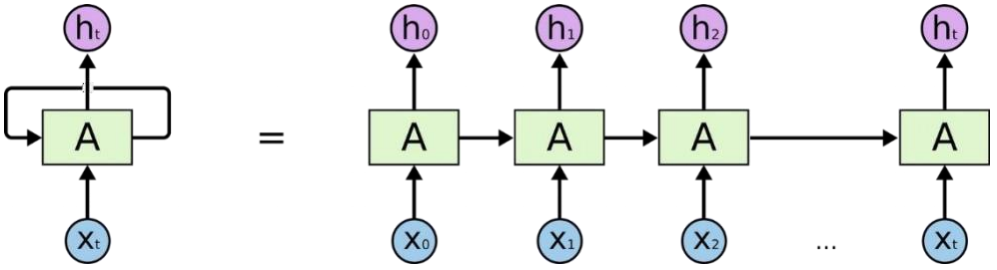
Understanding LSTM Networks

Recurrent Neural Networks with loops in them, allowing information to persist.



Recurrent Neural Networks have loops.

In the above diagram, a chunk of neural network, **A**, looks at some input $\mathbf{X_t}$ and outputs a value $\mathbf{h_t}$. A loop allows information to be passed from one step of the network to the next. These loops make recurrent neural networks seem kind of mysterious. However, it turns out that they aren't all that different than a normal neural network. A recurrent neural network can be thought of as multiple copies of the same network, each passing a message to a successor. Consider what happens if we unroll the loop:



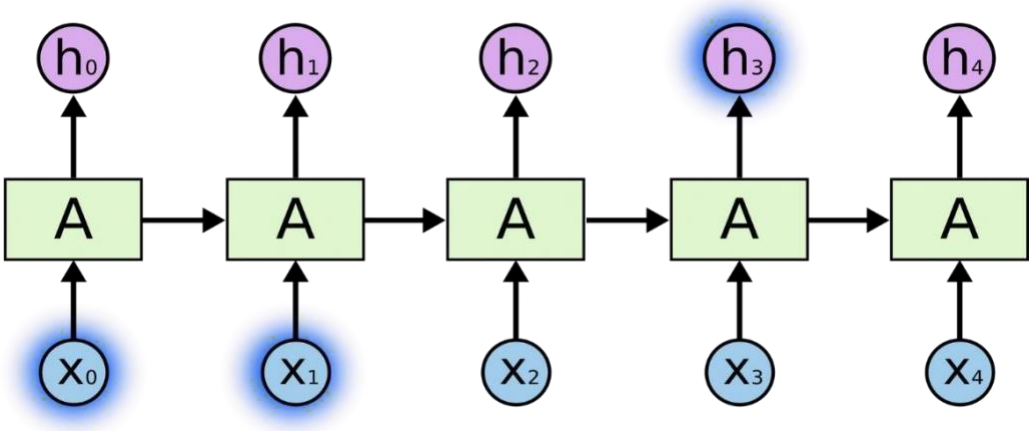
An unrolled recurrent neural network.

This chain-like nature reveals that recurrent neural networks are intimately related to sequences and lists. They're the natural architecture of neural network to use for such data.

In the last few years, there have been incredible success applying RNNs to a variety of problems: speech recognition, language modeling, translation, image captioning etc. Essential to these successes is the use of "LSTMs," a very special kind of recurrent neural network which works, for many tasks, much better than the standard version. Almost all exciting results based on recurrent neural networks are achieved with them. It's these LSTMs we will explore.

The Problem of Long-Term Dependencies

One of the appeals of RNNs is the idea that they might be able to connect previousinformation to the present task, such as using previous video frames might inform the understandingof the present frame. Sometimes, we only need to look at recent information to perform the present task. For example, consider a language model trying to predict the next word based on the previous ones. If we are trying to predict the last word in "the clouds are in the *sky*," we don't need any further context – it's pretty obvious the next word is going to be sky. In such cases, where the gap between the relevant information and the place that it's needed is small, RNNs can learn to use the past information.



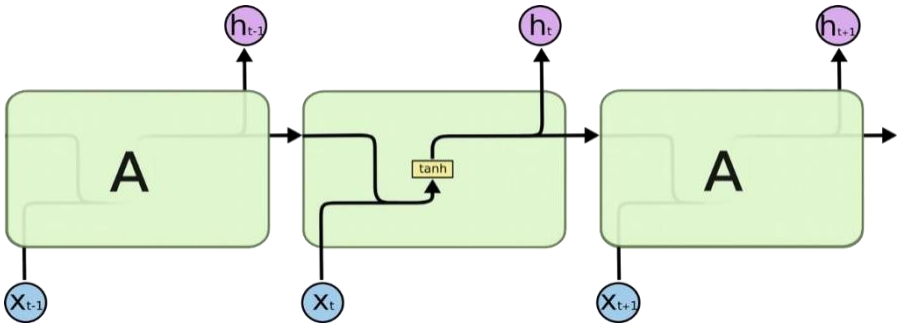
But there are also cases where we need more context. Consider trying to predict the last word in the text "I grew up in France… I speak fluent *French*." Recent information suggests that the next word is probably the name of a language, but if we want to narrow down which language, we need the context of France, from further back. It's entirely possible for the gap between the relevant information and the point where it is needed to become very large. Unfortunately, as that gap grows, RNNs become unable to learn to connect the information.

In theory, RNNs are absolutely capable of handling such "long-term dependencies." A human could carefully pick parameters for them to solve toy problems of this form. But in practice, RNNs don't seem to be able to learn them, it might be difficult. Thankfully, LSTMs don't have this problem.

LSTM Networks

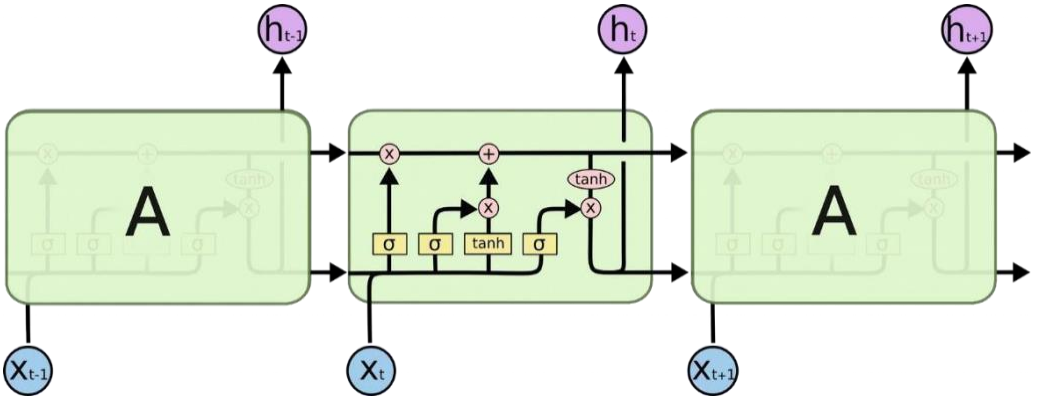
Long Short Term Memory networks – usually just called "LSTMs" – are a special kind of RNN, capable of learning long-term dependencies. They were introduced by Hochreiter & Schmidhuber (1997), this network model work tremendously well on a large variety of problems, and are now widely used. LSTMs are explicitly designed to avoid the long-term dependency problem. Remembering information for long periods of time is practically their default behavior.

All recurrent neural networks have the form of a chain of repeating modules of neural network. In standard RNNs, this repeating module will have a very simple structure, such as a single **tanh** layer.



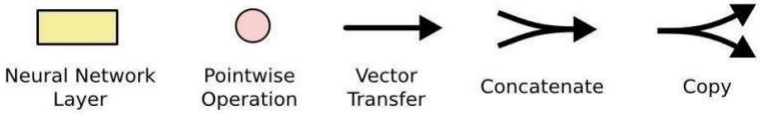
The repeating module in a standard RNN contains a single layer.

LSTMs also have this chain like structure, but the repeating module has a different structure. Instead of having a single neural network layer, there are four, interacting in a very special way.



The repeating module in an LSTM contains four interacting layers.

LSTM diagram notation used.

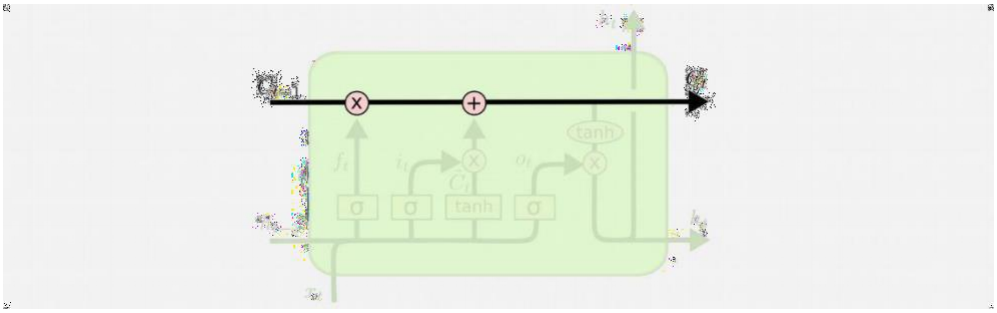


In the above diagram, each line carries an entire vector, from the output of one node to the inputs of others. The pink circles represent pointwise operations, like vector addition, while the yellow boxes are learned neural network layers. Lines merging denote concatenation, while a line forking denote its content being copied and the copies going to different locations.

The Core Idea Behind LSTMs

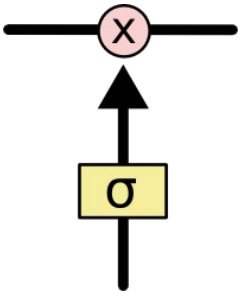
The key to LSTMs is the **cell state**, the horizontal line running through the top of the diagram. The cell state is kind of like a conveyor belt. It runs straight down the entire chain, with only some minor linear interactions. It's very easy for information to just flow along it unchanged.

Cell state:



The LSTM does have the ability to remove or add information to the cell state, carefully regulated by structures called **gates**. Gates are a way to optionally let information through. They are composed out of a sigmoid neural net layer and a pointwise multiplication operation.

Gate:

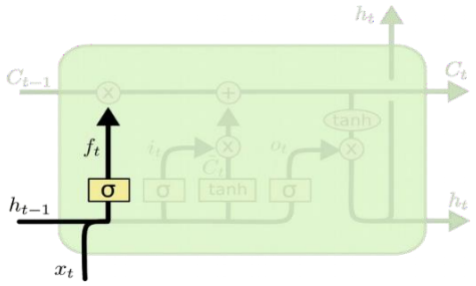


The sigmoid layer outputs numbers between zero and one, describing how much of each component should be let through. A value of zero means "let nothing through," while a value of one means "let everything through" An LSTM has three of these gates, to protect and control the cell state.

Step-by-Step LSTM Walk Through

The first step in our LSTM is to decide what information we're going to throw away from the cell state. This decision is made by a sigmoid layer called the "**forget gate layer**." It looks at h_{t-1} and x_t, and outputs a number between 0 and 1 for each number in the cell state C_{t-1}. A '1' represents "completely keep this" while a '0' represents "completely get rid of this."

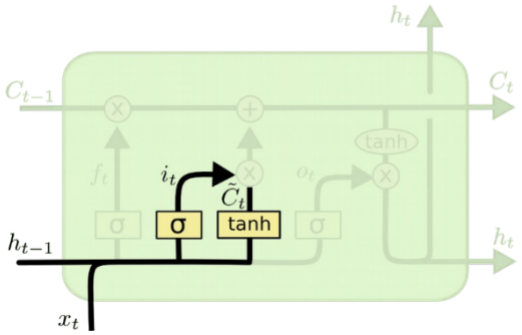
Let's go back to our example of a language model trying to predict the next word based on all the previous ones. In such a problem, the cell state might include the gender of the present subject, so that the correct pronouns can be used. When we see a new subject, we want to forget the gender of the old subject.



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

The **next step** is to decide **what new information we're going to store in the cell state**. This has two parts. First, a sigmoid layer called the "input gate layer" decides which values we'll update. Next, a tanh layer creates a vector of new candidate values, C_t, that could be added to the state. In the next step, we'll combine these two to create an update to the state.

In the example of our language model, we'd want to add the gender of the new subject to the cell state, to replace the old one we're forgetting.

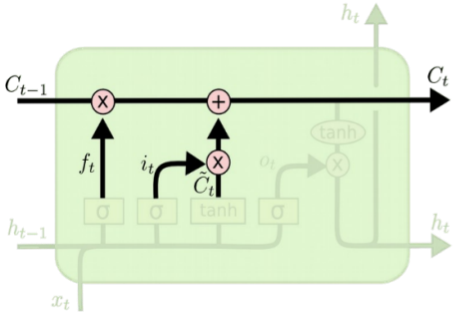


$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

It's now **time to update the old cell state**, C_{t-1}, into the new cell state C_t. The previous steps already decided what to do, we just need to actually do it.

We multiply the old state by f_t, forgetting the things we decided to forget earlier. Then we add $i_t * C_t$. This is the new candidate values, scaled by how much we decided to update each state value.

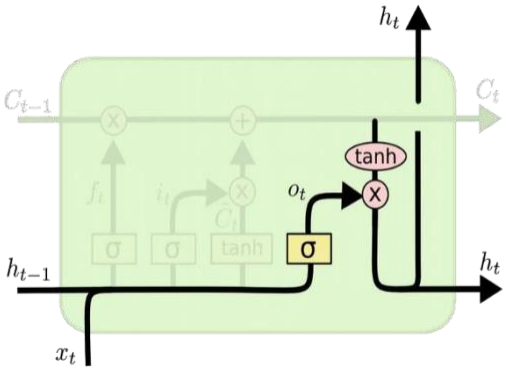
In the case of the language model, this is where we'd actually drop the information about the old subject's gender and add the new information, as we decided in the previous steps.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Finally, we need to decide what we're going to output. This output will be based on our cell state, but will be a filtered version. First, we run a sigmoid layer which decides what parts of the cell state we're going to output. Then, we put the cell state through . (to push the values to be between -1 and 1) and multiply it by the output of the sigmoid gate, so that we only output the parts we decided to.

For the language model example, since it just saw a subject, it might want to output information relevant to a verb, in case that's what is coming next. For example, it might output whether the subject is singular or plural, so that we know what form a verb should be conjugated into if that's what follows next.



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$
$$h_t = o_t * \tanh\left(C_t\right)$$

Remarkable results people are achieving with RNNs. Essentially all of these are achieved using LSTMs. They really work a lot better for most tasks

LSTMs were a big step in what we can accomplish with RNNs. There is a next step and it's attention!" The idea is to let every step of an RNN pick information to look at from some larger collection of information. Xu, *et al.* (2015) manuscript as starting point if you want to explore attention! There's been a number of really exciting results using attention, and it seems like a lot more are around the corner.

Ex no : 8
Date :

Prediction using LSTM

AIM:
To predict an arithemetic sequence using LSTM with tensorflow and keras.

Modules used:

Modules	Version
tensorflow	2.6.0
Numpy	1.19.5
Pandas	1.3.0
matplotlib	3.4.3
scikit-learn	0.24.2

Neural Network Architeture:

Layer (type)	Output Shape
LSTM	(None, 50)
Dense	(None, 1)

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import LSTM,Dense
from tensorflow.keras.optimizers import Adam,Adadelta
from sklearn.model_selection import train_test_split
```

```
X = np.arange(1,50)
Y = X*15
print("Sequence:",X,Y, sep="\n")
X = X.reshape(-1,1,1)
Y = Y.reshape(-1,1)
X_train,X_test,Y_train,Y_test = train_test_split(X,Y,shuffle=False)
```

Output:

```
Sequence:
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48
 49]
[ 15  30  45  60  75  90 105 120 135 150 165 180 195 210 225 240 255 270
 285 300 315 330 345 360 375 390 405 420 435 450 465 480 495 510 525 540
 555 570 585 600 615 630 645 660 675 690 705 720 735]
```

```
def get_model():
    model = Sequential()
    model.add(LSTM(50, activation='relu', input_shape=(1,1)))
    model.add(Dense(1))
    model.compile(optimizer=Adadelta(learning_rate=.03), loss='mse')
    return model
```

```
tf.random.set_seed(0)
model = get_model()
print(model.summary())
```

output:

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
lstm (LSTM)                  (None, 50)                10400
_________________________________________________________________
dense (Dense)                (None, 1)                 51
=================================================================
Total params: 10,451
Trainable params: 10,451
Non-trainable params: 0
_________________________________________________________________
None
```

```
model.fit(X_train, Y_train, epochs=2000, validation_split=0.2, batch_size=10)
tf.keras.models.save_model(model,"./models/seq_pred_model.h5")
model = tf.keras.models.load_model("./models/seq_pred_model.h5")
```

```
Epoch 1/2000
3/3 [==============================]  - 1s 119ms/step - loss: 62007.9180 -  val_loss
: 239098.1562
Epoch 2/2000
3/3 [==============================]  - 0s 12ms/step - loss: 62005.2695 - val_loss:
239088.6094
Epoch 3/2000
3/3 [==============================]  - 0s 12ms/step - loss: 62002.8125 - val_loss:
239079.7344
Epoch 4/2000
3/3 [==============================]  - 0s 11ms/step - loss: 62000.4727 - val_loss:
239071.0938
Epoch 5/2000
3/3 [==============================]  - 0s 11ms/step - loss: 61998.1875 - val_loss:
239062.3438
.
.
Epoch 1996/2000
```

```
3/3 [==============================]  - 0s 29ms/step - loss: 0.3635 - val_loss: 0.0
179
Epoch 1997/2000
3/3 [==============================]  - 0s 23ms/step - loss: 0.3630 - val_loss: 0.0
189
Epoch 1998/2000
3/3 [==============================]  - 0s 15ms/step - loss: 0.3628 - val_loss: 0.0
159
Epoch 1999/2000
3/3 [==============================]  - 0s 16ms/step - loss: 0.3618 - val_loss: 0.0
151
Epoch 2000/2000
3/3 [==============================]  - 0s 25ms/step - loss: 0.3618 - val_loss: 0.0
146
```

```
import pandas as pd
Y_pred = model.predict(X_test, verbose=0)
Y_diff = Y_test - Y_pred
pd.DataFrame(np.hstack([Y_test,Y_pred,Y_diff]),columns=["Actual","Predicted","Diffe
rence"])
```

output:

	Actual	Predicted	Difference
0	555.0	554.936829	0.063171
1	570.0	569.935181	0.064819
2	585.0	584.928589	0.071411
3	600.0	599.916504	0.083496
4	615.0	614.898804	0.101196
5	630.0	629.875366	0.124634
6	645.0	644.846375	0.153625
7	660.0	659.811462	0.188538
8	675.0	674.770752	0.229248

	Actual	Predicted	Difference
9	690.0	689.724365	0.275635
10	705.0	704.672180	0.327820
11	720.0	719.614685	0.385315
12	735.0	734.551636	0.448364

Conclusion:

Thus we have learned to implement and evaluate the Recurrent neural network by predicting numerical seqence output via DL keras Framework in Python.

9. Isolated word Speech Recognition

Introduction:

Speech is a natural mode of communication among human beings. Speech signal caries information about the message to be conveyed, speaker identity and language information. For communication among human beings, there is no need for speech processing, since they areendowed with both speech production and perception mechanisms. But, if a machine is placed inthe communication chain, it needs speech processing because it does not have the knowledge of production and perception. All the information required to perform the basic speech processing tasks is implicitly present in the speech. The fundamental issue in speech processing is how to extract specific features to perform the desired speech processing tasks.

In machine learning domain Speech recognition, computer **speech recognition**, or **speech- to-text**, is a capability which enables a program to process human **speech** into a written format. Automatic Speech Recognition (ASR) has historically been a driving force behind many machine learning (ML) techniques, including the ubiquitously used hidden Markov model, discriminative learning, Bayesian learning, and adaptive learning. Moreover, ML can and occasionally does use ASR as a large-scale, realistic application to rigorously test the effectiveness of a given technique, and to inspire new problems arising from the inherently sequential nature of speech. New insight from modern Deep Learning (DL) methodology shows great promise to advance the state-of-the-art in ASR technology.

Conventional Speech Recognition

Speech recognition systems consider that the speech signal is a realization of some message encoded as a sequence of one or more symbols. The essential goal is to "decode"this message and then convert it either into writing or into commands to be processed.

Types of Speech Recognition

Speech recognition systems can be divided into distinct classes by describing what types of utterances they can recognize. These classes are identified as the following:

Connected Words:

Connected word systems (or more precisely 'connected utterances'), allow apportioned utterances to be 'run-together' with a minimal pause between them.

Continuous Speech:

Continuous speech recognizers let users speak almost naturally, while the computer detects the content. Basically, it is a computer dictation.

Spontaneous Speech:

This can be thought of as speech that is natural sounding and not rehearsed. An ASR system for such a speech deals with a variety of natural speech features i.e. words being run together," ums"," ahs", and even slight stutters.

Isolated Words:

Isolated word recognizers usually need each utterance to have silence on both sides of the sample window. The user speaks individual words or phrases and the recognizer accepts single words or a single utterance at a time.

These systems have “Listen/Not-Listen” states, where they require the speaker to wait(pause) between utterances. The discrete utterance is dealt with in two implicit assumptions. The first assumption is that the speech consists of a signal that is going to be recognized as a complete entity with no explicit knowledge for the phonetic content of the word/phrase. The second assumption is that each spoken word/phrase has an apparently defined beginning and ending point. In this excercise, an isolated word speech recognition system was developed to recognize two spoken words

Digitizing Speech Signals

For processing speech signal, the acoustic variations (pressure variations) are to be represented in digital domain. This can be done as follows: A microphone is used to pickup these acoustic variations (air pressures) and convert them into equivalent analog electrical variations. This analog electrical signal is converted to digital signal by sampling followed by quantization according to nyquist criteria of sampling frequency.

Sampling the signal

An audio signal is a continuous representation of amplitude as it varies with time. Here, time can even be in picoseconds. That is why an audio signal is an analog signal. Analog signals are memory hogging since they have an infinite number of samples and processing them is highly computationally demanding. Therefore, we need a technique to convert analog signals to digital signals so that we can work with them easily. **Sampling the signal** is a process of converting an analog signal to a digital signal by selecting a certain number of samples per second from the analog signal. We are converting an audio signal to a discrete signal through sampling so that it can be stored and processed efficiently in memory. able to reconstruct an almost similar audio wave even after sampling the analog signal since we have chosen a high sampling rate.

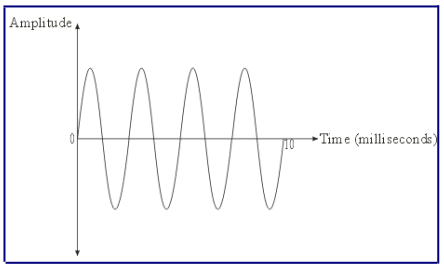
Feature Extraction Techniques for an Audio Signal

The first step in speech recognition is to extract the features from an audio signal which we will input to our model later. There are different ways of extracting features from the audio signal.

Time-domain

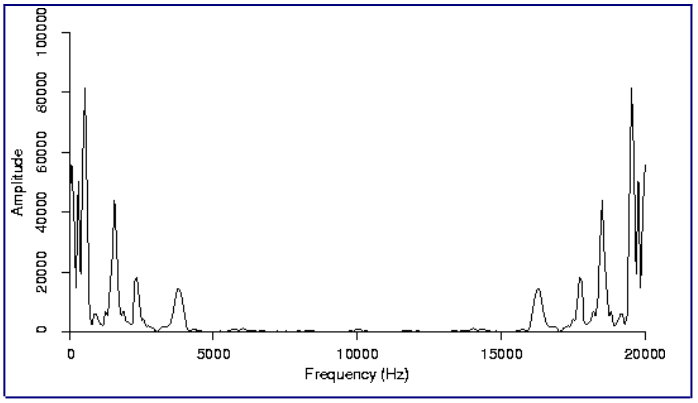
Here, the audio signal is represented by the amplitude as a function of time. In simple words, it is a plot between amplitude and time. The features are the amplitudes which are recorded at different time intervals.

The limitation of the time-domain analysis is that it completely ignores the informationabout the rate of the signal which is addressed by the frequency domain analysis. So let’s discuss that in the next section.



Frequency domain

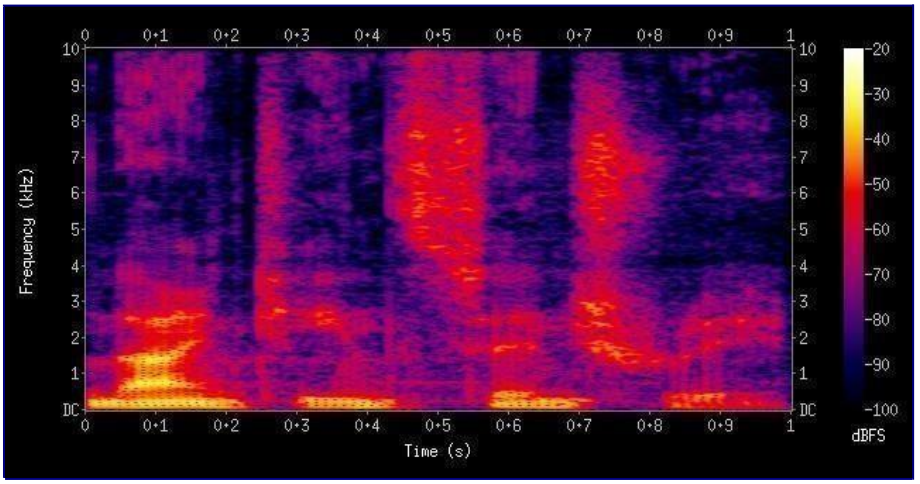
In the frequency domain, the audio signal is represented by amplitude as a function of frequency. Simply put – it is a plot between frequency and amplitude. The features are the amplitudes recorded at different frequencies.



The limitation of this frequency domain analysis is that it completely ignores the order or sequence of the signal which is addressed by time-domain analysis.

Time-domain analysis completely ignores the frequency component whereas frequency domain analysis pays no attention to the time component.

We can get the time-dependent frequencies with the help of a spectrogram.Spectrogram is a 2D plot



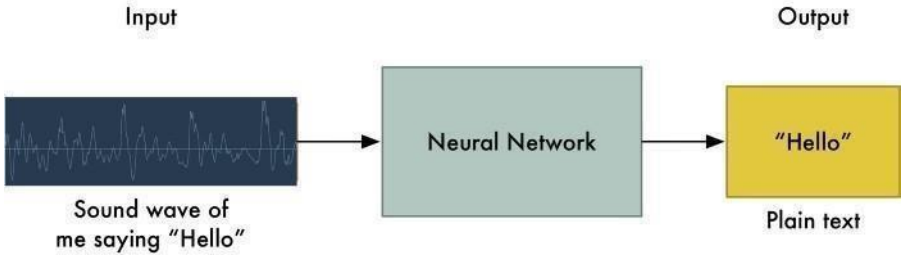
between time and frequency where each point in the plot represents the amplitude of a particular frequency at a particular time in terms of intensity of color. In simple

terms, the spectrogram is a spectrum (broad range of colors) of frequencies as it varies with time.

The right features to extract from audio depends on the use case we are working with. However with the advent of Deep Learning features were learned and extracted automaticaly according to the problemspace.

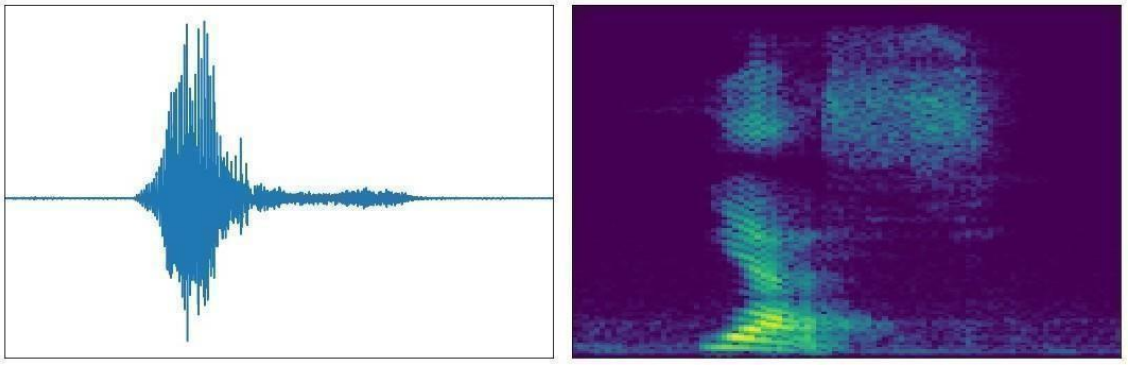
A common strategy for speech recognition is to first extract features from the raw waveform. Commonly used speech features like spectrograms, log-Mel filter banks and Mel-frequency cepstral coefficients (MFCC) convert the raw waveform into a time-frequency domain . These features are then used as an input to a model. Sainath and Paranda (2015) show how log-Mel filter banks can be used as input features to a neural network.

Dai et al. (2017) discuss the challenges with audio-feature-engineering, which requires certain domain knowledge without necessarily building optimal features. Instead of using feature engineering of any kind, this work aims to leverage the power of deep learning to discover features during training from the raw waveform



Single word speech Recognition:

Although single-word speech recognition differs a lot from full scale speech recognition, many of the underlying ideas are the same. Traditional speech recognition systems commonly use Hidden Markov models (HMMs) along with a lot of feature engineering and Gaussian mixture models (GMMs) for acoustic models. The first steps towards using neural networks in speech recognition were using neural networks for acoustic modeling instead of GMMs. (LeCun, Bengio &Hinton 2015) These have since been mostly replaced by end-to-end trained neural architectures such as Deep Speech (Hannun et al. 2014, Amodei et al. 2016).



To build a basic speech recognition network that recognizes ten different words. It's important to know that real speech and audio recognition systems are much more complex, but like MNIST for images, it should give you a basic understanding of the techniques involved. Once you've completed this exercise you'll have a model that tries to classify a one second audio clip as "down", "go", "left",

"no", "right", "stop", "up" and "yes".

Implementation Guide:

Reading audio files and their labels

The audio file will initially be read as a binary file, which you'll want to convert into a numerical tensor. To load an audio file, you will use tf.audio.decode_wav, which returns the WAV- encoded audio as a Tensor and the sample rate.

A WAV file contains time series data with a set number of samples per second. Each sample represents the amplitude of the audio signal at that specific time. In a 16-bit system, like the files in mini_speech_commands, the values range from -32768 to 32767. The sample rate for this dataset is 16kHz**. tf.audio.decode_wav**will normalize the values to the range [-1.0, 1.0].

The label for each WAV file is its parent directory.

Spectrogram:

Convert the waveform into a spectrogram, which shows frequency changes over time and can be represented as a 2D image. This can be done by applying the short-time Fourier transform (STFT) to convert the audio into the time-frequency domain.

A Fourier transform (**tf.signal.fft**) converts a signal to its component frequencies, but loses all time information. The STFT (**tf.signal.stft**) splits the signal into windows of time and runs a Fourier transform on each window, preserving some time information, and returning a 2D tensor that you can run standard convolutions on.

STFT produces an array of complex numbers representing magnitude and phase. However, we only need the magnitude for this excersice, which can be derived by applying **tf.abs** on the output of **tf.signal.stft.**

Choose frame_length and frame_step parameters such that the generated spectrogram "image" is almost square.

You also want the waveforms to have the same length, so that when you convert it to a spectrogram image, the results will have similar dimensions. This can be done by simply zero padding the audio clips that are shorter than one second.

Using the spectrogram input for every class of word we fit the CNN model which has to evaluated using test sample speech word signal data.

Ex no : 9 Date :	Isolated Word speech recognition

AIM:

To build isolated word speech recognition model using CNN on the spech commands dataset and test it with recorded audio which is not from the dataset

About the Dataset
(2018) Speech commands dataset version 2.
Available: http://download.tensorflow.org/data/speech_commands_v0.02.tar.gz

Note: Only the words bed, cat and happy are used in this exercise

Modules used:

Modules	Version
tensorflow	2.6.0
numpy	1.19.5
librosa	0.8.1
matplotlib	3.4.3
ipython	7.26.0

Neural Network Architecture

Layer (type)	Output Shape
Conv2D	(None, 11, 29, 32)
Conv2D	(None, 9, 27, 48)
Conv2D	(None, 6, 24, 64)
MaxPooling2D	(None, 1, 6, 64)
Dropout	(None, 1, 6, 64)
latten	(None, 384)

Layer (type)	Output Shape
Dense	(None, 128)
Dropout	(None, 128)
Dense	(None, 64)
Dropout	(None, 64)
Dense	(None, 3)

```
import numpy as np
import librosa
data_path = "./datasets/isolated_word_dataset"
labels = np.array(["bed","cat","happy"])
n_classes = labels.shape[0]
n_mfcc = 12 # no. of mfc coefficients
t = 30 # no. of time windows on which the mfc coefficients are computed
input_shape = (-1,n_mfcc,t,1)
```

```
def wav2mfcc(file,t,n_mfcc):
    data, sr = librosa.load(file,sr=None)
    mfcc = librosa.feature.mfcc(data,sr,n_mfcc =n_mfcc)
    if mfcc.shape[1]>t:
        mfcc = mfcc[:,:t]
    if mfcc.shape[1]<t:
        mfcc = np.pad(mfcc,pad_width=((0,0),(0,t- mfcc.shape[1])))
    return mfcc
def load_speech_dataset_features(data_path,labels):
    X,y = [],[]
    for i,label in enumerate(labels):
        file_name = f"{data_path}/{label}.npy"
        try:
            data = np.load(file_name)
        except(FileNotFoundError):
            data = np.array([
                wav2mfcc(file,t = t,n_mfcc=n_mfcc)
                for file in librosa.util.find_files(f"{data_path}/{label}/")
            ])
            np.save(file_name,data)
        X.append(data)
        y.append(np.full((data.shape[0],1),i))
    X = np.vstack(X).reshape(input_shape)
    y = np.vstack(y)==np.arange(n_classes) # y is one hot encoded
    return X,y
```

```
X,y = load_speech_dataset_features(data_path,labels)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=.2)
X_train,X_test = X_train,X_test
```

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dropout, Flatten,
Dense
from tensorflow.keras.losses import CategoricalCrossentropy
from tensorflow.keras.optimizers import Adadelta
```

```
def get_model():
    model = Sequential()

    model.add(Conv2D(
        32, kernel_size=(2, 2), activation='relu',
        input_shape=(n_mfcc,t , 1)
    ))
    model.add(Conv2D(48, kernel_size=(3, 3), activation='relu'))
    model.add(Conv2D(64, kernel_size=(4, 4), activation='relu'))
    model.add(MaxPooling2D(pool_size=(4, 4)))
    model.add(Dropout(0.25))

    model.add(Flatten())

    model.add(Dense(128, activation='relu'))
    model.add(Dropout(0.25))
    model.add(Dense(64, activation='relu'))
    model.add(Dropout(0.3))
    model.add(Dense(n_classes, activation='softmax'))
    model.compile(
        loss=CategoricalCrossentropy(),
        optimizer=Adadelta(.3),
        metrics=['accuracy']
    )
    return model
```

```
tf.random.set_seed(0)
model = get_model()
print(model.summary())
```

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 11, 29, 32)        160
```

conv2d_1 (Conv2D)	(None, 9, 27, 48)	13872
conv2d_2 (Conv2D)	(None, 6, 24, 64)	49216
max_pooling2d (MaxPooling2D)	(None, 1, 6, 64)	0
dropout (Dropout)	(None, 1, 6, 64)	0
flatten (Flatten)	(None, 384)	0
dense (Dense)	(None, 128)	49280
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8256
dropout_2 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 3)	195

```
=================================================================
Total params: 120,979
Trainable params: 120,979
Non-trainable params: 0
_________________________________________________________________
None
```

```
model.fit(
    X_train, y_train, batch_size=50, epochs=50,
    verbose=True, validation_data=(X_test, y_test)
)
tf.keras.models.save_model(model,"./models/isolated_word_speech_recognition_mo
del.h5")
model =
tf.keras.models.load_model("./models/isolated_word_speech_recognition_model.h5")
```

```
Epoch 1/50
83/83 [==============================] - 10s 112ms/step - loss: 1.6681 - accuracy:
0.4725 - val_loss: 0.6629 - val_accuracy: 0.7669
Epoch 2/50
83/83 [==============================] - 9s 108ms/step  - loss: 0.7353 - accuracy:
0.6896 - val_loss: 0.4412 - val_accuracy: 0.8487
Epoch 3/50
83/83 [==============================] - 9s 109ms/step  - loss: 0.5424 - accuracy:
0.7945 - val_loss: 0.3388 - val_accuracy: 0.8844
.
.
.
83/83 [==============================] - 9s 110ms/step  - loss: 0.0224 - accuracy:
0.9947 - val_loss: 0.0575 - val_accuracy: 0.9798
Epoch 50/50
83/83 [==============================] - 9s 113ms/step  - loss: 0.0203 - accuracy:
0.9925 - val_loss: 0.0822 - val_accuracy: 0.9778
```

```
from IPython.display import Audio
for label in labels:
    file_name = f"{data_path}/recorded_test_audios/{label}.wav"
    test_audio,sr = librosa.load(file_name,sr=None,mono=True)
    print(f"File Nme: {label}.wav")
    display(Audio(test_audio, rate = sr))
    features = wav2mfcc(file_name,t,n_mfcc).reshape(input_shape)
    print("Predicted Output :", labels[np.argmax(model.predict(features))])
    print("-"*40)
```

output:

```
File Nme: bed.wav
Predicted Output : bed
----------------------------------------
File Nme: cat.wav
Predicted Output : cat
----------------------------------------
File Nme: happy.wav
Predicted Output : happy
----------------------------------------
```

Conclusion:

Thus we learned to implement and evaluate the Speech Recognition Model output via DL keras Framework in Python

10. Face Detection and Tracking

Introduction:

Image or Object Detection is a computer technology that processes the image and detects objects in it.Its a generalization of object localization process. Also It has applications in many areas of computer vision, including image retrieval and video surveillance.

Face detection and tracking:

Face detection only (not recognition) - The goal is to distinguish faces from non-faces (detection is the first step in the recognition process) in an video image frame. Face recognition system identify the person and face verification system verify the person who is claimed to be (by matching it in certain database). Its applications are found in various fields like school, colleges, organisations, factories, public places, surveillance etc. These techniques are gaining momentum worldwide and extended with human emotion recognition, its applications are huge.

Object Detection using Haar feature-based cascade classifiers is an effective object detection method most commonly adopted for simple applications and academic demonstration proposed by Paul Viola and Michael Jones in their paper, “Rapid Object Detection using a Boosted Cascade of Simple Features” in 2001. The key aspect in face recognition is detecting relevant features in human face like eyes, eyebrows, nose, lips for which Haar wavelets of haar features were used through the detection algorithm called as Viola-Jones Algorithm.

Haar features are sequence of rescaled square shape functions proposed by Alfred Haar in 1909. We will apply these haar features to all relevant parts of face so as to detect human face. To detect eyebrow, 2 rectangle Haar feature shown in the fig: is used because forehead and eyebrow form lighter pixels - darker pixel like image. Similarly, to detect lips we use similar to Haar like feature (3 rectangle feature fig.) with lighter-darker-lighter pixels. To detect nose, we might use darker-lighter Harr like feature from (image(1)). And so on. So for this feature extration Viola– Jones requires full view frontal upright faces. Thus in order to be detected, the entire face mustpoint towards the camera and should not be tilted to either side.

The algorithm has four stages:

1. Haar Feature Selection
2. Creating an Integral Image
3. Adaboost Training
4. Cascading Classifiers

There are three key contributions of this algorithm. The first is the introduction of a new image representation called the “Integral Image” which allows the features used by our detector to be computed very quickly. The second is a learning algorithm, based on AdaBoost, which selects a small number of critical visual features and yields extremely efficient classifiers. The third contribution is a method for combining classifiers in a “cascade” which allows background regions of the image to be quickly discarded while spending more computation on promising object-like regions.

Haar Feature selection:

Each feature is a single value obtained by subtracting sum of pixels under white rectangle from sum of pixels under black rectangle.

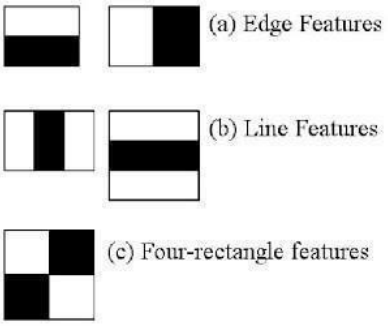
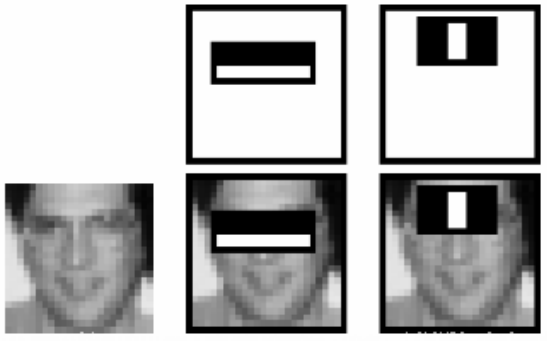
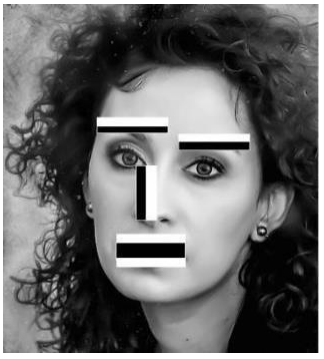


Fig.1. Haar Feature

Haar Feature Matching:





$$\Delta = dark - white = \frac{1}{n}\sum_{dark}^{n} I(x) - \frac{1}{n}\sum_{white}^{n} I(x)$$

0	0	1	1
0	0	1	1
0	0	1	1
0	0	1	1

for Instance:
Ideal case : Delta = (1/8)*(8) — (1/8)*0 = 1

Real case: Delta = (1/8)*(5.9) — (1/8)*(1.3)=0.575

0.1	0.2	0.6	0.8
0.3	0.2	0.6	0.8
0.2	0.1	0.8	0.6
0.2	0.1	0.8	0.9

(For greyscale image, assume we have set White-Dark threshold to 0.3. Meaning pixels with value less than or equal to 0.3 are considered white and anything greater that 0.3 is considered as dark)

According to Viola-Jonas algorithm, to detect Haar like feature present in an image, above formula should give result closer to 1. The closer the value is to 1, the greater the chance of detecting Haar feature in image.

Now all possible sizes and locations of each kernel is used to calculate plenty of features. (Just imagine how much computation it needs? Even a 24x24 window results over 160000 features). For each feature calculation, we need to find sum of pixels under white and black rectangles. To solve this, they introduced the integral images. It simplifies calculation of sum of pixels, how large may be the number of pixels, to an operation involving just four pixels. It makes things computation faster.

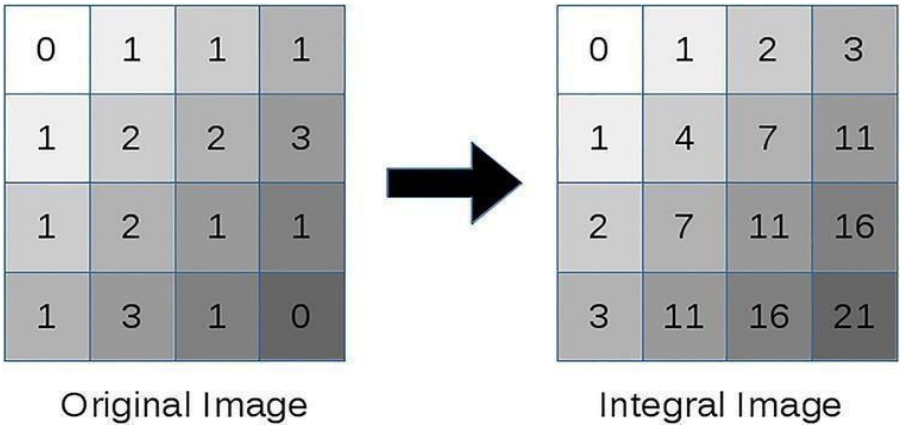
Integral Images

To calculate a value for each feature, we need to perform computations on all the pixels inside that particular feature. In reality, these calculations can be very intensive since the number of pixels would be much greater when we are dealing with a large feature.

The integral image plays its part in allowing us to perform these intensive calculations quickly so we can understand whether a feature of several features fit the criteria.

An integral image (also known as a summed-area table) is the name of both a data structure and an algorithm used to obtain this data structure. It is used as a quick and efficient way to calculate the sum of pixel values in an image or rectangular part of an image.

In an integral image, the value of each point is the sum of all pixels above and to the left, including the target pixel:



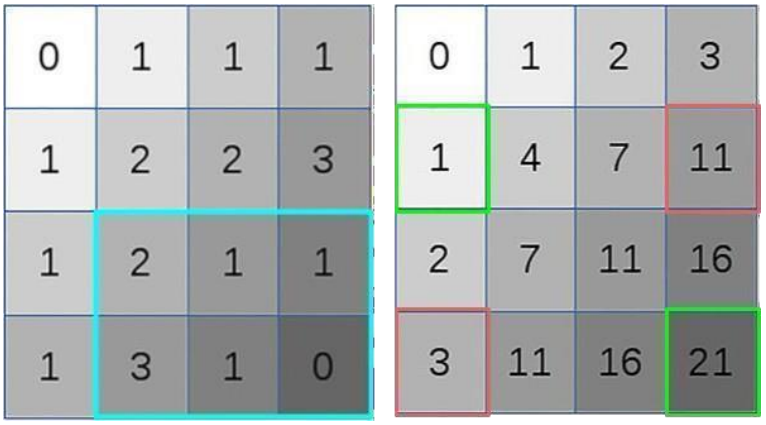
Using these integral images, we save a lot of time calculating the summation of all the pixels in a rectangle as we only have to perform calculations on four edges of the rectangle. See the example below to understand.

When we add the pixels in the blue box, we get 8 as the sum of all pixels and here we had six elements involved in your calculation. Now to calculate the sum of these same pixels using the integral image, you just need to find the corners of the rectangle and then add the vertices which aregreen and subtract the vertices in the red boxes. Now doing that here

```
21+1 - 11 -3 =8
```

We get the same answer and only four numbers are involved in calculations. No matter how many pixels are in the rectangle box, we will just need to compute on these 4 vertices.

To calculate the value of any haar-like feature, we have a simple way to calculate the difference between the sums of pixel values of two rectangles.



But among all these features we have calculated most of them are irrelevant. For example, consider the image below. Top row shows two good features. The first feature selected seems to focus on the property that the region of the eyes is often darker than the region of the nose and cheeks. The second feature selected relies on the property that the eyes are darker than the bridge ofthe nose. But the same windows applying on cheeks or any other place is irrelevant. So for selectingthe best features out of 160000+ features **Adaboost** is deployed.

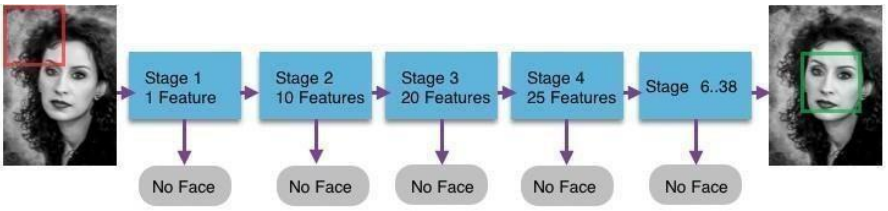
For this, we apply each and every feature on all the training images. For each feature, it finds the best threshold which will classify the faces to positive and negative, But obviously, there will be errors or misclassifications. We select the features with minimum error rate, which means they are the features that best classifies the face and non-face images. (The process is not as simple as this. Each image is given an equal weight in the beginning. After each classification, weights of misclassified images are increased. Then again same process is done, New error rates are calculated.Also new weights. The process is continued until required accuracy or error rate is achieved or required number of features are found).

Final classifier is a weighted sum of these weak classifiers. It is called weak because it alone can't classify the image, but together with others forms a strong classifier. The viola jones research manuscript says even 200 features provide detection with 95% accuracy. Their final setup had around 6000 features. (a reduction from 160000+ features to 6000 features).

So in an image take each 24x24 window. Apply 6000 features to it to Check if it is face or not. But it's a little inefficient and time consuming so domain experts (viola-jones) comeup with a solution in which, a simple method to check if a window is not a face region. If it is not, discard it in a single shot. Don't process it again. Instead focus on region where there can be a face. This way, we can

find more time to check a possible face region. Since most of the image region is non-face region. So it is a better idea to discard these region without processing.

"Instead of applying all the 6000 features on a window, group the features into different stages of classifiers and apply one-by-one. (Normally first few stages will contain very less number of features). If a window fails the first stage, discard it. We don't consider remaining features on it. If it passes, apply the second stage of features and continue the process. The window which passes all stages is a face region." - Face Detection using Haar Cascades.



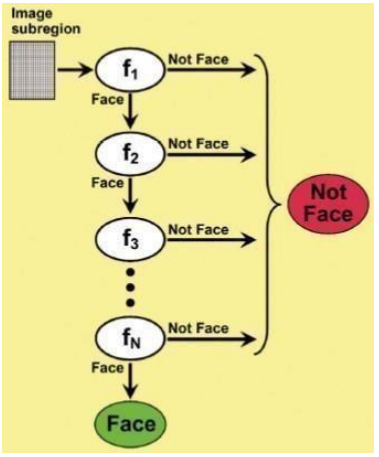


Fig.1. Cascaded Classifiers

It is a machine learning based approach in which a cascade function is trained from a lot of positive and negative images, then it is used to detect objects in other images, this concept is know as **Cascade of Classifiers**.

They are series of classifiers or features (as we have seen above) used to identify object in an image. Using sliding windows and number of haar features (increases as number of stages increase), finally leading to detect face or not. There are total 38 stages defined for Viola Jonas Method, Depending upon the sliding windows size and face location, number of features, face can be detected at a certain stage.

Instead of applying all the 6000 features on a window, group the features into different stages of classifiers and apply one-by-one. (Normally first few stages will contain very less number of

features). If a window fails the first stage, discard it. We don't consider remaining features on it. If it passes, apply the second stage of features and continue the process. The window which passes all stages is a face region.

Viola jone's detector had 6000+ features with 38 stages with 1, 10, 25, 25 and 50 features in first five stages. (Two features in the above image is actually obtained as the best two features from Adaboost). According to authors, on an average, 10 features out of 6000+ are evaluated per sub- window.

Haar-cascade Detection in OpenCV

Face detection using Haar cascades is a machine learning based approach where a cascade function is trained with a set of input data. OpenCV already contains many pre-trained classifiers for face, eyes, smiles, etc.. In our experiment we will use the face classifier.

OpenCV comes with a trainer as well as detector. If you want to train your own classifier for any object like car, planes etc. you can use OpenCV to create one. i.e Cascade Classifier Training. These classifiers as XML files are stored in `opencv/data/haarcascades/` folder.

First we need to load the required XML classifiers. Then load our input image (or video) in grayscale mode.

```
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
```

Then we find the faces in the image. If faces are found, it returns the positions of detected faces as Rect(x,y,w,h).

Once we get these locations, we can create a ROI for the face.

Ex no : 10
Date :

Face Detection and Tracking using OpenCV

AIM:
To Implement Face Detection and Tracking using OpenCV

MODULES REQUIRED:

Module	version
opencv-python	4.5.1.48

```
import cv2
face_cascade = cv2.CascadeClassifier('models/haarcascade_frontalface_default.xml')
```

```
cap = cv2.VideoCapture('datasets/CERN_Higgs boson_EDIT.mp4')
while True:
    ok, img = cap.read()
    if not ok:
        print('Video Ending')
        cap.release()
        cv2.destroyAllWindows()
        break
    # Convert to grayscale
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # Detect the faces
    faces = face_cascade.detectMultiScale(gray, 1.1,4)
    # Draw the rectangle around each face
    for (x, y, w, h) in faces:
        cv2.rectangle(img, (x, y), (x+w, y+h),(0,0,255))
    # Display
    cv2.imshow('img', img)
    # quit Press Key Q to quit and Close window
    if cv2.waitKey(1) & 0xFF == ord('q'):
        # Release the VideoCapture object
        cap.release()
        # Close all window
        cv2.destroyAllWindows()
        break
```

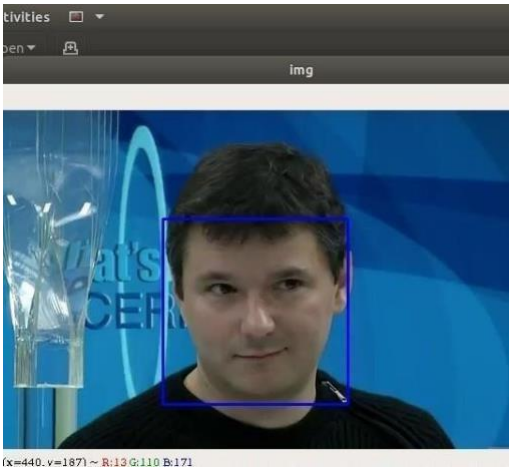
Output:

Sample Video Contains Bounding Box around the Face Object













Conclusion:

Thus we learned to implement the Face detection and tracking 'ViolaJones'- algorithm via Open-cv Python.

11. Object Recognition

Introduction:

Object recognition is a computer vision technique for identifying objects in images or videos. Object recognition is a key output of machine learning algorithms. When humans look at a photograph or watch a video, we can readily spot people, objects, scenes, and visual details. The goal is to teach a computer to do what comes naturally to humans: to gain a level of understanding of what an image contains. This technology has been time tested also the work horse behind driverless cars, enabling them to recognize a stop sign or to distinguish a pedestrian from alamppost. It is also useful in a variety of applications such as disease identification in bio-imaging, industrial inspection, and robotic vision.

An object recognition system finds objects in the real world from an image of the world, using object models which are known a priori. This task is surprisingly difficult. Humans perform object recognition effortlessly and instantaneously. Algorithmic description of this task for implementation on machines has been very difficult. In another aspect in machine learning technique object recognition problem can be defined as a labeling problem based on models of known objects. Formally, given an image containing one or more objects of interest (and background) and a set of labels corresponding to a set of models known to the system, the system should assign correct labels to regions, or a set of regions, in the image.

Object detection and object recognition are similar techniques for identifying objects, but they vary in their execution. Object detection is the process of finding instances of objects in images. In the case of deep learning, object detection is a subset of object recognition, where the object is not only identified but also located in an image. This allows for multiple objects to be identified and located within the same image. In classical approach Object recognition is closelytied to the segmentation problem: without at least a partial recognition of objects, segmentation cannot be done, and without segmentation, object recognition is not possible with the advent ofdeep learning this dependency of segmentation has eliminated.

Classical approach vs Modern Paradigm

The traditional approach is to use well-established Computer Vision (CV) techniques such as feature descriptors (SIFT, SURF, BRIEF, etc.) for object detection. Before the emergence of Deep Learning (DL), a step called feature extraction was carried out for tasks such as image classification. Features are small “interesting”, descriptive or informative patches in images. SeveralCV algorithms, such as edge detection, corner detection or threshold segmentation may be involved in this step. As many features as practicable are extracted from images and these features form a definition (known as a bag-of-words) of each object class. At the deployment stage, these definitions are searched for in other images. If a significant number of features from one bag-of- words are in another image, the image is classified as containing that specific object (i.e. chair, horse, etc.).

Difficulty with the traditional approach is that it is necessary to choose which features are important in each given image. As the number of classes to classify increases, feature extraction becomes more and more cumbersome. It is up to the CV engineer's judgment and a long trial and error process to decide which features best describe different classes of objects. Moreover, each feature definition requires dealing with a plethora of parameters, all of which must be fine-tuned by the CV engineer.

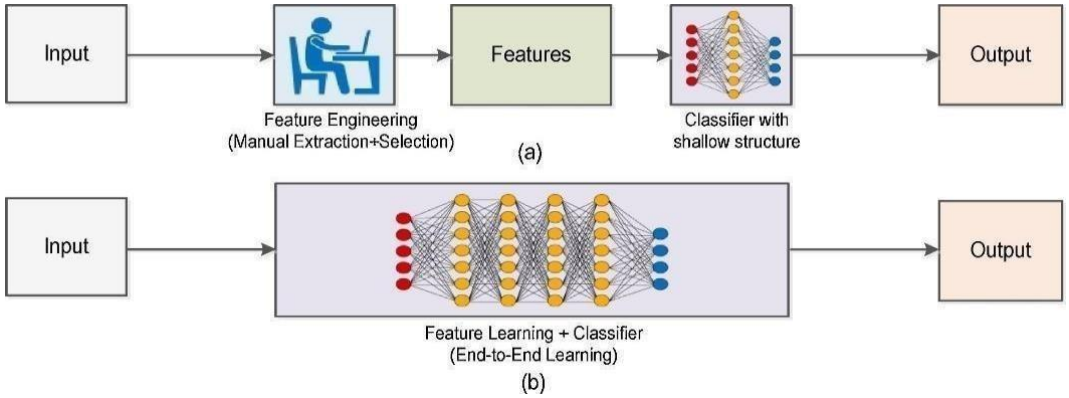


Fig.1. Comparative block diagram of

(a) Classical Machine Learning paradigm and

(b) Modern Deep Learning Paradigm

The traditional feature-based approaches such as those listed below have been shown to be useful in improving performance in CV tasks:

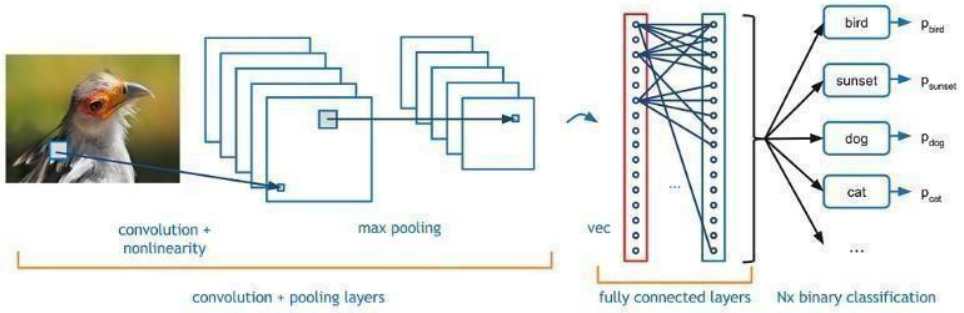
- Hough transforms
- Scale Invariant Feature Transform (SIFT)
- Speeded Up Robust Features (SURF)
- HoG Histogram of Gradient

Feature descriptors such as SIFT and SURF are generally combined with traditional machine learning classification algorithms such as Support Vector Machines and K-Nearest Neighbours to solve the object recognition sCV problems

Deep Learning for Object Recognition/Classification Problem:

DL introduced the concept of end-to-end learning where the machine is just given a dataset of images which have been annotated with what classes of object are present in each image. Thereby a DL model is 'trained' on the given data, where neural networks discover the underlying patterns in classes of images and automatically works out the most descriptive and salient features perform far better than traditional algorithms, albeit with trade-offs with respect to computing requirements.

Fig.2. CNN



The development of CNNs has had a tremendous influence in the field of CV in recent years and is responsible for a big jump in the ability to recognize objects. This burst in progress has been enabled by an increase in computing power, as well as an increase in the amount of data available for training neural networks.

CNNs make use of kernels (also known as filters), to detect features (e.g. edges) throughout an image. A kernel is just a matrix of values, called weights, which are trained to detect specific features. As their name indicates, the main idea behind the CNNs is to spatially convolve the kernel on a given input image check if the feature it is meant to detect is present. To provide a value representing how confident it is that a specific feature is present, a convolution operation is carried out by computing the dot product of the kernel and the input area where kernel is overlapped (the area of the original image the kernel is looking at is known as the receptive field).

To facilitate the learning of kernel weights, the convolution layer's output is summed with a bias term and then fed to a non-linear activation function. Activation Functions are usually non-linear functions like Sigmoid, tanh and ReLU (Rectified Linear Unit). Depending on the nature of data and classification tasks, these activation functions are selected accordingly. For example, ReLUs are known to have more biological representation (neurons in the brain either fire or they don't). As a result, it yields favourable results for image recognition tasks as it is less susceptible to the vanishing gradient problem and it produces sparser, more efficient representations.

To speed up the training process and reduce the amount of memory consumed by the network, the convolutional layer is often followed by a pooling layer to remove redundancy present in the input feature. For example, max pooling moves a window over the input and simply outputs the maximum value in that window effectively reducing to the important pixels in an image. As shown in Fig. 2, deep CNNs may have several pairs of convolutional and pooling layers . Finally, a Fully Connected layer flattens the previous layer volume into a feature vector and then an output layer which computes the scores (confidence or probabilities) for the output classes/features through a dense network. This output is then passed to a regression function such as Softmax, for example, which maps everything to a vector whose elements sum up to one.

In our excercise we deploy CNN via Deep Learning Keras Framework for Demonstrating object recognition problem.

Ex no : 11
Date :

Object Recognition using CNN

AIM:

To implement Object Recognition in Cidataset using CNN.

Modules used:

Modules	Version
tensorflow	2.6.0
numpy	1.19.5
pandas	1.3.0
matplotlib	3.4.3
joblib	1.0.1

Neural Network Architecture:

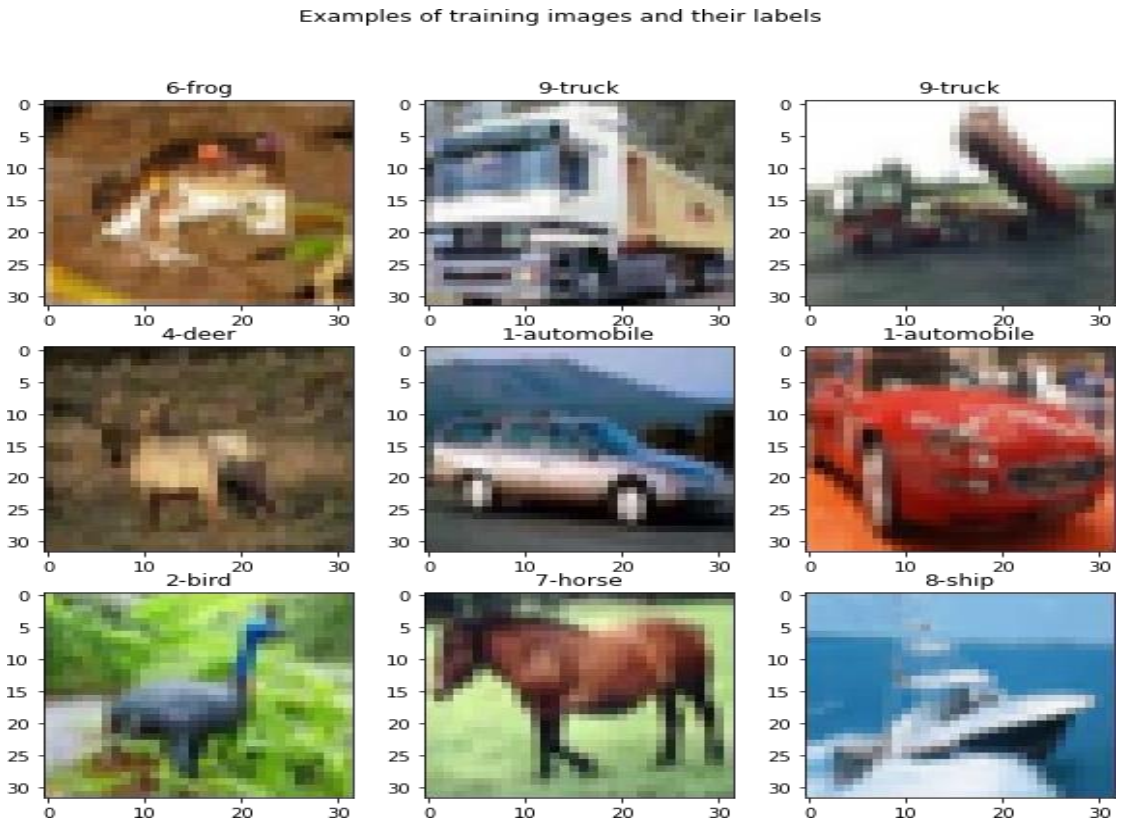
Layer (type)	Output Shape
Conv2D	(None, 32, 32, 25)
Conv2D(1)	(None, 32, 32, 50)
MaxPooling2D	(None, 16, 16, 50)
Dropout	(None, 16, 16, 50)
Conv2D(2)	(None, 16, 16, 70)
MaxPooling2D(1)	(None, 8, 8, 70)
Dropout(1)	(None, 8, 8, 70)
Flatten	(None, 4480)
Dense	(None, 500)
Dropout(2)	(None, 500)
Dense(1)	(None, 250)
Dropout(3)	(None, 250)
Dense(2)	(None, 10)

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
from tensorflow.keras import Sequential
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.layers import Dense, Dropout, Conv2D, MaxPool2D, Flatten
from tensorflow.keras.callbacks import Callback, ModelCheckpoint, EarlyStopping
from tensorflow import random
import joblib
import pandas as pd
```

```
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
cifar_classes =np.array( ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog',
'frog','horse', 'ship', 'truck'])
```

```
plt.figure(figsize=(10,10))
plt.suptitle('Examples of training images and their labels')
for i in range(9):
    plt.subplot(331 + i,title=f'{y_train[i,0]}-{cifar_classes[y_train[i,0]]}')
    plt.imshow(X_train[i])
plt.show()
```

output:




```
y_train = to_categorical(y_train, num_classes=10)
y_test = to_categorical(y_test, num_classes=10)
X_train = X_train.astype('float32')/255
X_test = X_test.astype('float32')/255
print(f"Shape of training data:\nX-train_shape={X_train.shape} , y-
train_shape={y_train.shape}")
print(f"Shape of testing data:\nX-test_shape={X_test.shape} , y-
test_shape={y_test.shape}")
Shape of training data:
X-train_shape=(50000, 32, 32, 3) , y-train_shape=(50000, 10)
Shape of testing data:
X-test_shape=(10000, 32, 32, 3) , y-test_shape=(10000, 10)
```

```
def get_model():
    model = Sequential()
    # convolutional layer
    model.add(Conv2D(25, kernel_size=(3,3), strides=(1,1), padding='same',
activation='relu', input_shape=(32, 32, 3)))
```

```
# convolutional layer
    model.add(Conv2D(50, kernel_size=(3,3), strides=(1,1), padding='same',
activation='relu'))
    model.add(MaxPool2D(pool_size=(2,2)))
    model.add(Dropout(0.25))
    model.add(Conv2D(70, kernel_size=(3,3), strides=(1,1), padding='same',
activation='relu'))
    model.add(MaxPool2D(pool_size=(2,2)))
    model.add(Dropout(0.25))
    # flatten output of conv
    model.add(Flatten())
    # hidden layer
    model.add(Dense(500, activation='relu'))
    model.add(Dropout(0.4))
    model.add(Dense(250, activation='relu'))
    model.add(Dropout(0.3))
    # output layer
    model.add(Dense(10,    activation='softmax'))
    # Compiling the Model
    model.compile(loss='categorical_crossentropy',
metrics=['accuracy'],optimizer='adam')
    return model
```

```
random.set_seed(0)
model=get_model()
print(model.summary())
```

output:

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 32, 32, 25)        700
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 32, 32, 50)        11300
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 16, 16, 50)        0
_________________________________________________________________
dropout (Dropout)            (None, 16, 16, 50)        0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 16, 16, 70)        31570
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 8, 8, 70)          0
_________________________________________________________________
dropout_1 (Dropout)          (None, 8, 8, 70)          0
```

flatten (Flatten)	(None, 4480)	0
dense (Dense)	(None, 500)	2240500
dropout_2 (Dropout)	(None, 500)	0
dense_1 (Dense)	(None, 250)	125250
dropout_3 (Dropout)	(None, 250)	0
dense_2 (Dense)	(None, 10)	2510

```
=================================================================
Total params: 2,411,830
Trainable params: 2,411,830
Non-trainable params: 0
_________________________________________________________________
None
```

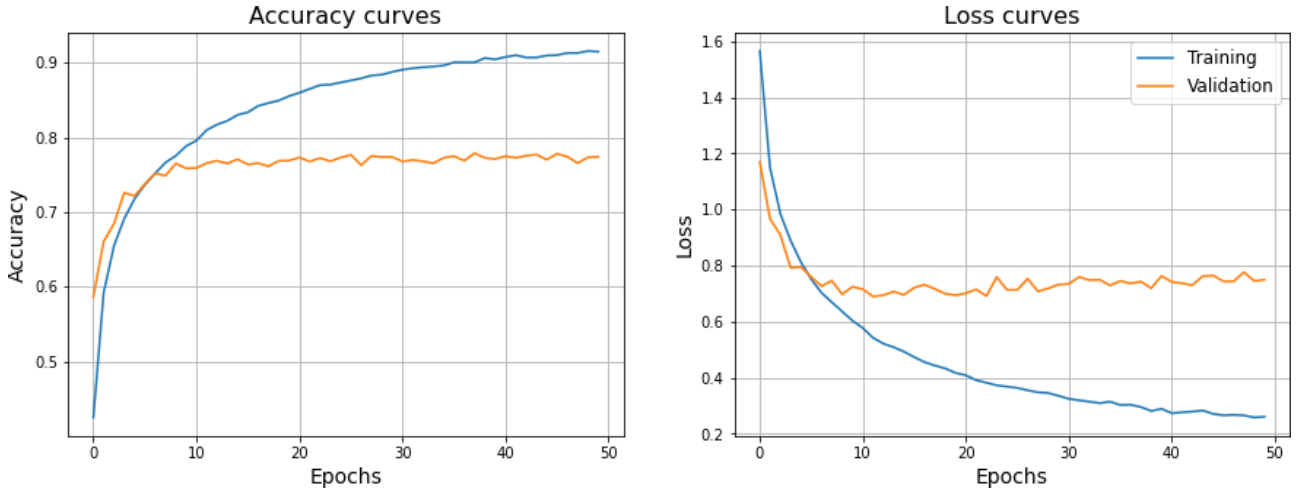
```
checkpoint=
ModelCheckpoint("./model/cifar10_epo_cnn.h5",monitor='val_accuracy',mode='
ma x', save_best_only=True)
early_stop = EarlyStopping(monitor="val_accuracy", patience=5, mode="max")
call_backs=[early_stopcheckpoint]
history = model.fit(X_train, y_train, batch_size=32,
epochs=50, verbose=True,validation_split=0.2)
model.save("./models/cifar10_epo_cnn.h5")
joblib.dump(history.history,
```

output:

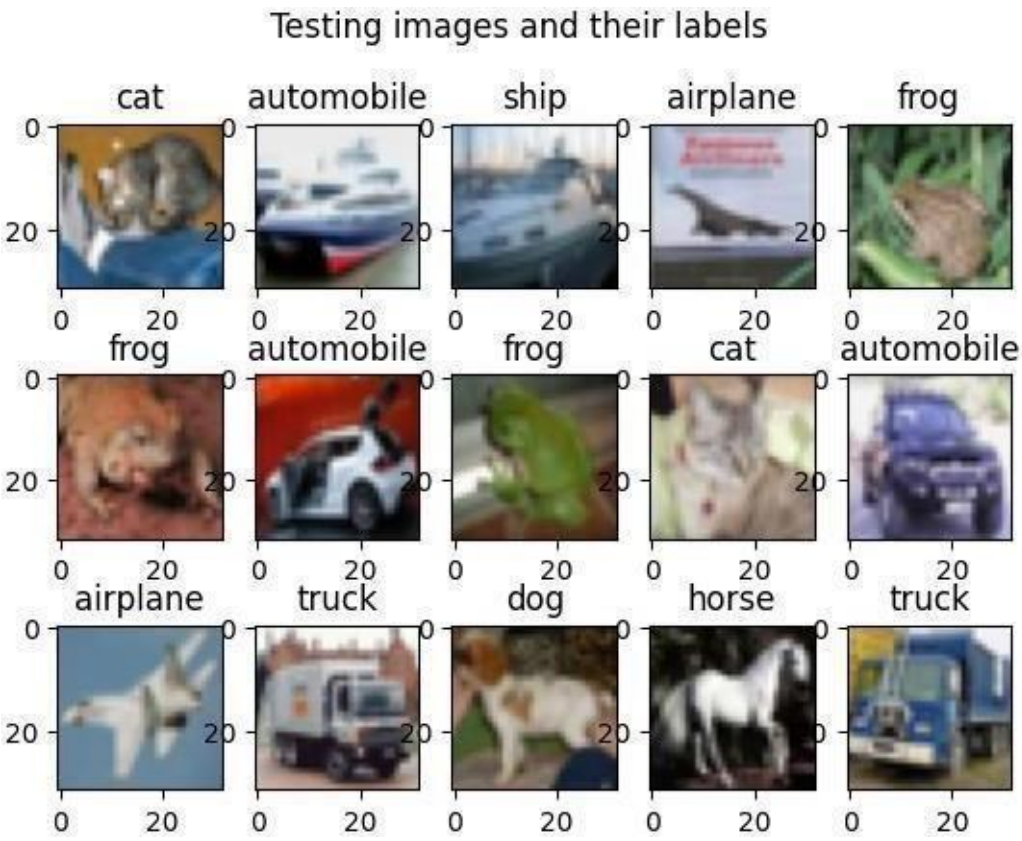
```
Epoch 1/50
1250/1250 [==============================] - 34s 10ms/step - loss: 1.5650 - accur
acy: 0.4250 - val_loss: 1.1693 - val_accuracy: 0.5866
.
.
1250/1250 [==============================] - 11s 9ms/step - loss: 0.2650 - accura
cy: 0.9128 - val_loss: 0.7762 - val_accuracy: 0.7655
Epoch 50/50
1250/1250 [==============================] - 11s 9ms/step - loss: 0.2603 - accura
cy: 0.9148 - val_loss: 0.7488 - val_accuracy: 0.7740
['./models/cifar10_epo_cnn.history']
```

```
# Use only the below lines if model is not re trained
from keras.models import load_model
model = load_model('./models/cifar10_epo_cnn.h5')
history = joblib.load("./models/cifar10_epo_cnn.history")
```

```
fig,ax=plt.subplots(1,2,figsize=(15,5))
# plotting Accuracy curves
ax[0].plot(history['accuracy'],'C0')
ax[0].plot(history['val_accuracy'],'C1')
ax[0].set_title(label="Accuracy curves",fontsize=16)
ax[0].set_xlabel('Epochs',fontsize=14)
ax[0].set_ylabel('Accuracy',fontsize=14)
ax[0].grid()
# plotting Loss curves
ax[1].plot(history['loss'],'C0')
ax[1].plot(history['val_loss'],'C1')
ax[1].set_title(label="Loss curves",fontsize=16)
ax[1].set_xlabel('Epochs',fontsize=14)
ax[1].set_ylabel('Loss',fontsize=14)
ax[1].grid()
plt.legend(['Training','Validation'],fontsize=12)
plt.show()
```




```
m=15
y_pred_=model.predict(X_test[:m])
y_pred_class=y_pred_.argmax(-1)
unique,counts=np.unique(y_pred_class,return_counts=True)
plt.suptitle('Testing images and their labels')
for i in range(15):
    plt.subplot(3,5,i+1,title=f'{cifar_classes[y_pred_class[i]]}')
    plt.imshow(X_test[i])
plt.show()
freq=list(zip(cifar_classes[unique],counts))
pd.DataFrame(freq,columns=["class name","count"])
```



	class name	count
0	airplane	2
1	automobile	3
2	cat	2
3	dog	1
4	frog	3
5	horse	1
6	ship	1
7	truck	2

Conclusion

Thus we learned to build fit CNN object Recognition MODEL using CIFAR-10 Dataset via Python.